

CS 189/289

Today's lecture outline

1. Finish backprop
2. More on Neural Networks
3. Convolutional Neural Networks

Assigned reading:

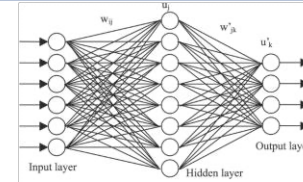
7.2.5 (NN initialization);
7.4 up to including 7.4.2
(data normalization; batch
norm)
10 up to including 10.2.8,
10.3.2 (CNNs)

Overall set-up is same as before

- Training Data: $S = \{(x_i, y_i)\}_{i=1}^N$ $x \in R^D$
 $y \in \{-1, +1\}$

- Model Class:

$$f(x|w) =$$



Composition of functions

- Loss Function:

$$LL(w) = \sum_i \log p(y_i | x_i, w)$$

Log Likelihood

- Learning Objective:

$$\hat{w} = \operatorname{argmin}_w \{-LL(w)\}$$

Optimization Problem

gradient descent



backprop!

```
w(0) = Initialize()
for τ in range(1, τmax):
    w(τ) = w(τ-1) - η ∇J(w(τ-1))
    if Converged(w(τ), w(τ-1), ε): terminate
```

$$J(w) = -LL(w)$$

Systematizing derivatives with computation graphs

General backpropagation algorithm:

Let $v_1, \dots, v_N$ be a **topological ordering** of the computation graph (i.e. parents come before children.)

v_N denotes the variable we're trying to compute derivatives of (e.g. loss).

Computing the loss:

$$v_3 = z = wx + b$$

$$v_4 = y = \sigma(z)$$

$$v_5 = \mathcal{L} = \frac{1}{2}(y - t)^2$$

forward pass

For $i = 1, \dots, N$

Compute v_i as a function of $\text{Pa}(v_i)$

Computing the derivatives:

$$\overline{v_4} = \overline{y} = y - t$$

$$\overline{v_3} = \overline{z} = \overline{y} \cdot \sigma'(z)$$

$$\overline{v_2} = \overline{w} = \overline{z} \cdot x$$

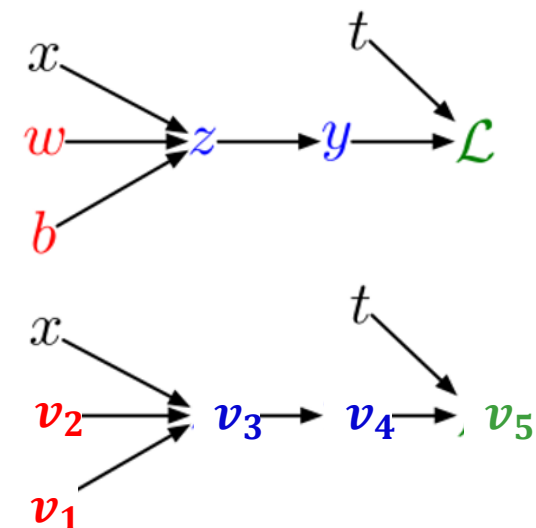
$$\overline{v_1} = \overline{b} = \overline{z}$$

backward pass

$$\overline{v_N} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1$$

For $i = N - 1, \dots, 1$

$$\overline{v_i} = \sum_{j \in \text{Ch}(v_i)} \overline{v_j} \frac{\partial v_j}{\partial v_i}$$



Computing the derivatives:

$$\overline{v_4} = \overline{v_5} \frac{\partial v_5}{\partial v_4}$$

$$\overline{v_3} = \overline{v_4} \frac{\partial v_4}{\partial v_3}$$

$$\overline{v_2} = \overline{v_3} \frac{\partial v_3}{\partial v_2}$$

$$\overline{v_1} = \overline{v_3} \frac{\partial v_3}{\partial v_1}$$

$$\frac{d\mathcal{L}}{dy} = y - t$$

$$\frac{d\mathcal{L}}{dz} = \frac{d\mathcal{L}}{dy} \cdot \sigma'(z)$$

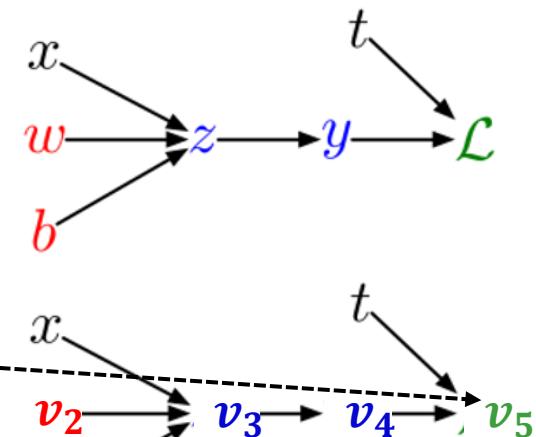
$$\frac{\partial \mathcal{L}}{\partial w} = \frac{d\mathcal{L}}{dz} \cdot x$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{d\mathcal{L}}{dz}$$

Systematizing derivatives with computation graphs

So why do I need the forward pass?

- to compute derivatives
- to track the loss during training



Computing the loss:

$$\begin{aligned} v_3 &= z = wx + b \\ v_4 &= y = \sigma(z) \\ v_5 &= \mathcal{L} = \frac{1}{2}(y - t)^2 \end{aligned}$$

forward pass

For $i = 1, \dots, N$

Compute v_i as a function of $\text{Pa}(v_i)$

Computing the derivatives:

$$\begin{aligned} \bar{v}_4 &= \bar{y} = y - t \\ \bar{v}_3 &= \bar{z} = \bar{y} \cdot \sigma'(z) \\ \bar{v}_2 &= \bar{w} = \bar{z} \cdot x \\ \bar{v}_1 &= \bar{b} = \bar{z} \end{aligned}$$

backward pass

$$\bar{v}_N = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1$$

For $i = N - 1, \dots, 1$

$$\bar{v}_i = \sum_{j \in \text{Ch}(v_i)} \bar{v}_j \frac{\partial v_j}{\partial v_i}$$

Computing the derivatives:

$$\begin{aligned} \bar{v}_4 &= \bar{v}_5 \frac{\partial v_5}{\partial v_4} \\ \bar{v}_3 &= \bar{v}_4 \frac{\partial v_4}{\partial v_3} \\ \bar{v}_2 &= \bar{v}_3 \frac{\partial v_3}{\partial v_2} \\ \bar{v}_1 &= \bar{v}_3 \frac{\partial v_3}{\partial v_1} \end{aligned}$$

$$\begin{aligned} \frac{d\mathcal{L}}{dy} &= y - t \\ \frac{d\mathcal{L}}{dz} &= \frac{d\mathcal{L}}{dy} \cdot \sigma'(z) \\ \frac{\partial \mathcal{L}}{\partial w} &= \frac{d\mathcal{L}}{dz} \cdot x \\ \frac{\partial \mathcal{L}}{\partial b} &= \frac{d\mathcal{L}}{dz} \end{aligned}$$

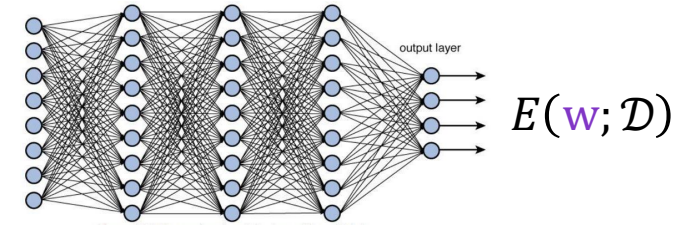
Could we use finite difference instead?

Computing the **gradient** using numerical differentiation (2 calls to loss function):

$$\begin{array}{l} \mathbf{w} \in \mathbb{R}^L \\ w_i \in \mathbb{R} \end{array} \quad \frac{\partial}{\partial w_i} E(\mathbf{w}; \mathcal{D}) \approx \frac{E(\mathbf{w} + \epsilon \mathbf{I}_i; \mathcal{D}) - E(\mathbf{w} - \epsilon \mathbf{I}_i; \mathcal{D})}{2\epsilon}$$

For a L dimensional weight vector $\mathbf{w} \in \mathbb{R}^L$ requires:

$$\left[\frac{\partial}{\partial w_1} E(\mathbf{w}; \mathcal{D}), \dots, \frac{\partial}{\partial w_L} E(\mathbf{w}; \mathcal{D}) \right]$$



$2L$ loss function calculations. Each **loss calculation** requires $\mathcal{O}(NL)$ computation for N datapoints, each has to do one forward pass touching all L parameters.

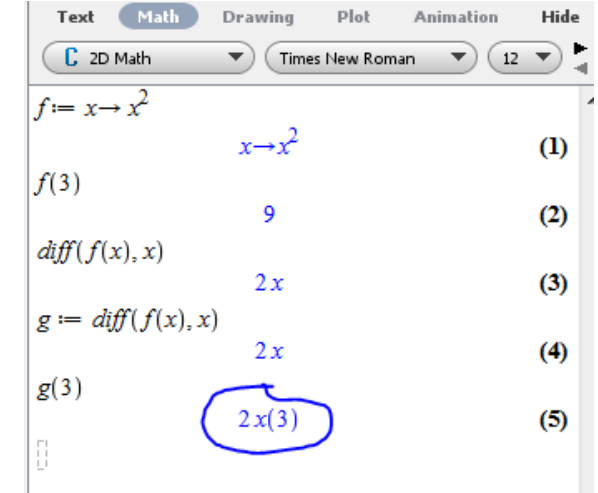
Total cost: $\mathcal{O}(NL^2)$ per gradient step! (**Quadratic in # of parameters**)

- For real problems (minibatch) often $N \approx 1000$ and $L \gg 10^6 \rightarrow NL^2 \gg 10^{3+2*6} = 10^{15}$
- Stochastic gradient with batch size 1: $L^2 \gg 10^{12}$

Could we use symbolic algebra systems?

Computer algebra systems like SymPy can be used to automate the differentiation process.

- Express functions **symbolically**; automatically construct **derivative expressions**



The screenshot shows a software interface with tabs for Text, Math, Drawing, Plot, Animation, and Hide. The Math tab is active, showing a sequence of operations: $f := x \rightarrow x^2$, $f(3)$, $\text{diff}(f(x), x)$, $g := \text{diff}(f(x), x)$, and $g(3)$. To the right of each operation is its result: $x \rightarrow x^2$, 9, $2x$, $2x$, and $2x(3)$. The results are numbered (1) through (5). The expression $2x(3)$ is circled in blue.

Disadvantages:

- **Inefficient:** algebraic derivative expressions could be significantly larger and contain many repeated expressions: “expression swell”.

$$f(x) = u(x)v(x) \quad (8.42)$$

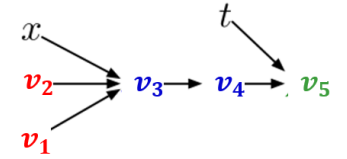
$$f'(x) = u'(x)v(x) + u(x)v'(x). \quad (8.43)$$

We see that there is redundant computation in that $u(x)$ and $v(x)$ must be evaluated both for the calculation of $f(x)$ and for $f'(x)$. If the factors $u(x)$ and $v(x)$ themselves involve factors, then we end up with a nested duplication of expressions, which rapidly grow in complexity. This problem is called *expression swell*.

From backpropagation to auto-differentiation

Back in the day, we used to **apply backpropagation** by:

- i. Figuring the computation graph by hand.
- ii. Coding the forward pass by hand
- iii. Writing down the (“local”) **derivatives by hand**.
- iv. Coding up the derivatives.
- v. Checking that backprop matched finite difference.



$$\overline{v_N} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1$$

For $i = N - 1, \dots, 1$

$$\overline{v_i} = \sum_{j \in \text{Ch}(v_i)} \overline{v_j} \frac{\partial v_j}{\partial v_i}$$

Today, we **apply backpropagation automatically** using “autodiff”:
we specify only the forward network, and then the autodiff engine:

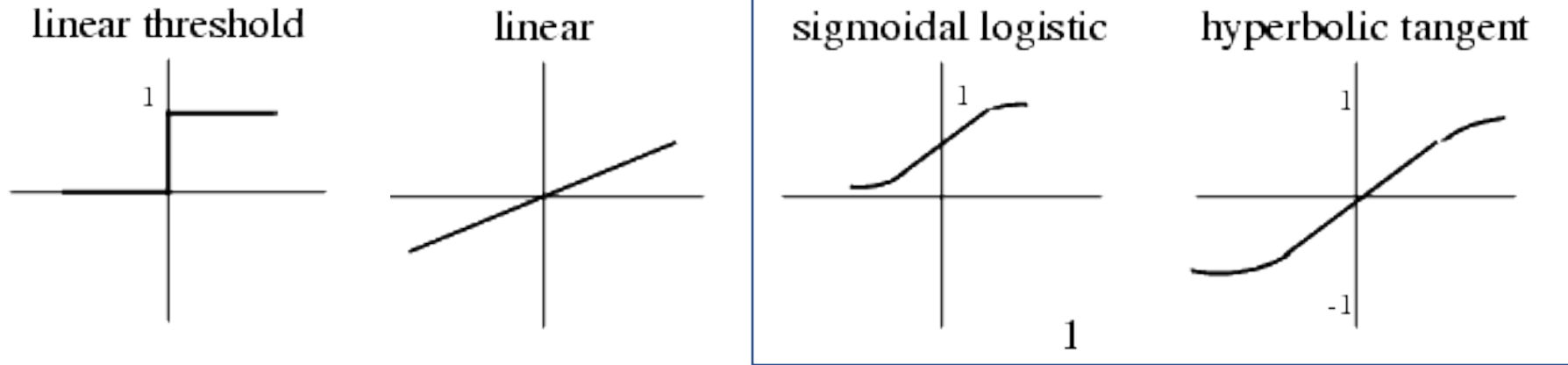
- i. Figures out the computation graph.
- ii. Calculates all needed derivatives.
- iii. Applies the backpropagation algorithm.

CS 189/289

Today's lecture outline

1. Finish backprop
2. More on Neural Networks
3. Convolutional Neural Networks

Back to activation functions

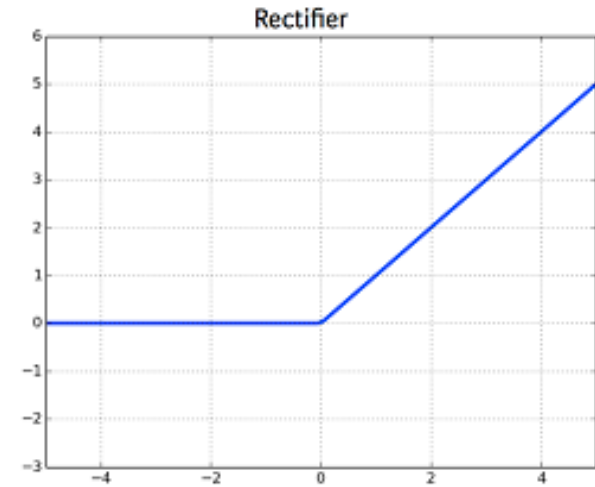


- What's the common issue wrt [sigmoidal/hyperbolic tangent](#) activations and gradient descent?
- These functions have asymptotes at the top and bottom, and thus have zero gradient there. Easy to get "stuck" with zero gradients—can become "dead units".

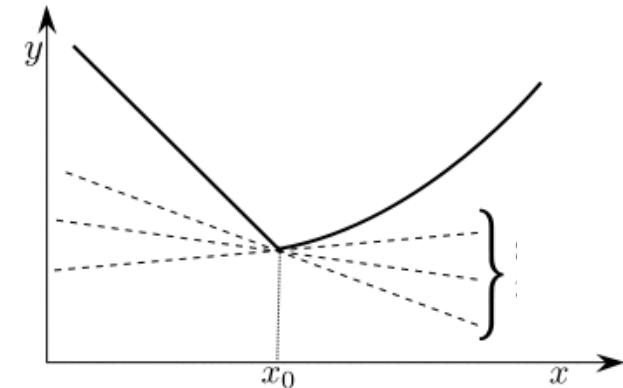
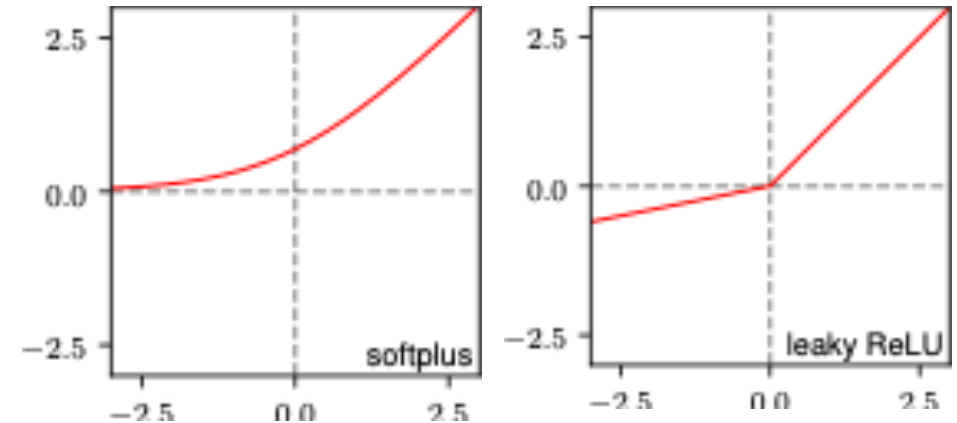
ReLU to the (half) rescue

Rectified Linear Unit

- ReLU gets rid of half this problem ($x > 0$).
- Still have dead units when $x < 0$, why is this catastrophic?
- Many tricks to reduce # of dead units, such as smaller learning rate, "batch normalization", tweaking ReLU to be "leaky".
- Note, for derivative through ReLU, use "subgradients", **0** for $x < 0$, and **1** for $x \geq 0$.



$$RELU(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$



So why are DNNs so good?!

- Linear regression with (arbitrarily high order) polynomial features are also universal function approximators.
- So why do DNNs dominate empirically sometimes?
- Not fully understood—something to do with the optimization landscape created by these massive architectures.
- Active area of research among theorists.

So why are DNNs so good?!

Deep Learning as a Magic Mirror

- Linear rec
polynomi
approxim
- So why d
- Not fully
optimizat
architectu
- Active are



David Donoho Every theorist who looks at it see what they wish

s?
the

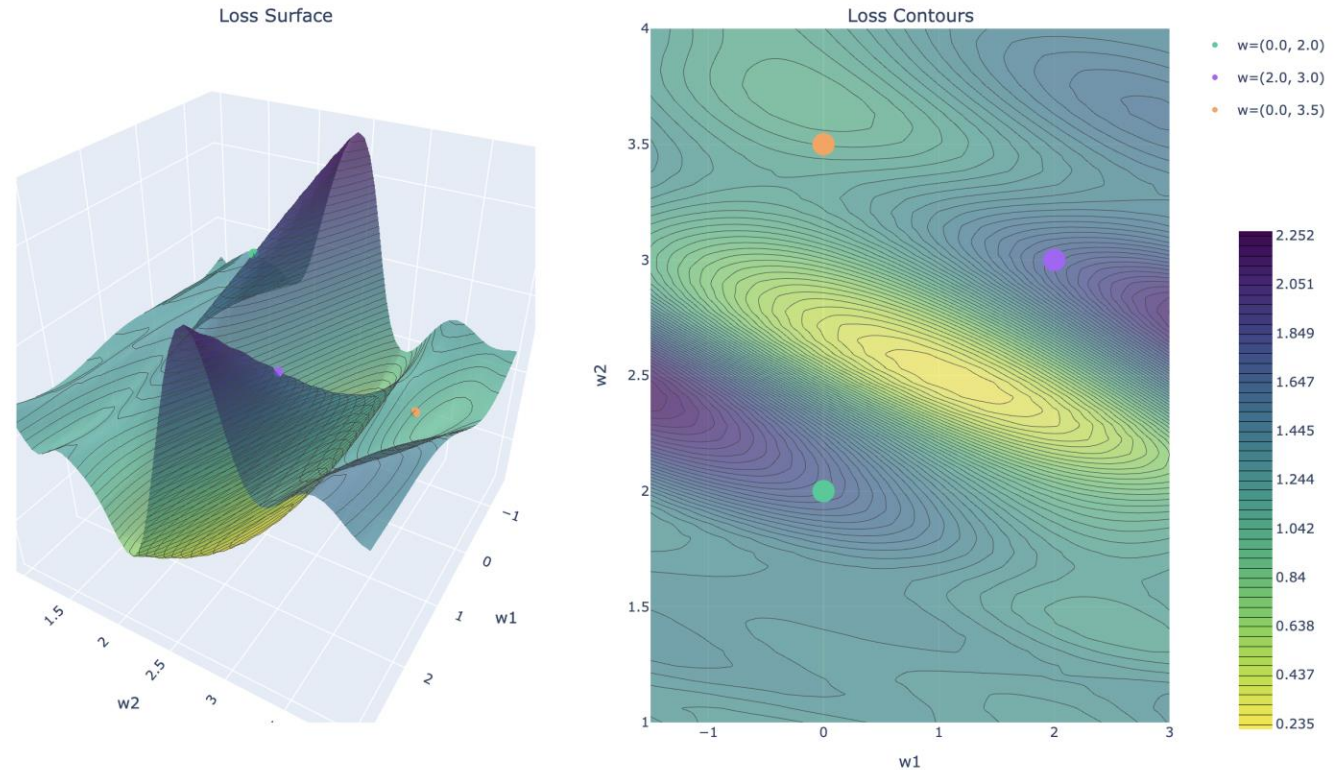
Choice of Initial Parameters Matters

The neural network loss surface is **highly non-convex** so choice of starting point can result in:

- different quality solutions
- slow convergence

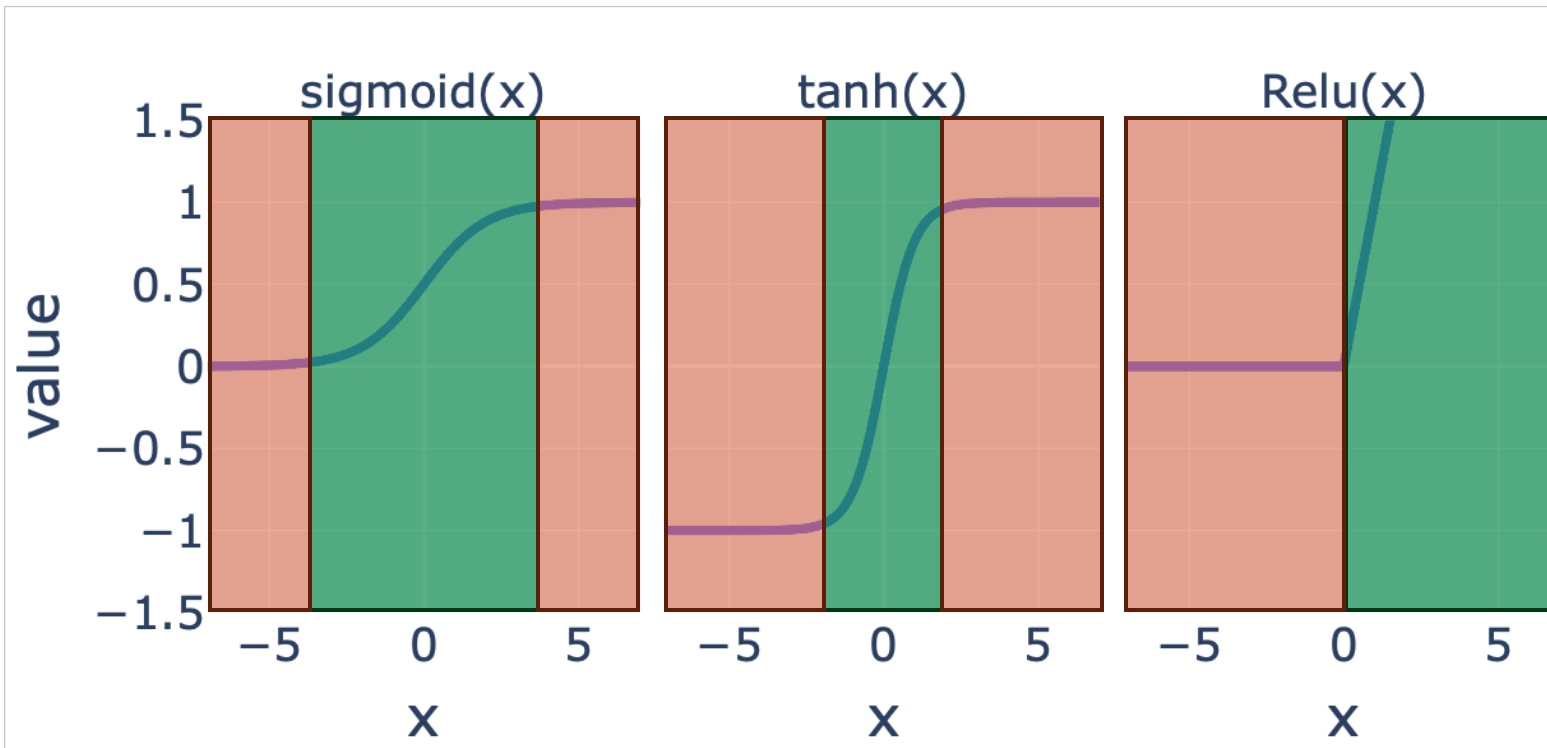
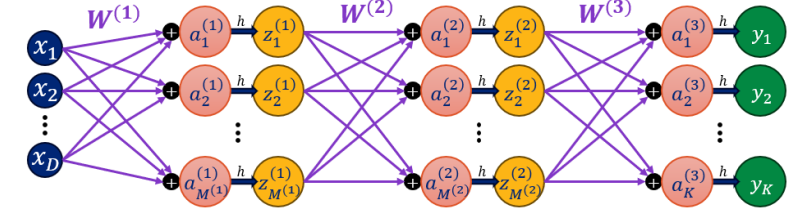
What makes a good starting point?

Think gradients.



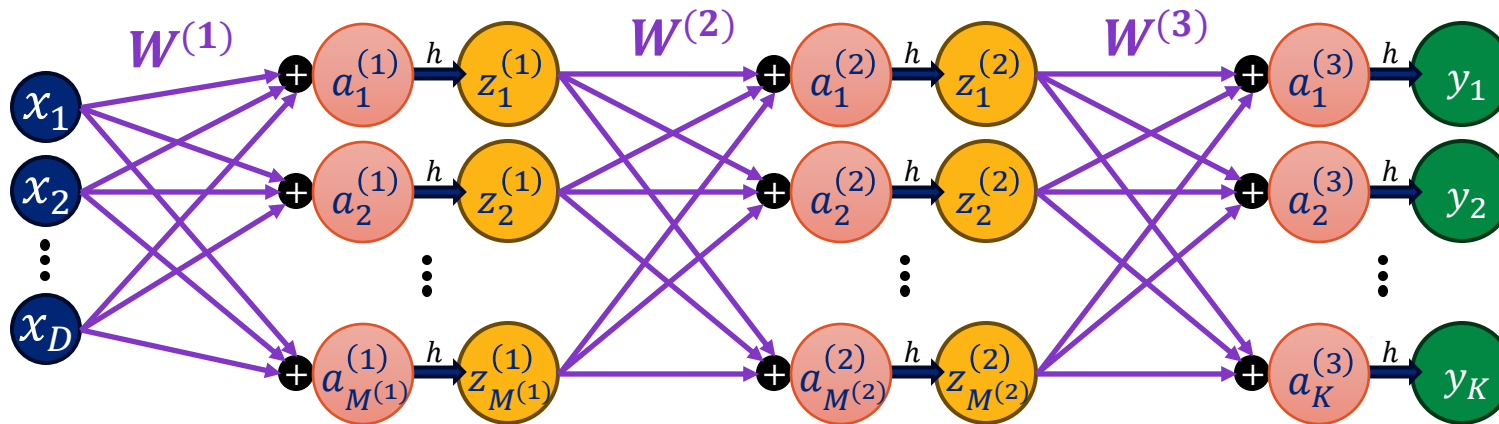
Gradient “Sweet-spot” of the Non-Linearity

- Want largest gradients for fastest learning.
- Where is the largest wrt activation function?
- Typically near zero.
- So lets initialize our weights to zero!



Starting at Zero-Weights

- What happens if we initialize all our weights to zero?
- Gradient for each parameter is the same!

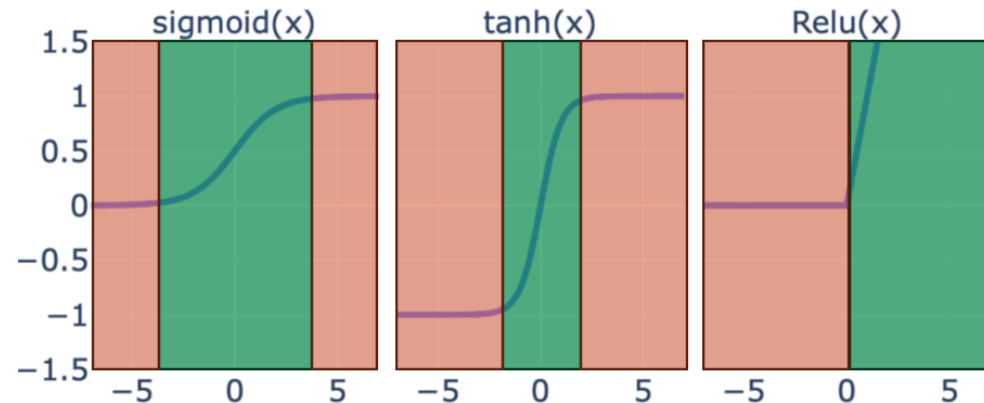


$$a^{(1)} = W^{(1)}x \quad z^{(1)} = h(a^{(1)})$$

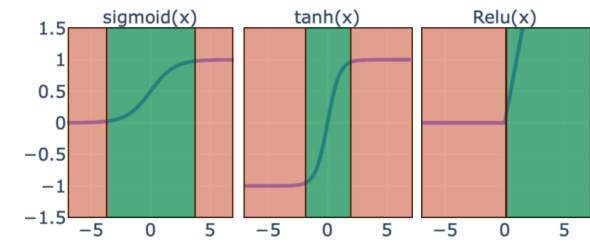
$$a^{(2)} = W^{(2)}z^{(1)} \quad z^{(2)} = h(a^{(2)})$$

$$a^{(3)} = W^{(3)}z^{(2)} \quad y = h(a^{(3)})$$

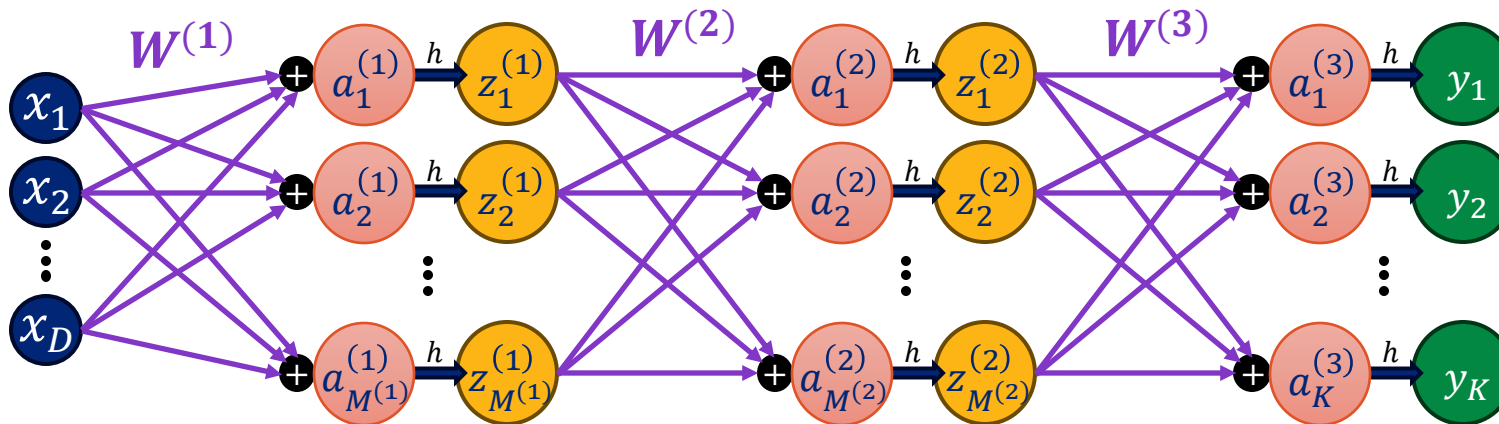
$$y = h\left(W^{(3)}h\left(W^{(2)}h\left(W^{(1)}x\right)\right)\right)$$



Start *near* Zero-Weights



- Practically, initialize weights *near* zero, stochastically, $w_i^l \sim N(0, \epsilon_{(l)}^2)$.
(e.g., “He” init for ReLU dictates variance, $\epsilon_{(l)}^2 = 2/n_{inputs}$)
- (Also use multiple initializations and pick best network or average their results.)



$$a^{(1)} = W^{(1)}x \quad z^{(1)} = h(a^{(1)})$$

$$a^{(2)} = W^{(2)}z^{(1)} \quad z^{(2)} = h(a^{(2)})$$

$$a^{(3)} = W^{(3)}z^{(2)} \quad y = h(a^{(3)})$$

$$y = h\left(W^{(3)}h\left(W^{(2)}h\left(W^{(1)}x\right)\right)\right)$$

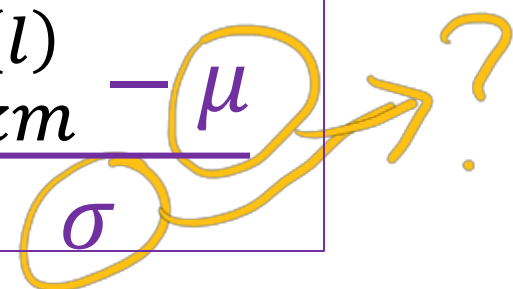
$$\overline{v}_i = \sum_{j \in \text{Ch}(v_i)} \overline{v}_j \frac{\partial v_j}{\partial v_i}$$

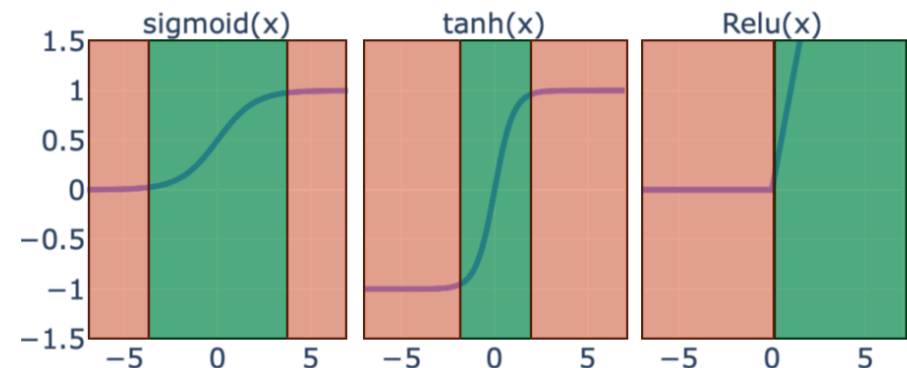
How to keep the gradient flowing?

- Initialization helps us to ensure the activation functions have “healthy” gradient by starting with small weights.
- Once we start learning, all bets are off!
What to do?
- Control the inputs to the non-linearities to encourage that we keep getting gradients: *batch normalization*.

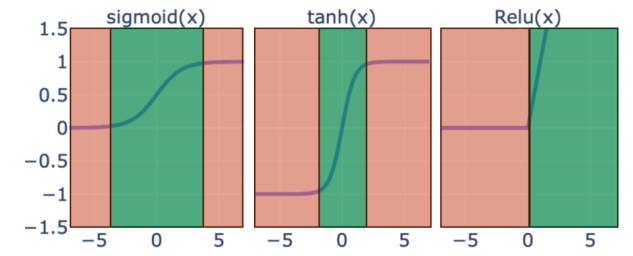
$$\hat{z}_{km}^{(l)} = \frac{z_{km}^{(l)} - \mu}{\sigma}$$

$a^{(2)} = W^{(2)}z^{(1)}$ $z^{(2)} = h(a^{(2)})$





Batch Normalization



Normalize pre-activations $\hat{a}_{km}^{(l)}$ during learning by computing μ_i and σ_i within a mini-batch of SGD.

$$\hat{a}_{km}^{(l)} = \frac{a_{km}^{(l)} - \mu_m^{(l)}}{\sqrt{(\sigma_m^{(l)})^2 + \delta}}$$

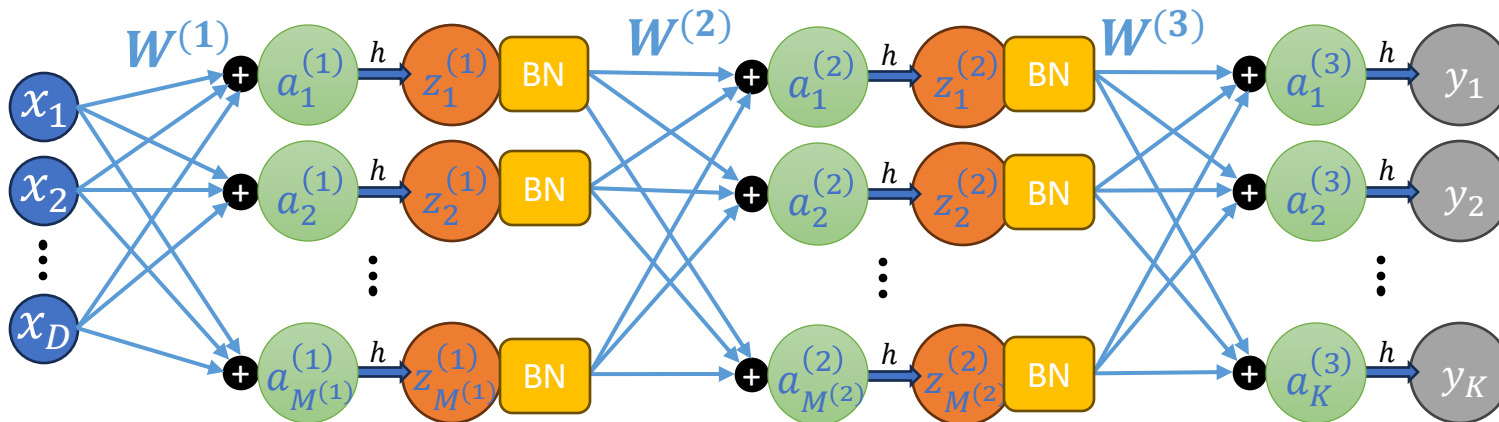
Pre-activation m on layer l for the k^{th} datapoint in the mini-batch

$$\mu_m^{(l)} = \frac{1}{K} \sum_{k=1}^K a_{km}^{(l)}$$

Mean over the mini-batch of K datapoints

$$(\sigma_m^{(l)})^2 = \frac{1}{K} \sum_{k=1}^K (a_{km}^{(l)} - \mu_m^{(l)})^2$$

Variance over the mini-batch of K datapoints



Training & Making Predictions with Batch Norm

- Training: these are just linear operations—use backprop!
- Predictions/test time:
 - Want the mean $\mu^{(l)}$ and variance $(\sigma^{(l)})^2$ for each layer, for all of the training data, but doing SGD (batches).
 - Ideally, we would compute them from the entire training dataset (after training), but too expensive.
 - Instead, compute running decaying exponential average from SGD batches.

per layer, per batch:

$$\hat{a}_{km}^{(l)} = \frac{a_{km}^{(l)} - \mu_m^{(l)}}{\sqrt{(\sigma_m^{(l)})^2 + \delta}} \quad \mu_m^{(l)} = \frac{1}{K} \sum_{k=1}^K a_{km}^{(l)} \quad (\sigma_m^{(l)})^2 = \frac{1}{K} \sum_{k=1}^K (a_{km}^{(l)} - \mu_m^{(l)})^2$$

Also typically normalize the training data

- Input variables may lie in very different ranges (e.g. height in meters, pinky to width in mm).
- These variations can make gradient descent challenging, as some directions move faster than others.
- Thus, for continuous variables, typically normalize them to have zero mean and unit variance:

$$\mu_i = \frac{1}{N} \sum_{n=1}^N x_{ni}$$

$$\sigma_i^2 = \frac{1}{N} \sum_{n=1}^N (x_{ni} - \mu_i)^2$$

$$\tilde{x}_{ni} = \frac{x_{ni} - \mu_i}{\sigma_i}$$

Must do exactly the same transformation for validation/test data.

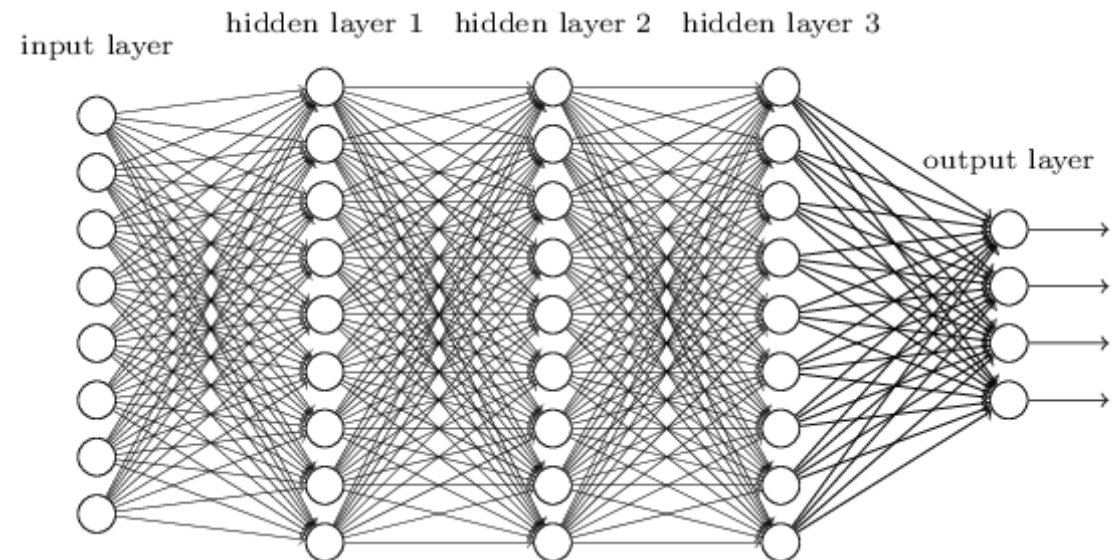
CS 189/289

Today's lecture outline

1. Finish backprop
2. Convolutional Neural Networks

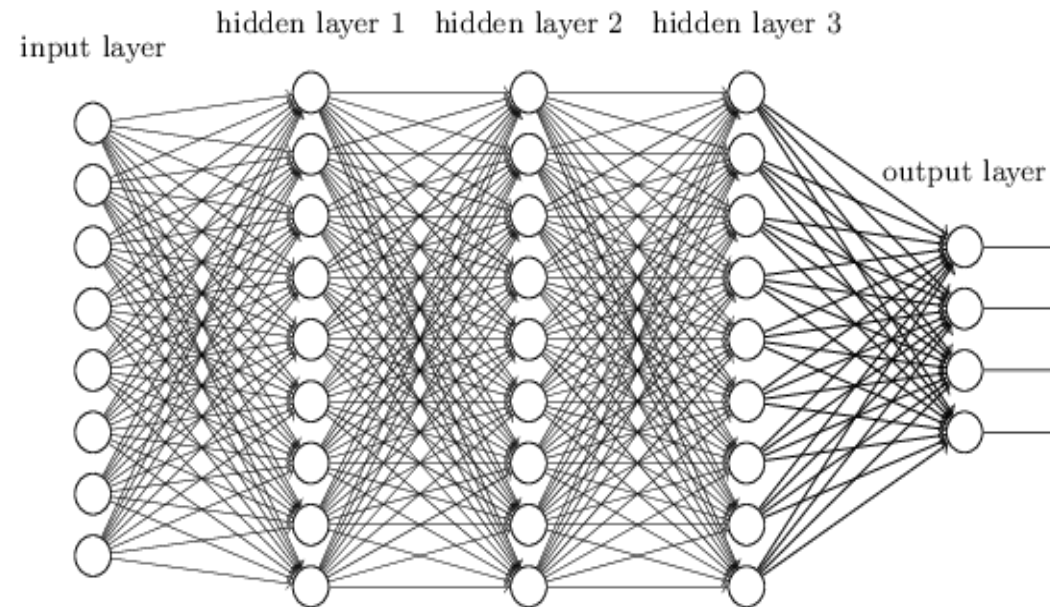
Pondering fully connected neural networks

- We have so far talked about NNs with “fully connected” (FC) layers.
- If there were $n_i(l)$ inputs (features) and $n_o(l)$ outputs (hidden/output nodes) at layer l , then this yields $n_i(l) \times n_o(l)$ parameters in W_l .
- This adds up quickly!



Pondering fully connected neural networks

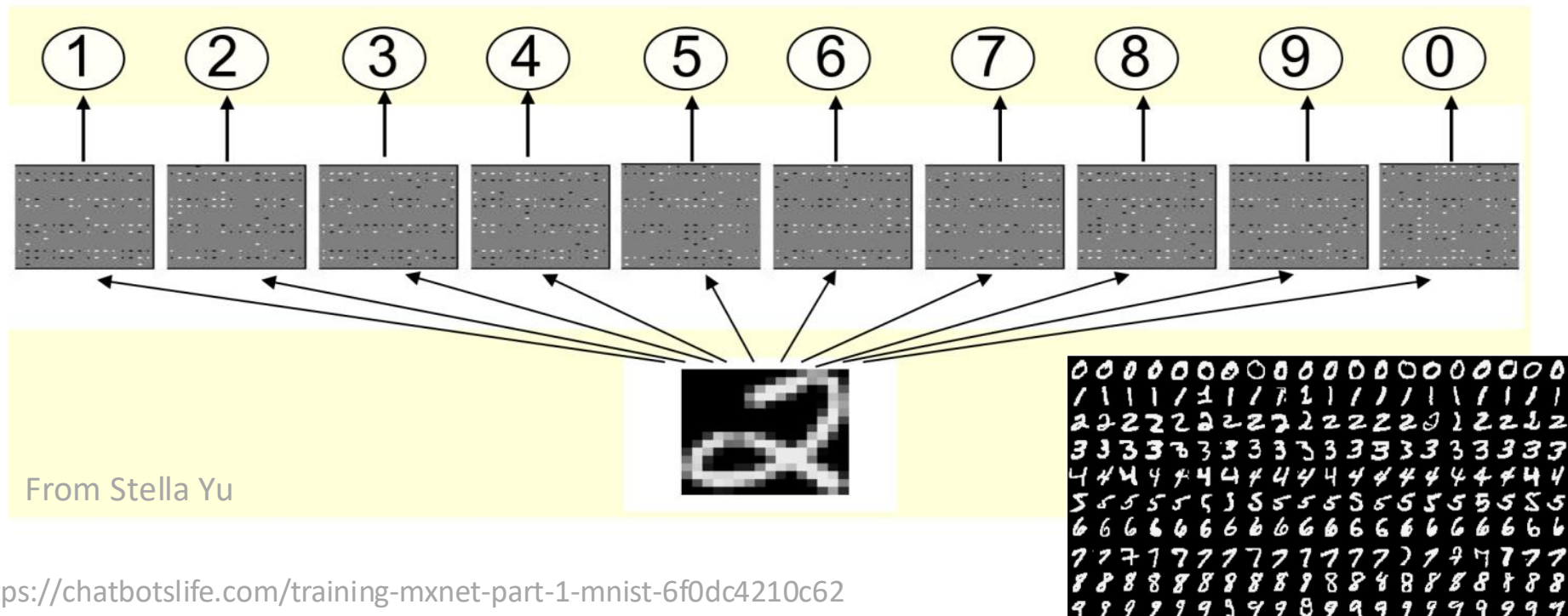
With fully connected layers, “features” cannot be reused, because every region of the image (or hidden nodes) has its own weight parameters.



Pondering fully connected neural networks

- Uses “global template matching”.
- e.g. one W matrix per class (single layer):

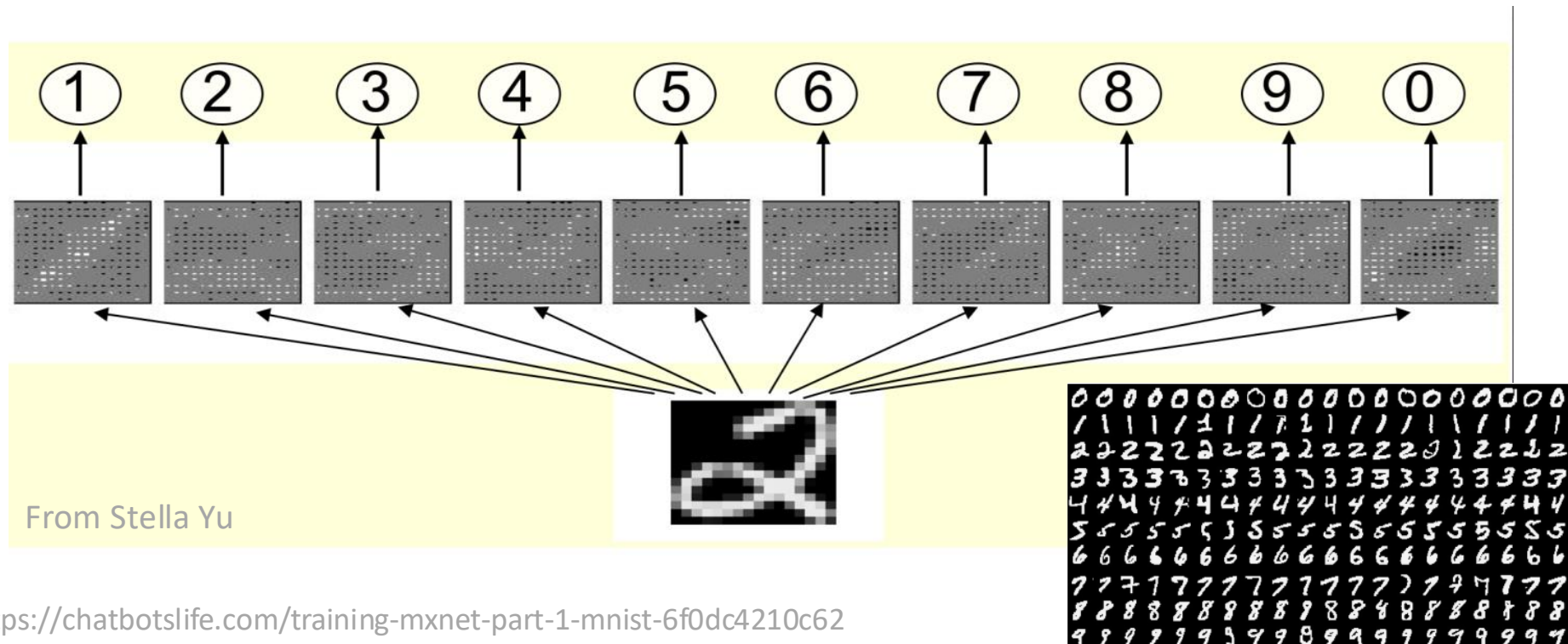
Iteration 1 of training:



Pondering fully connected neural networks

- Uses “global template matching”.
- e.g. one W matrix per class (single layer):

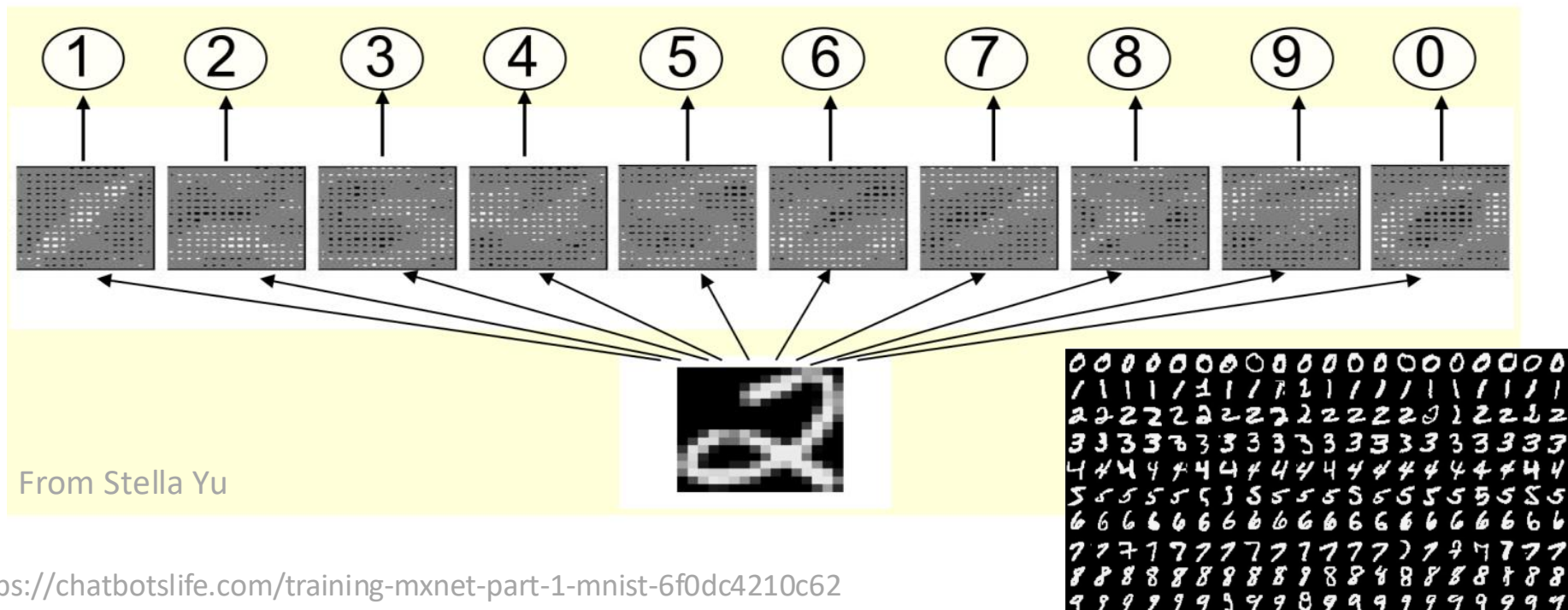
Iteration 2 of training:



Pondering fully connected neural networks

- Uses “global template matching”.
- e.g. one W matrix per class (single layer):

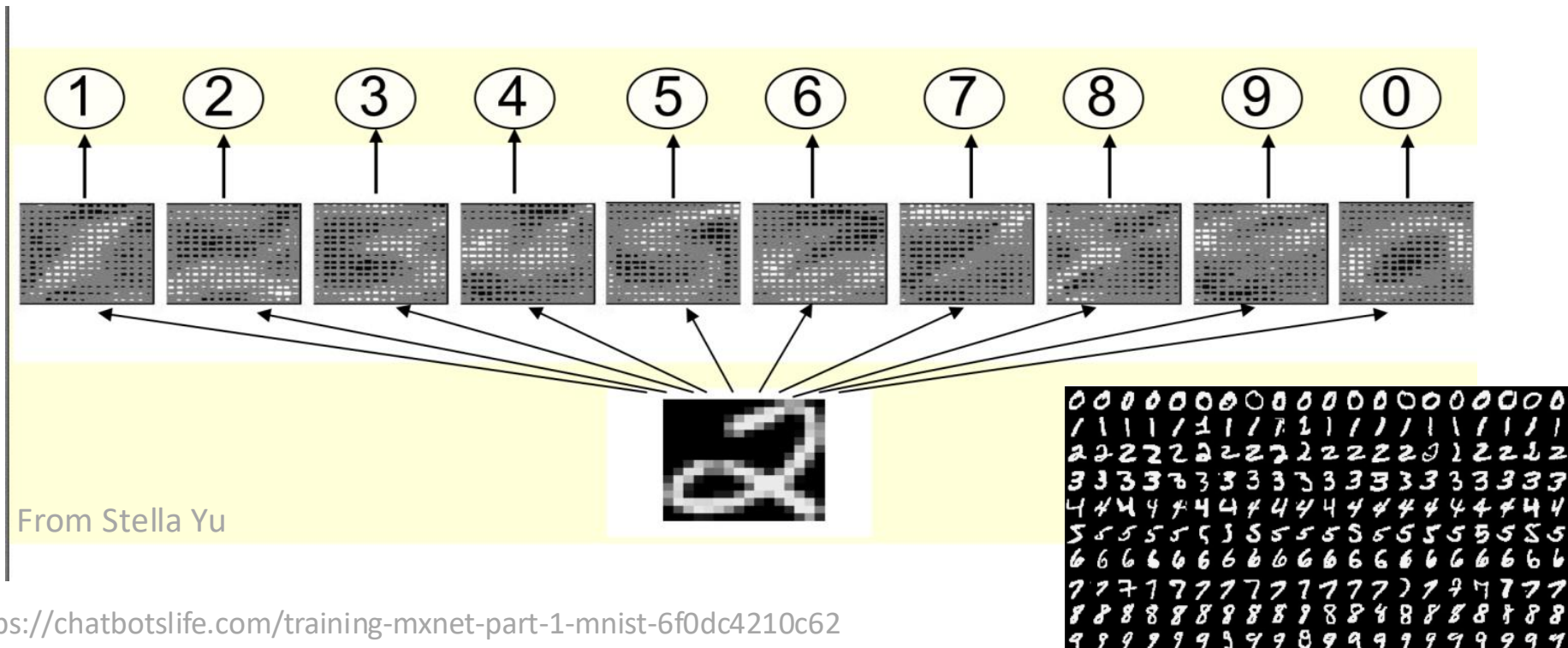
Iteration 3 of training:



Pondering fully connected neural networks

- Uses “global template matching”.
- e.g. one W matrix per class (single layer):

Iteration 7 of training:



Pondering fully connected neural networks

Fully Connected: whole template matching

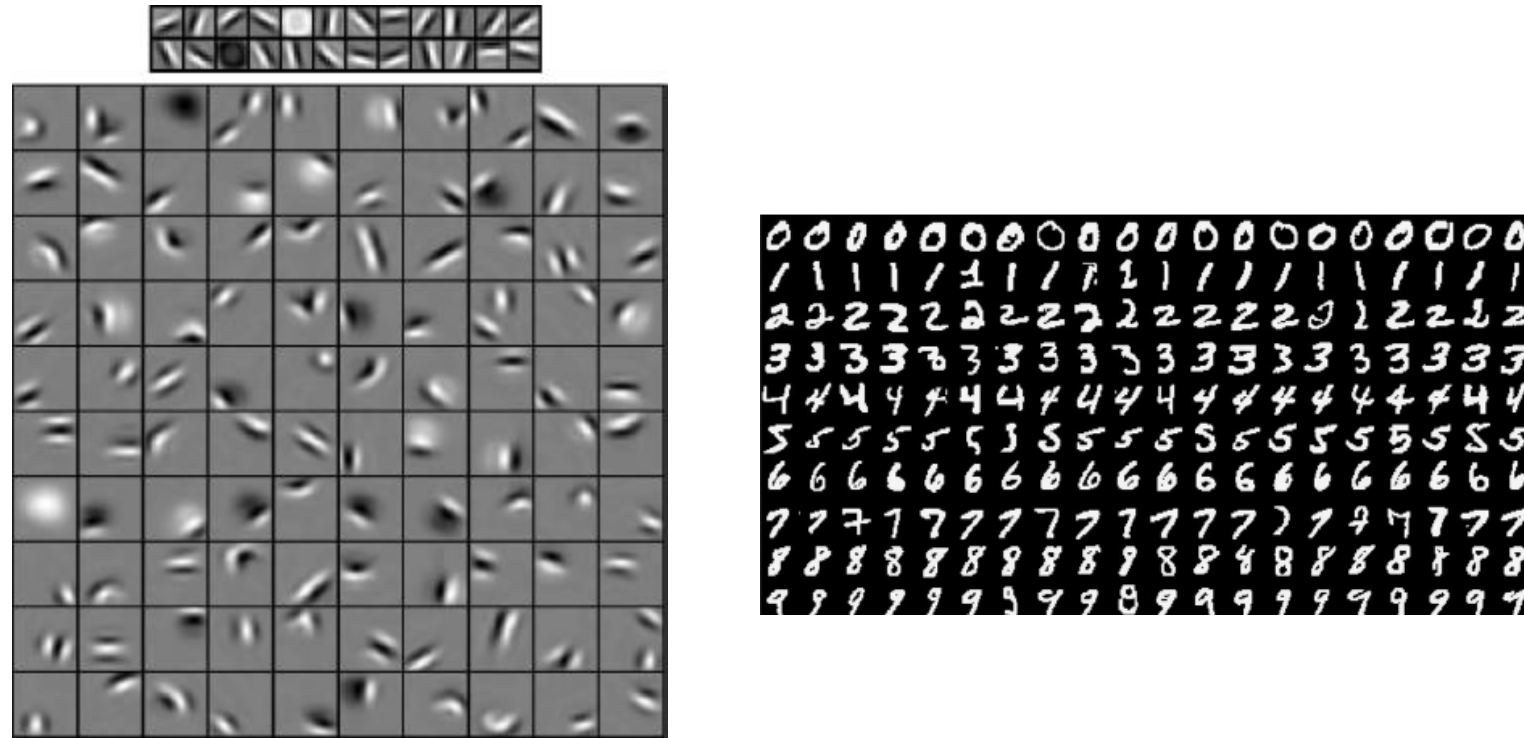
- That last e.g. had a single rigid template for each class (0,1, 2...9).
- The predicted class is the template that best matches the image.

Desiderata: Parts and Compositions

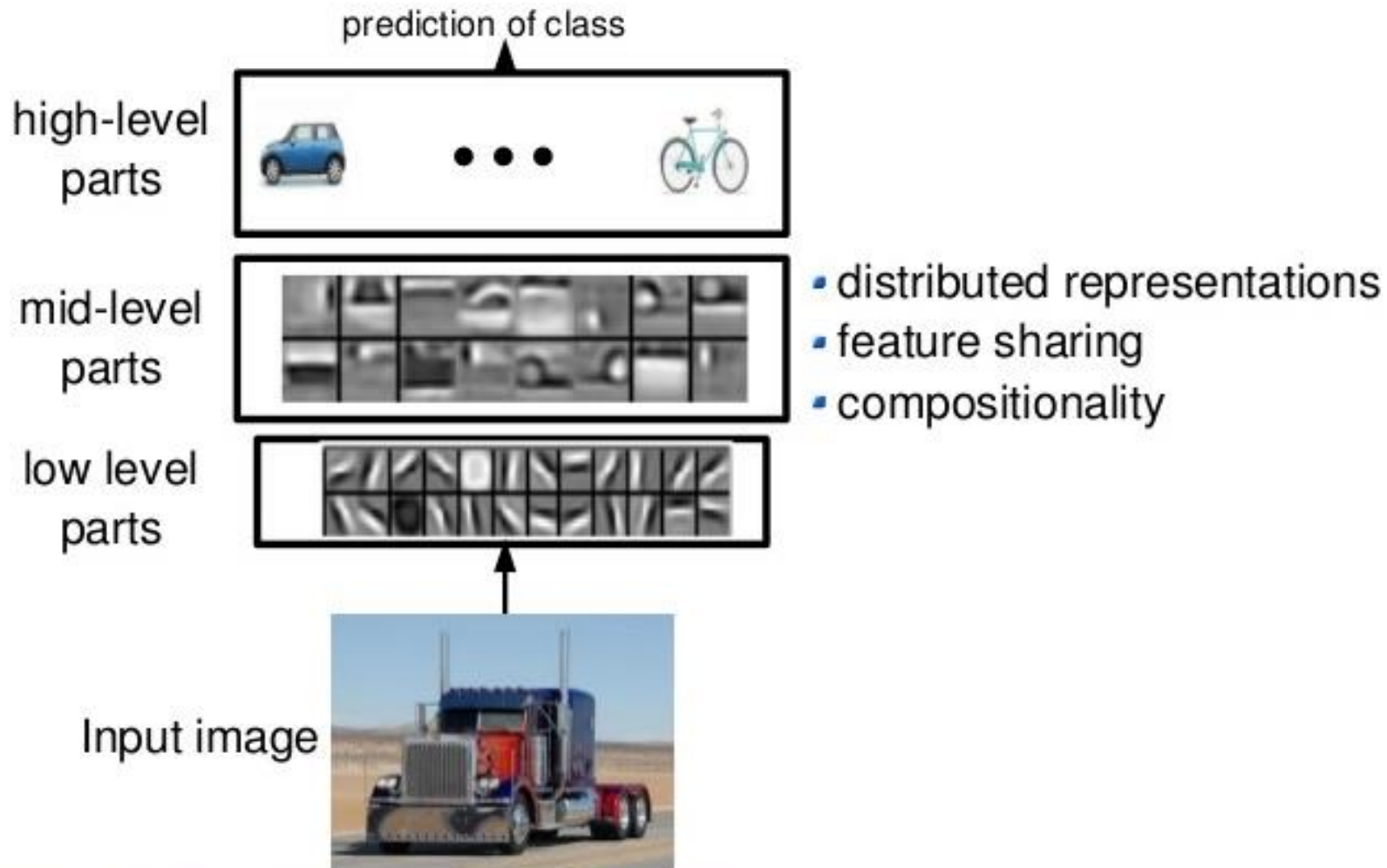
- A more compact and robust approach would be to break down digit images into small components, and then then combine them to make predictions.

Pondering fully connected neural networks

Could imagine using hierarchical “local feature detectors” instead of these global templates.



Convolutional NNs (CNNs/"Convnets")



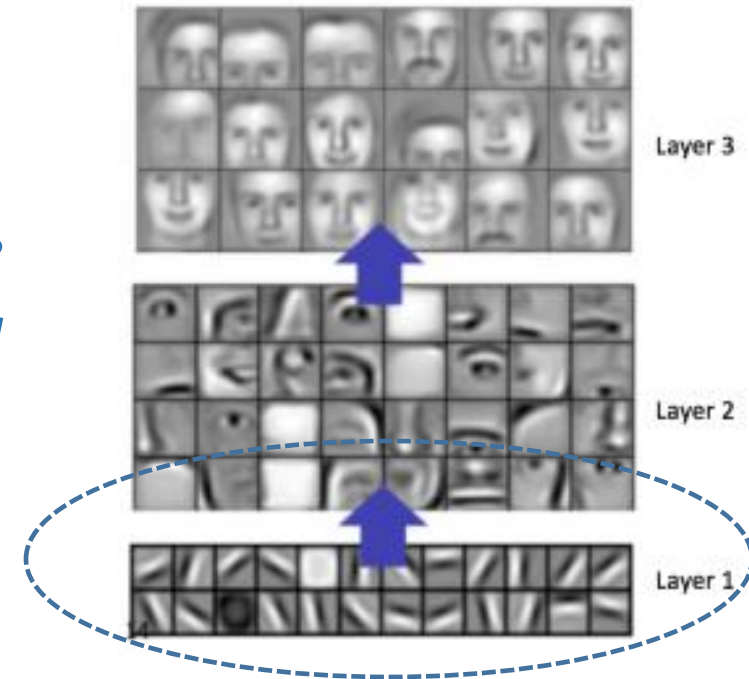
Lee et al. "Convolutional DBN's ..." ICML 2009

Convolutional NNs (CNNs/"Convnets")

- We want a NN architecture that lets us learn a set of *feature detectors* that are applied at all image locations:
- Convnets will let us do this.

Various types of "edge feature detectors" that emerge during learning of a ConvNet.

Get combined hierarchically.

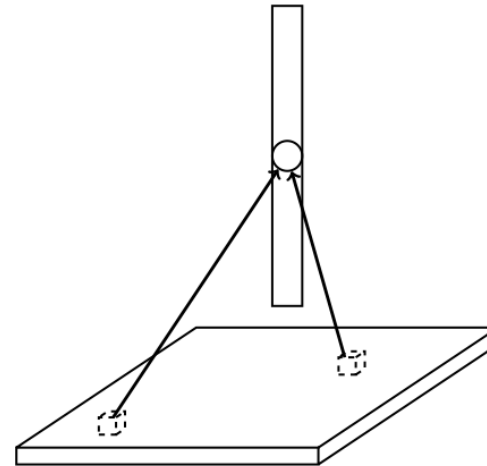
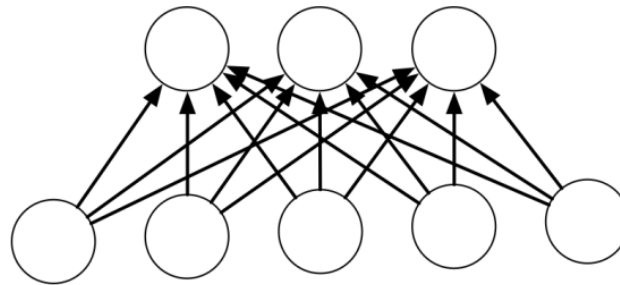


Convolutional NNs (CNNs/"Convnets")

How will this work?

1. We will have hidden nodes which are only "locally connected" to its inputs.

Fully connected layers:



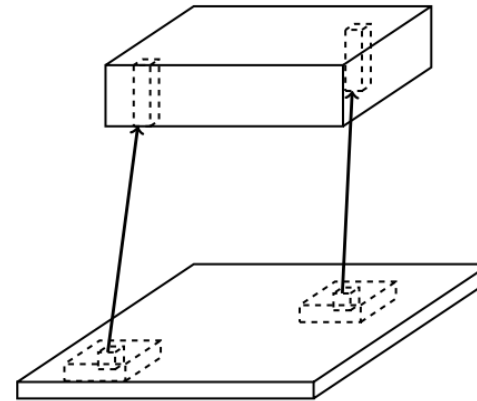
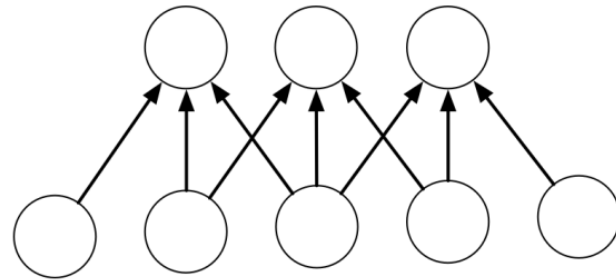
Each hidden unit looks at the entire image.

Convolutional NNs (CNNs/"Convnets")

How will this work?

1. We will have hidden nodes which are only "locally connected" to its inputs.

Locally connected layers:



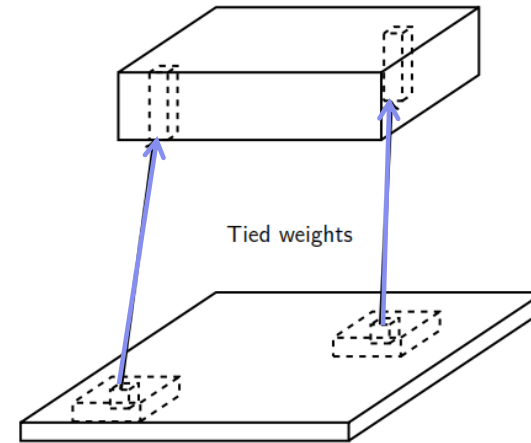
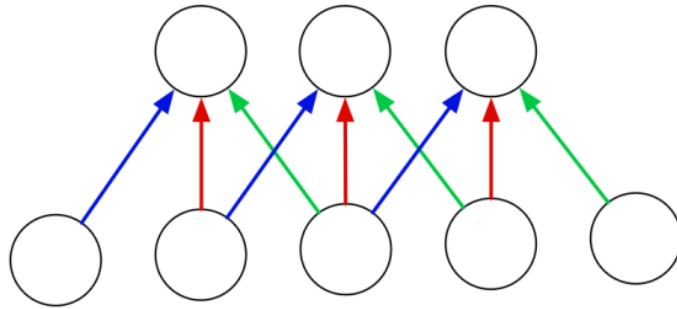
Each column of hidden units looks at a small region of the image.

Convolutional NNs (CNNs/"Convnets")

How will this work?

2. We will force the **weights to be shared**.

Convolution layers:

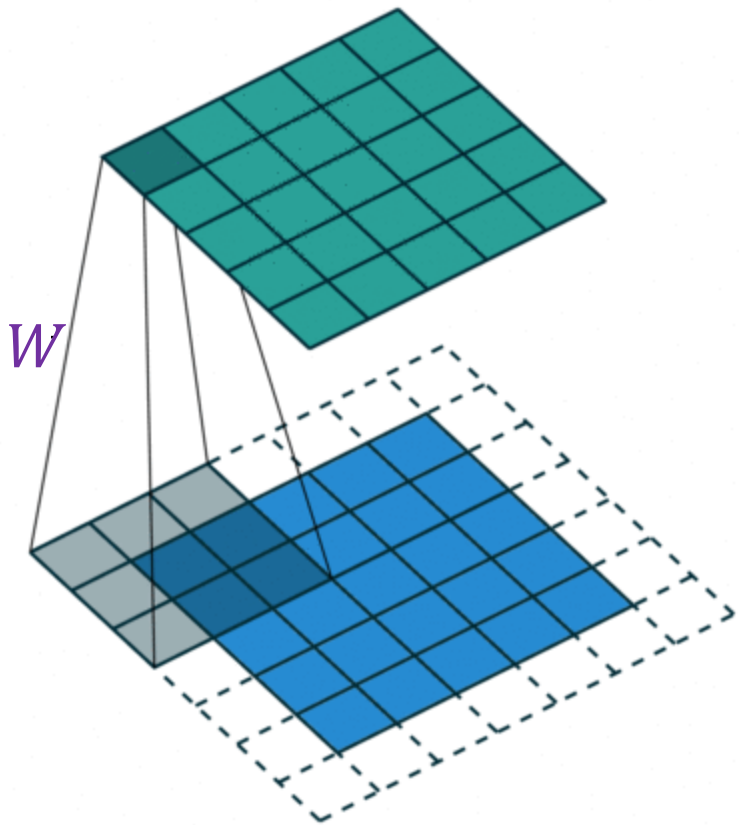


Each column of hidden units looks at a small region of the image, and the weights are shared between all image locations.

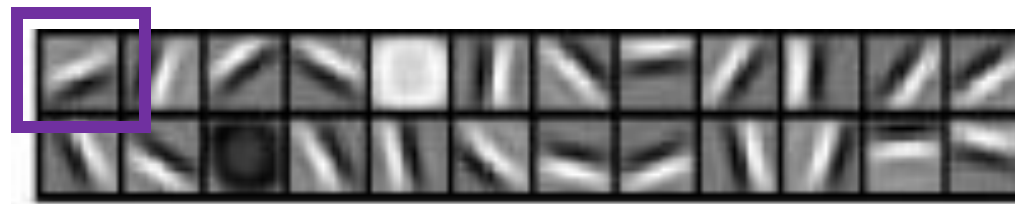
Convolutional NNs (CNNs/"Convnets")

How will this work?

2. We will force the **weights to be shared**.

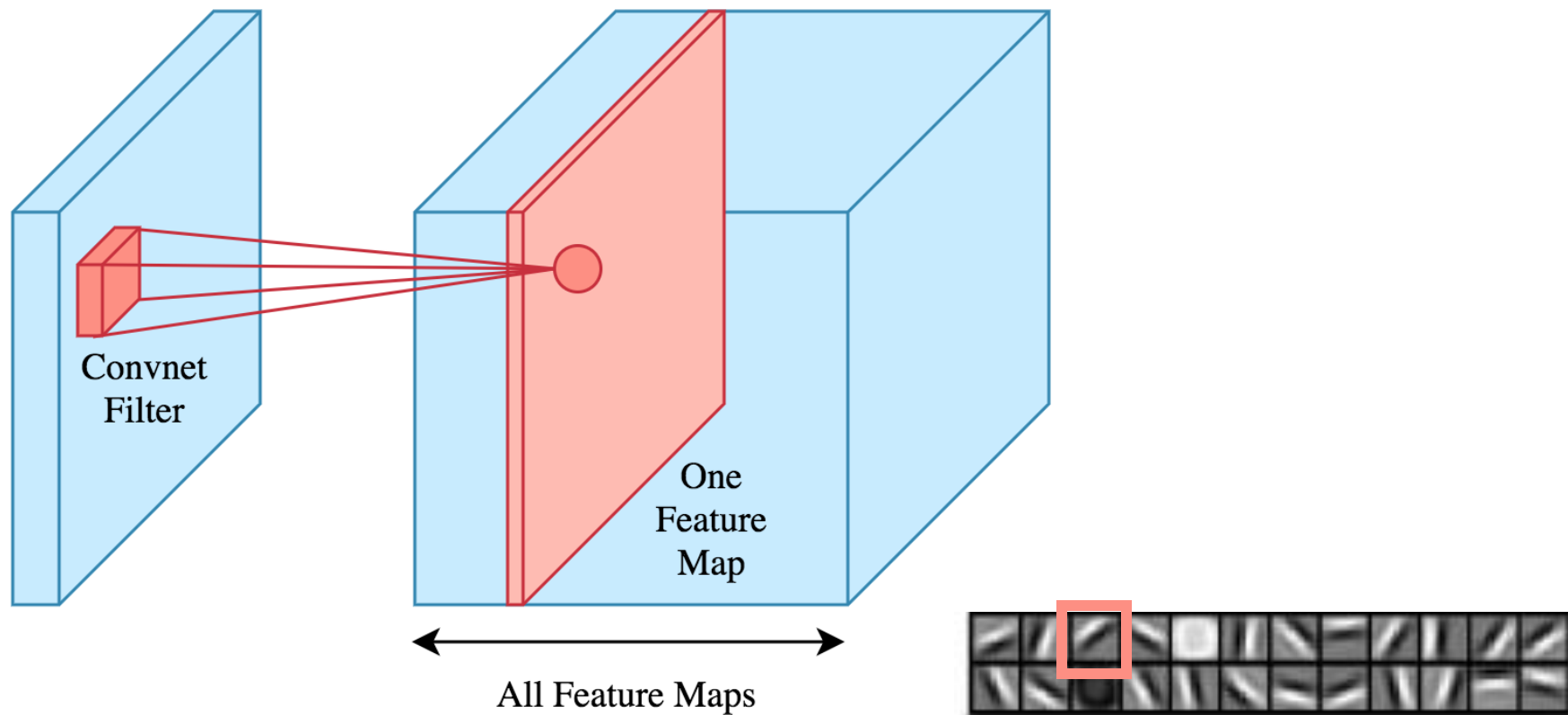


Convolution: one "*filter*", W , maps the *inputs* to the *outputs* ("feature map"), by scanning



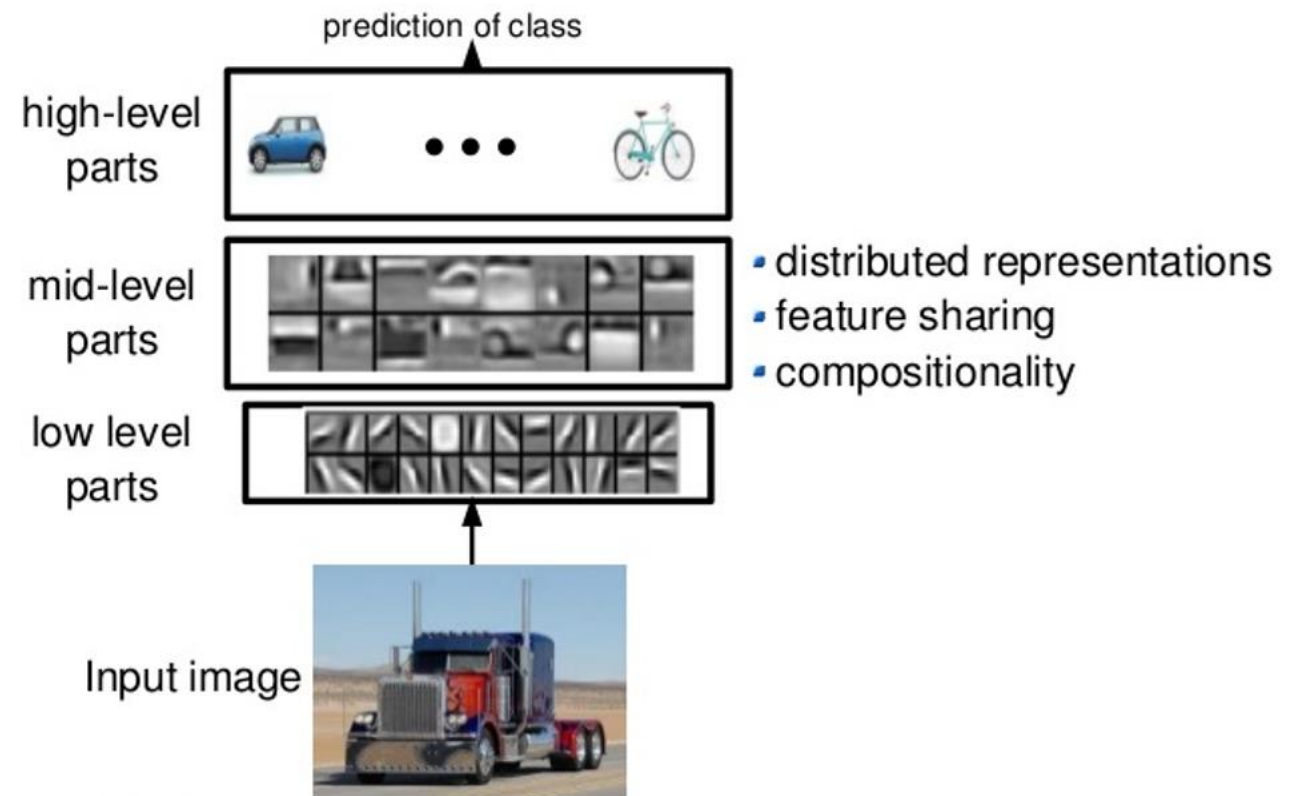
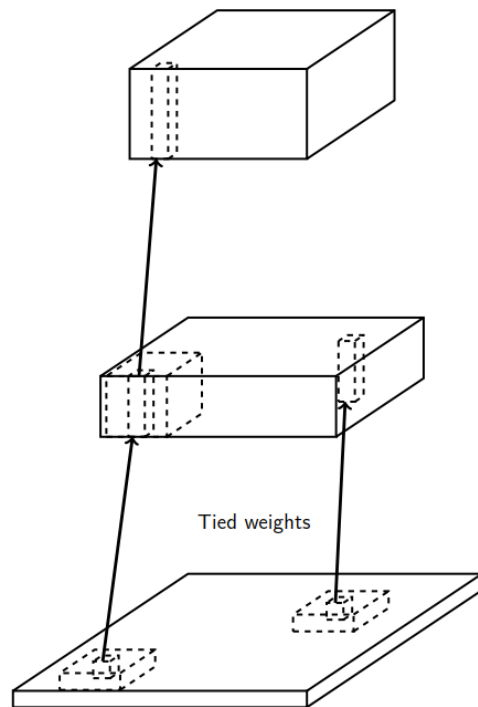
Convolutional NNs (CNNs/"Convnets")

Depth of output volume is number of filters=#feature maps:



Convolutional NNs (CNNs/"Convnets")

- Convolutional layers can be stacked.
- This gives us a hierarchical parts decomposition approach.



Lee et al. "Convolutional DBN's ..." ICML 2009

CNNs start winning competitions

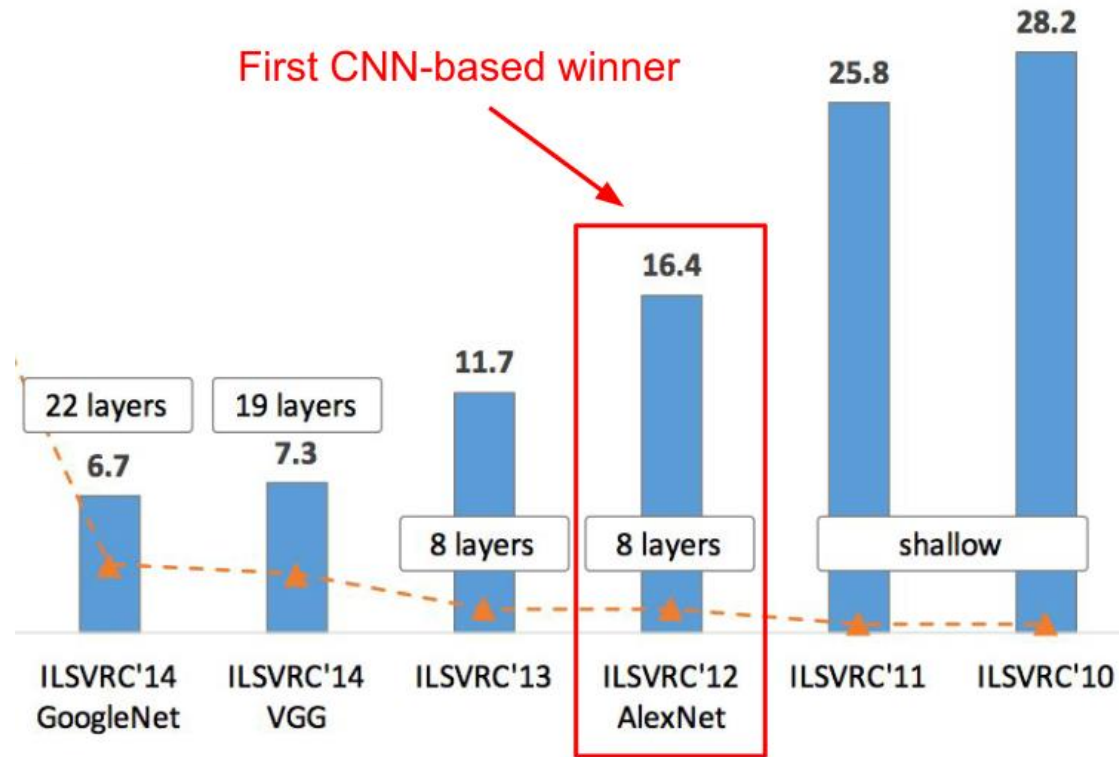
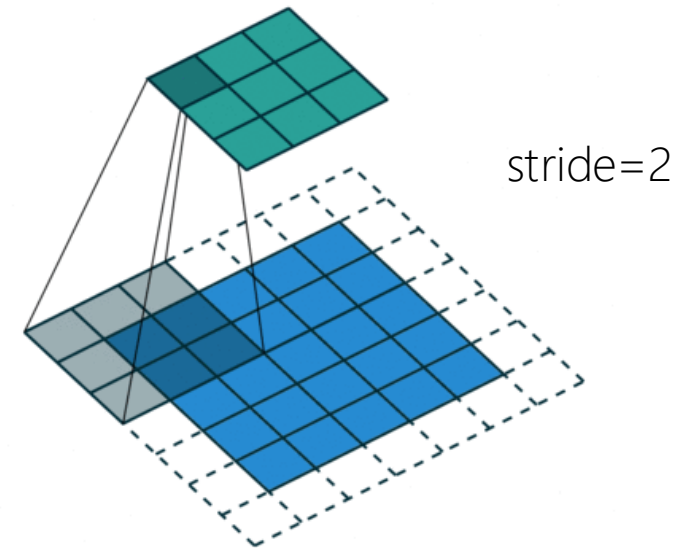
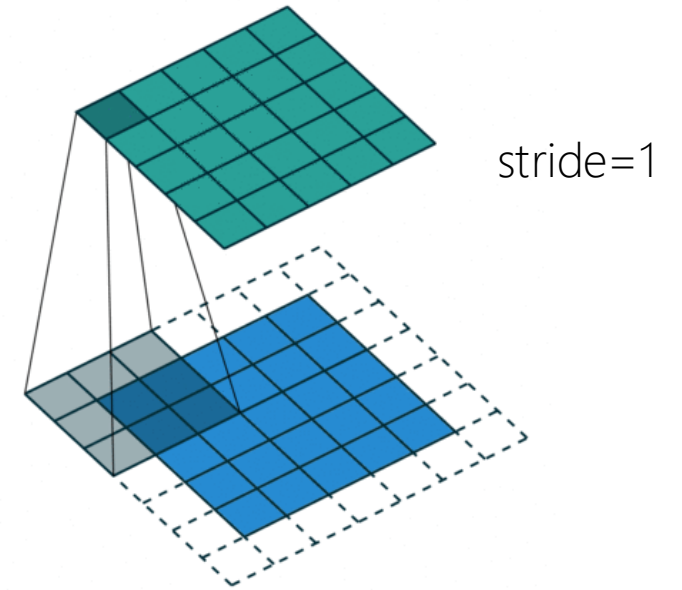


Figure copyright Kaiming He, 2016.

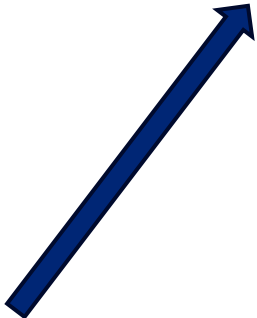
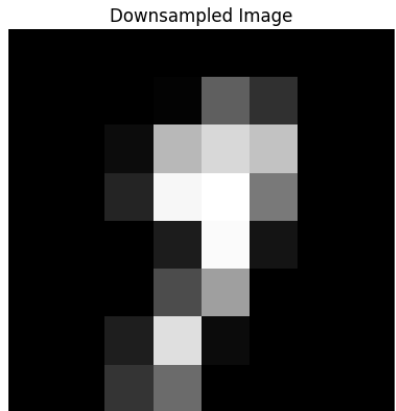
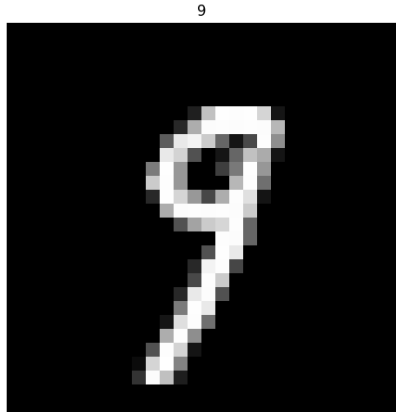
ImageNet Large scale visual recognition challenge (ILSVRC)

Back to details of convolution

- Intuitively convolution is scanning a “filter” across the image at every position, each time doing an element-wise product and summing up.
- “filter” = “kernel” = “template”
- Sometimes skip positions when scanning, dictated by the “stride” (no skip, stride=1)



Example calculation of 2D convolution



0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

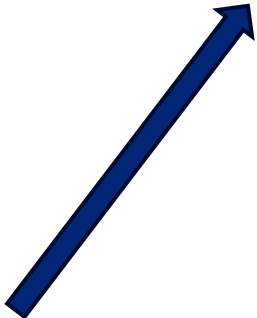
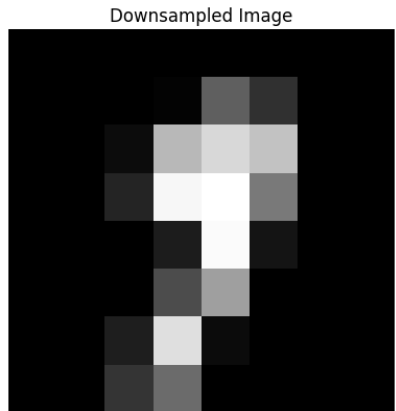
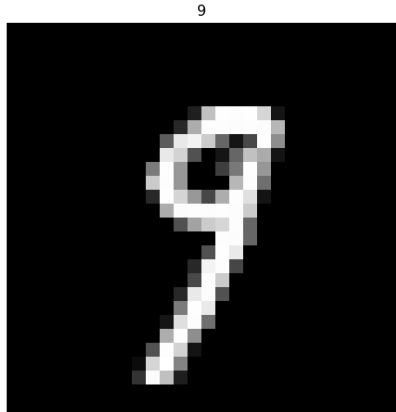
Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line detector



Example calculation of 2D convolution



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

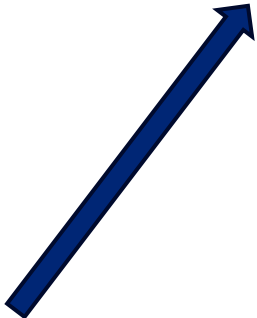
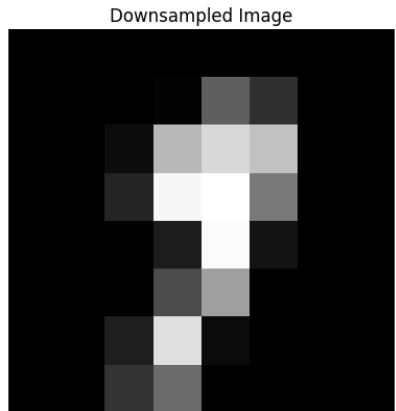
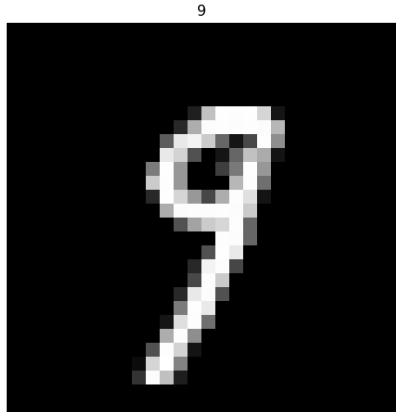
$\odot$

-1

0

Horizontal line detector

Example calculation of 2D convolution



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

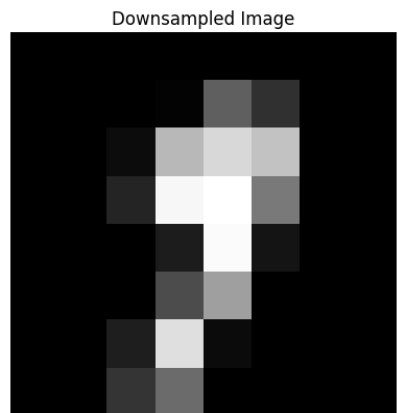
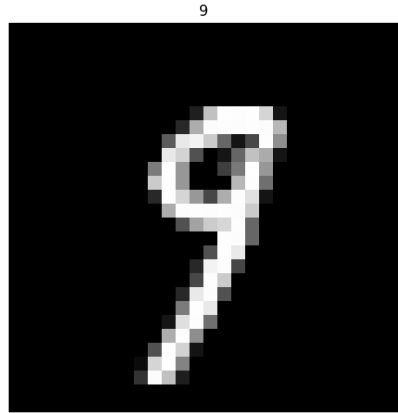
-1	-1	-1
2	2	2
-1	-1	-1

$\odot$



Horizontal line detector

Example calculation of 2D convolution



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

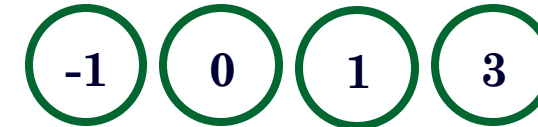
Feature extractor

Or kernel

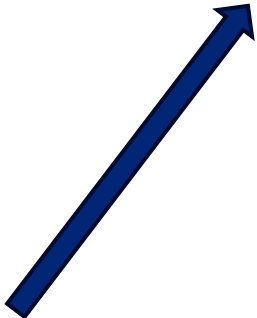
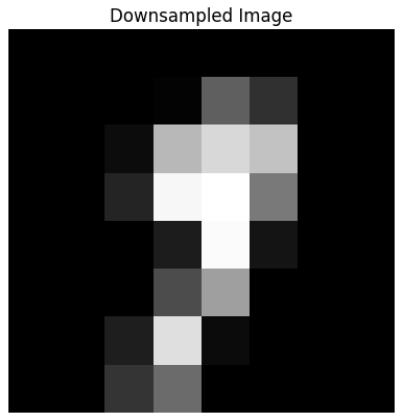
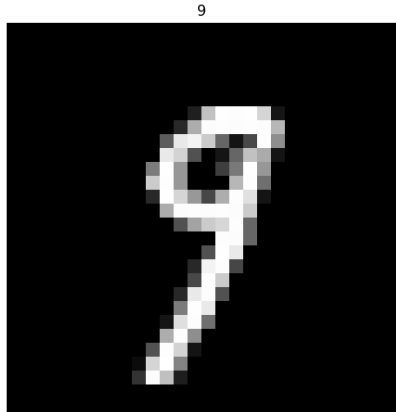
-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line detector

$\odot$



Example calculation of 2D convolution



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

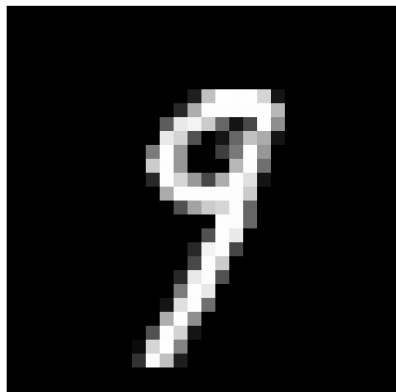
Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

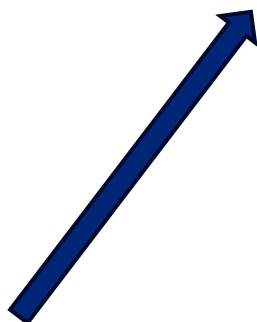
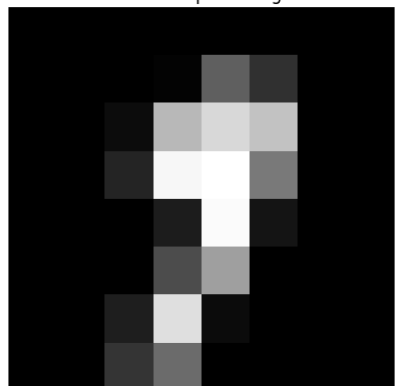
Horizontal line detector



9



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

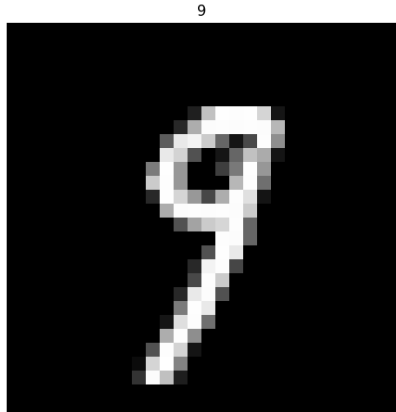
Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

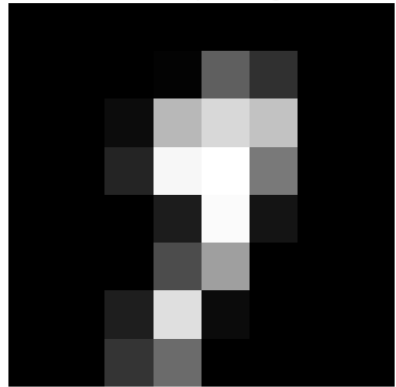
⊙

Horizontal line
detector

Example calculation of 2D convolution



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

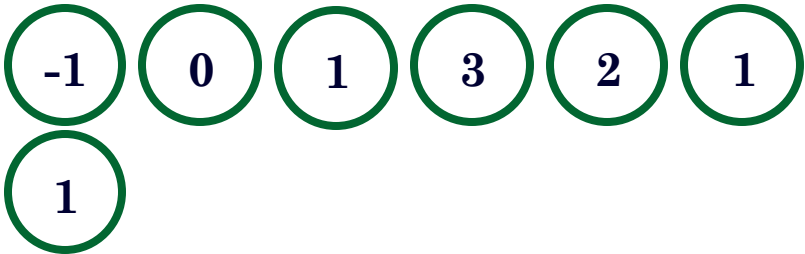
Feature extractor

Or kernel

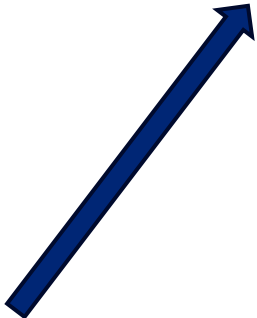
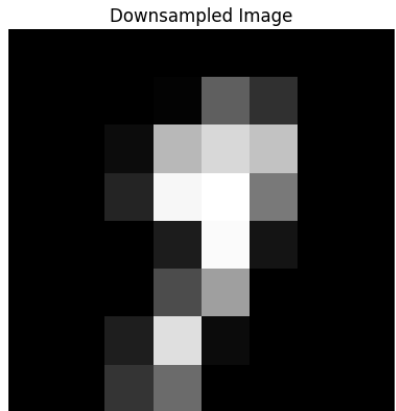
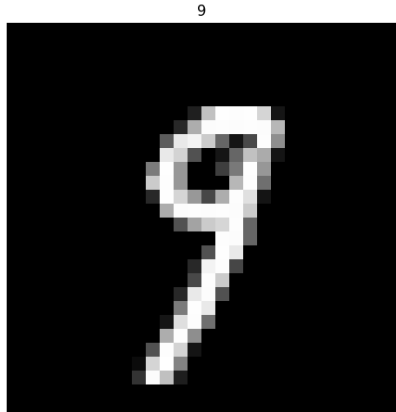
-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line detector

$\odot$



Example calculation of 2D convolution



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

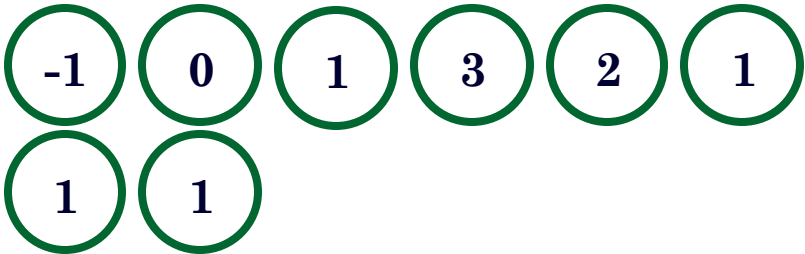
Feature extractor

Or kernel

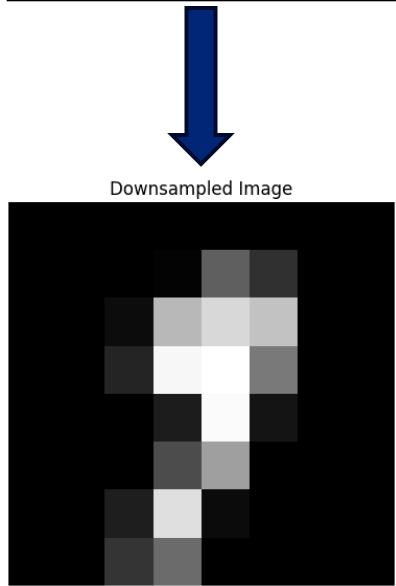
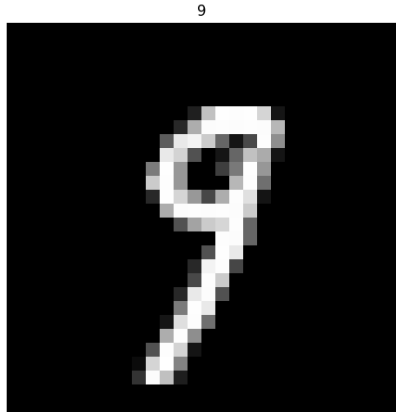
-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

Horizontal line detector



Example calculation of 2D convolution



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

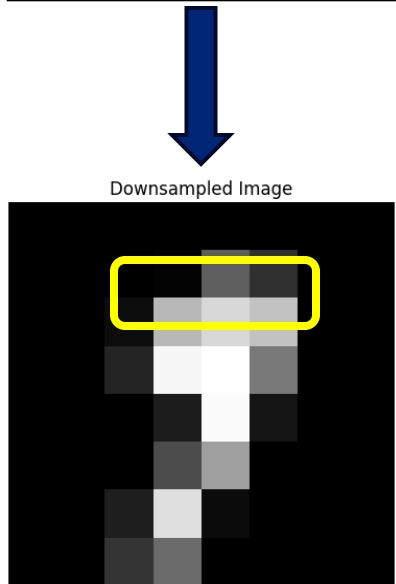
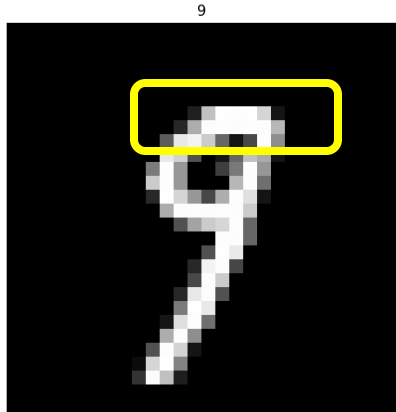
-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line detector



-1	0	1	3	2	1
1	1	1	0	0	0
1	1	1	0	0	0
-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1
1	1	2	1	1	0

Example calculation of 2D convolution



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

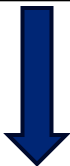
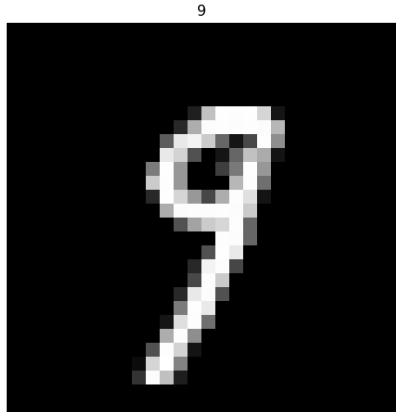
-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line detector

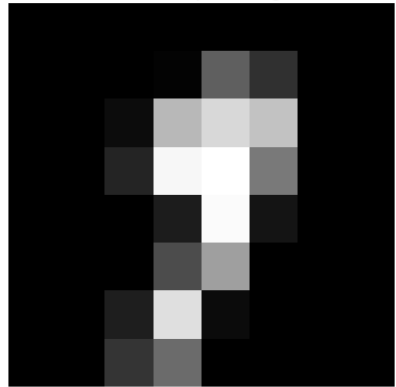
$\odot$

-1	0	1	3	2	1
1	1	1	0	0	0
1	1	1	0	0	0
-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1
1	1	2	1	1	0

Example calculation of 2D convolution



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Horizontal line detector

-1	-1	-1
2	2	2
-1	-1	-1

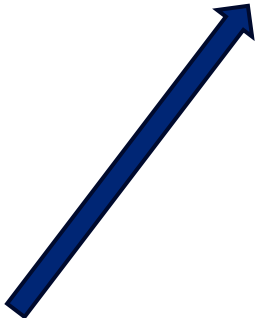
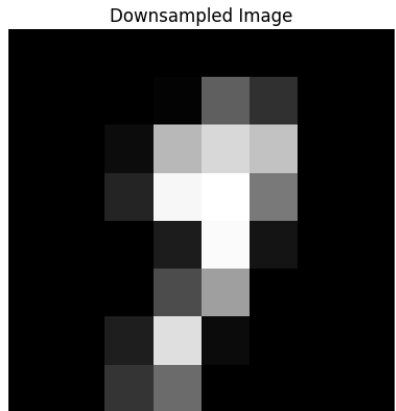
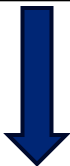
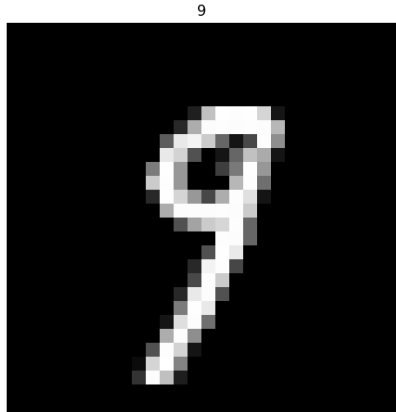
Vertical line detector

-1	2	-1
-1	2	-1
-1	2	-1

$\odot$

-1	0	1	3	2	1
1	1	1	0	0	0
1	1	1	0	0	0
-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1
1	1	2	1	1	0

Example calculation of 2D convolution



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

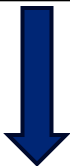
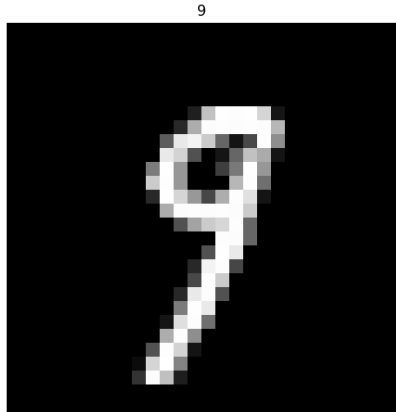
Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

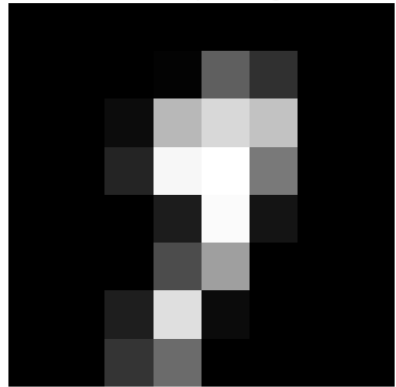
$\odot$

-1	0	1	3	2	1
1	1	1	0	0	0
1	1	1	0	0	0
-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1
1	1	2	1	1	0

Example calculation of 2D convolution



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

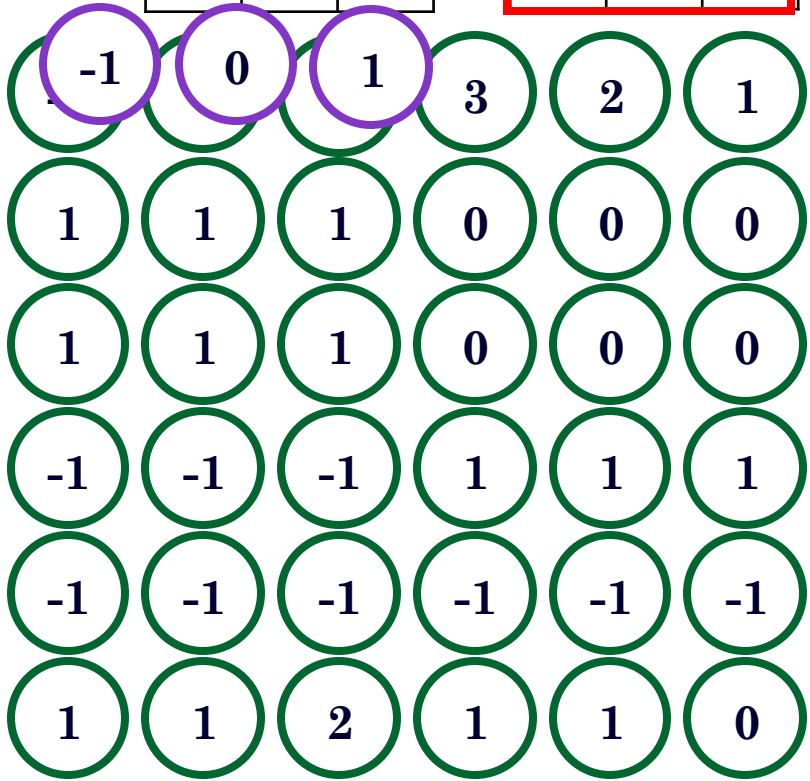
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

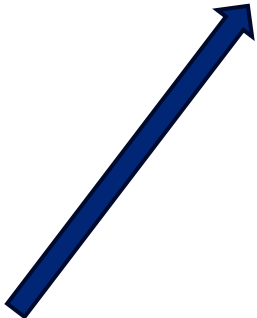
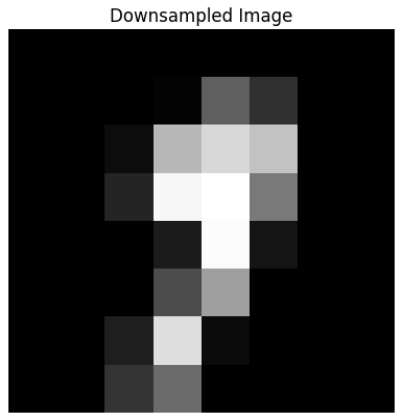
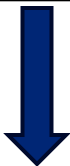
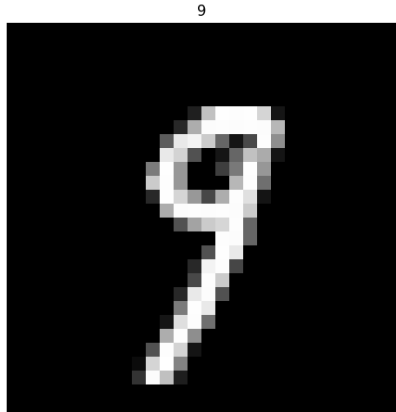
Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

$\odot$



Example calculation of 2D convolution



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

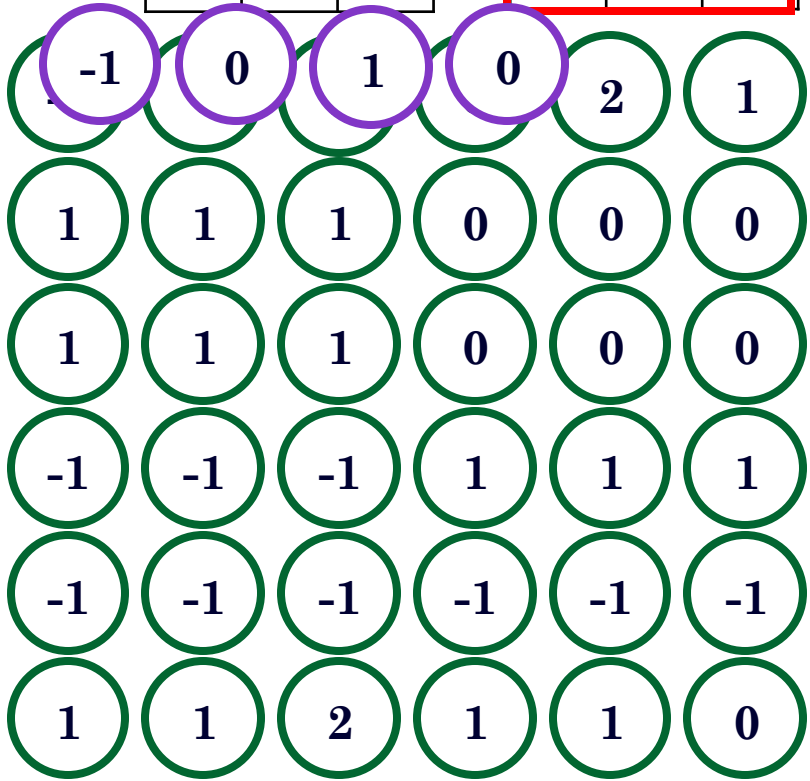
Horizontal line detector

-1	-1	-1
2	2	2
-1	-1	-1

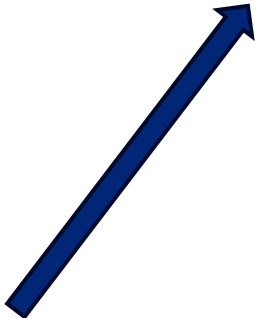
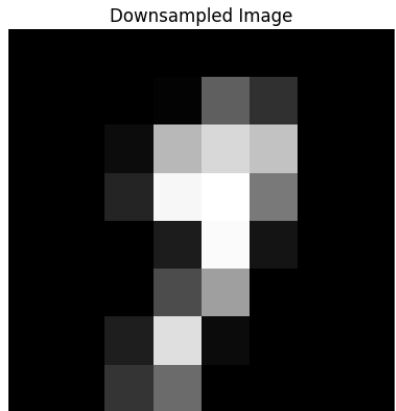
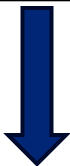
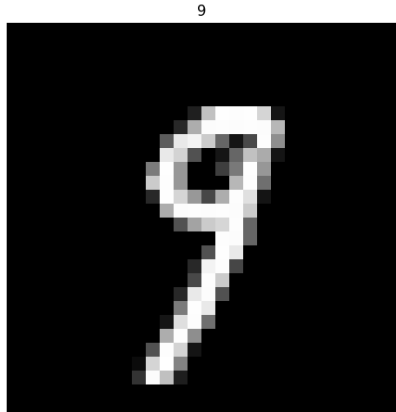
Vertical line detector

-1	2	-1
-1	2	-1
-1	2	-1

$\odot$



Example calculation of 2D convolution



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

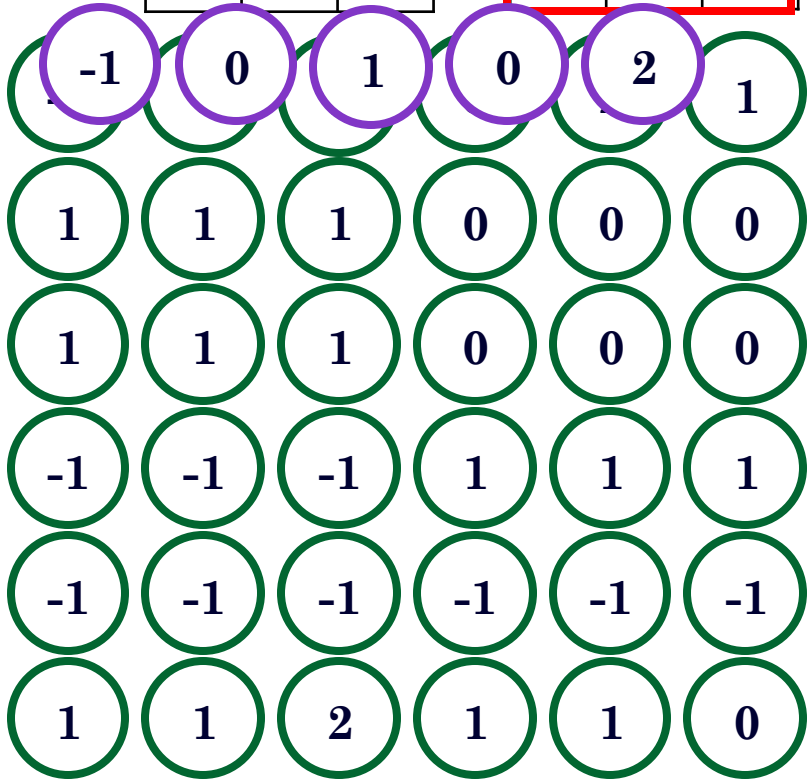
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

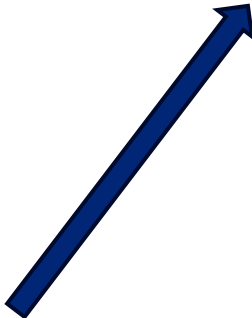
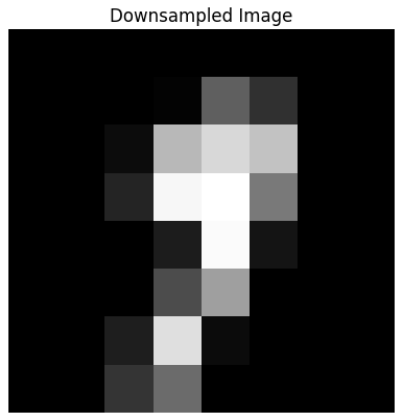
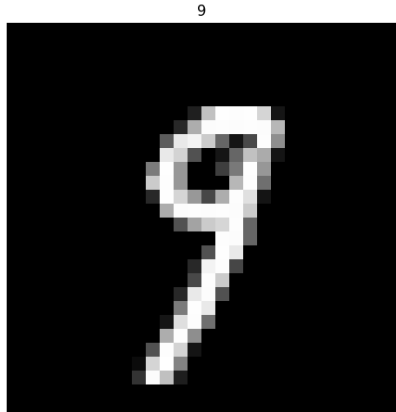
Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

$\odot$



Example calculation of 2D convolution



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Horizontal line detector

-1	-1	-1
2	2	2
-1	-1	-1

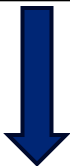
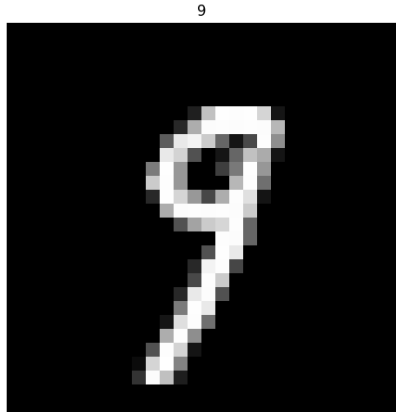
Vertical line detector

-1	2	-1
-1	2	-1
-1	2	-1

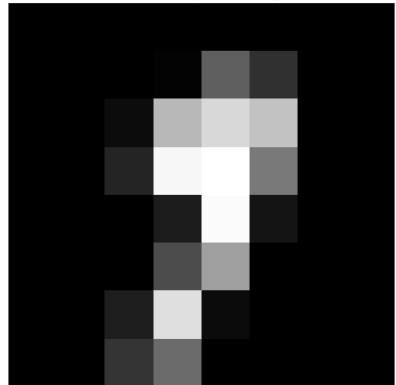
⊙

-1	0	1	0	2	-2
1	1	1	0	0	0
1	1	1	0	0	0
-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1
1	1	2	1	1	0

Example calculation of 2D convolution



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

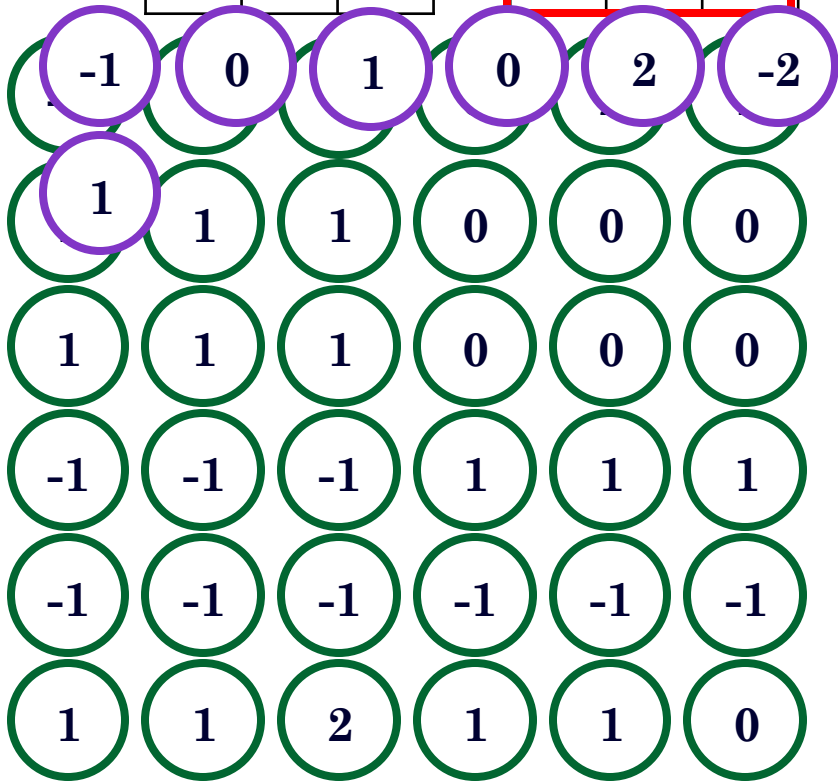
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

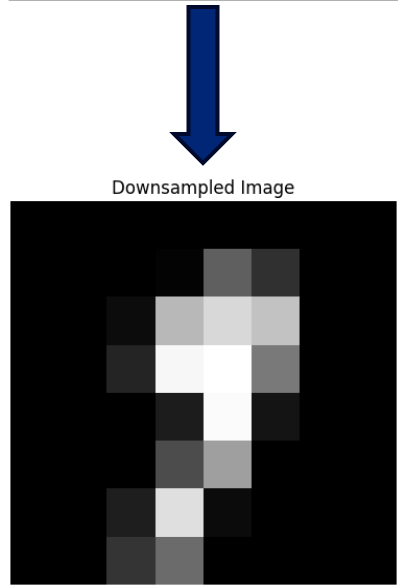
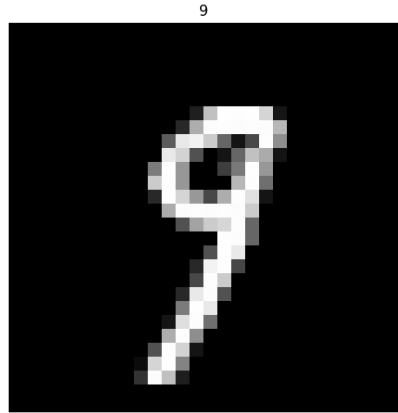
Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

$\odot$



Example calculation of 2D convolution



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

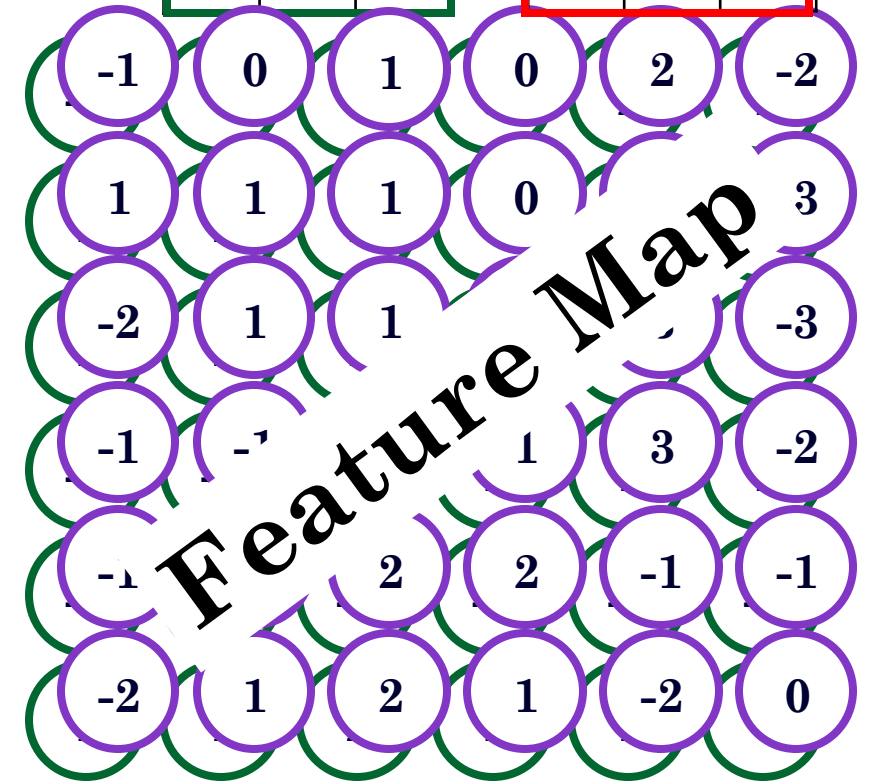
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

$\odot$



Mathematically: 1D convolution

- For “n”D convolution, we use an “n”D filter.
- So 1D convolution has a 1D filter[

Let's look at the 1-D case first. If a and b are two arrays,

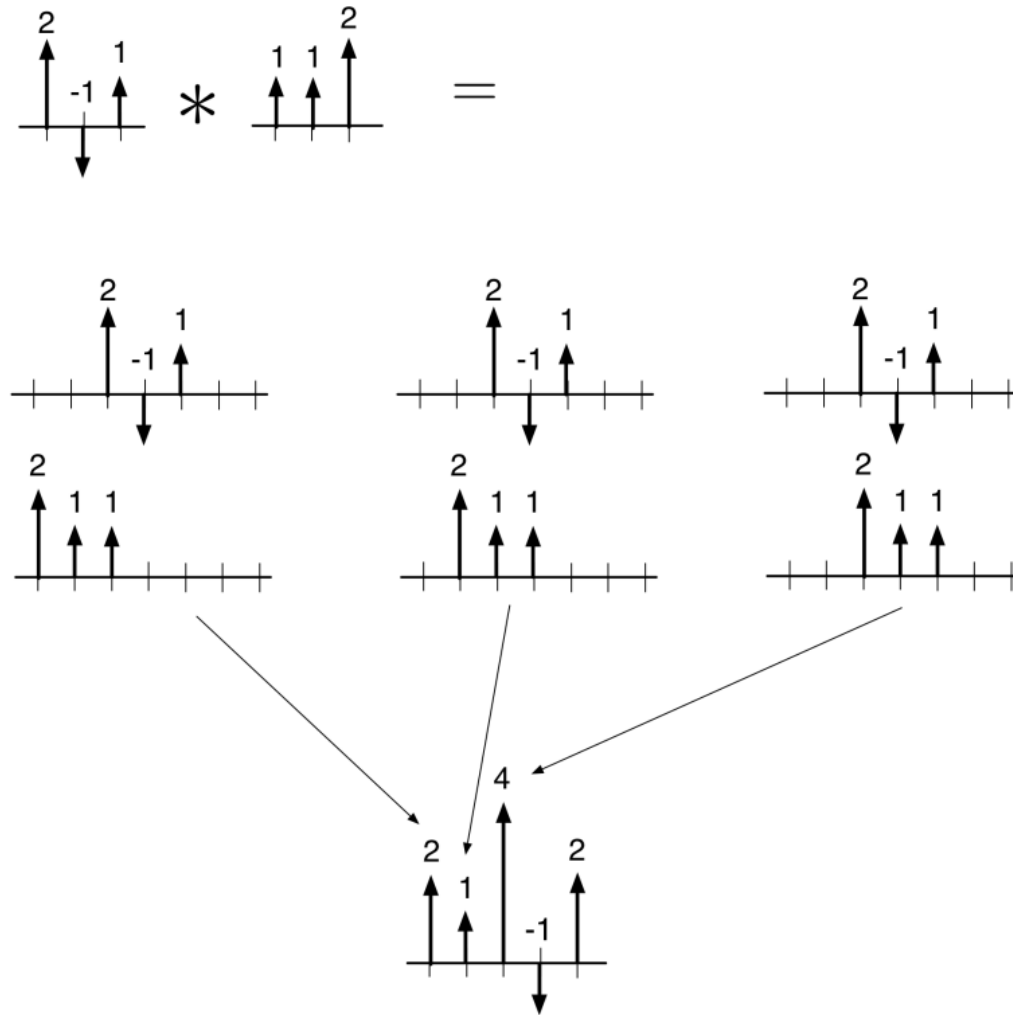
t 'th element of the convolution $\rightarrow (a * b)_t = \sum_{\tau} a_{\tau} b_{t-\tau}.$

*Note: “flip” the filter before doing the element-wise product.
(Here τ is the index of the filter element).*

1D convolution

$$(a * b)_t = \sum_{\tau} a_{\tau} b_{t-\tau}$$

Method 2: flip-and-filter



1D convolution

$$(a * b)_t = \sum_{\tau} a_{\tau} b_{t-\tau}$$

Convolution can also be viewed as matrix multiplication:

$$(2, -1, 1) * (1, 1, 2) = \overset{W_k}{\begin{pmatrix} 1 & & \\ 1 & 1 & \\ 2 & 1 & 1 \\ & 2 & 1 \\ & & 2 \end{pmatrix}} \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix}$$

Note that the parameter, matrix W_k , has size 5×3 , which means it has 15 entries, yet there are only 3 parameters. This is what causes Convnets to have relatively few parameters.

1D convolution

Some properties of convolution:

- Commutativity

$$a * b = b * a$$

- Linearity

$$a * (\lambda_1 b + \lambda_2 c) = \lambda_1 a * b + \lambda_2 a * c$$

From 1D to 2D convolution

2-D convolution is defined analogously to 1-D convolution.

If A and B are two 2-D arrays, then:

$$(A * B)_{ij} = \sum_s \sum_t A_{st} B_{i-s, j-t}.$$

2D convolution

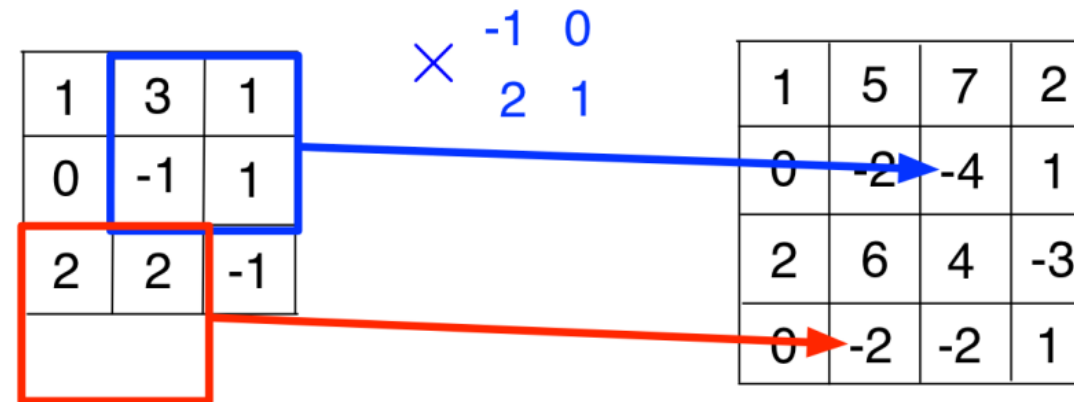
$$(A * B)_{ij} = \sum_s \sum_t A_{st} B_{i-s, j-t}$$

Method 2: Flip-and-Filter

1	3	1
0	-1	1
2	2	-1

 $*$

1	2
0	-1



2D convolution

Can we get intuition about the numeric values of some kernels/filters?



*

0	1	0
1	4	1
0	1	0

2D convolution

Can we get intuition about the numeric values of some kernels/filters?



*

0	1	0
1	4	1
0	1	0



"blurring" filter

2D convolution

Can we get intuition about the numeric values of some kernels/filters?



*

1	0	-1
2	0	-2
1	0	-1

2D convolution

Can we get intuition about the numeric values of some kernels/filters?



*

1	0	-1
2	0	-2
1	0	-1



"oriented edges"

2D convolution

Can we get intuition about the numeric values of some kernels/filters?



*

0	-1	0
-1	4	-1
0	-1	0

2D convolution

Can we get intuition about the numeric values of some kernels/filters?



*

0	-1	0
-1	4	-1
0	-1	0



"sharpen"

2D convolution

But remember, in a neural network, we're going to learn these filters from the data, using backprop+



*

w_{11}	w_{12}	w_{13}
w_{21}	w_{22}	w_{23}
w_{31}	w_{32}	w_{33}



2D convolution

How big is the output of a convolutional layer?

Assume:

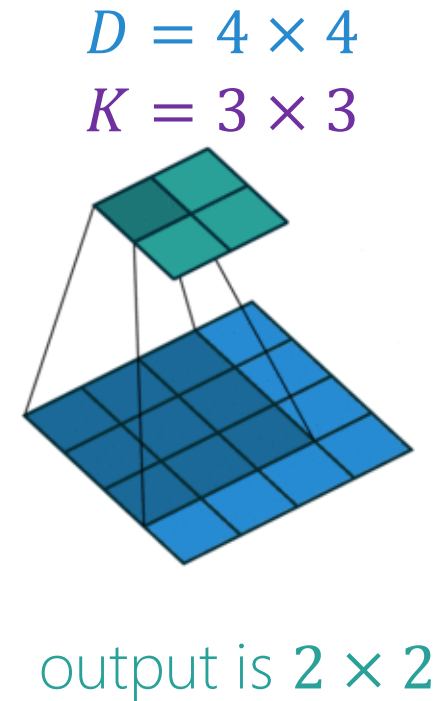
- Image is $D \times D$.
- N filters each of size $K \times K$.
- Stride is 1 (don't skip any pixels).
- No zero-padding.

Then output from one filter has size:

$$(D - K + 1) \times (D - K + 1).$$

For all filters,

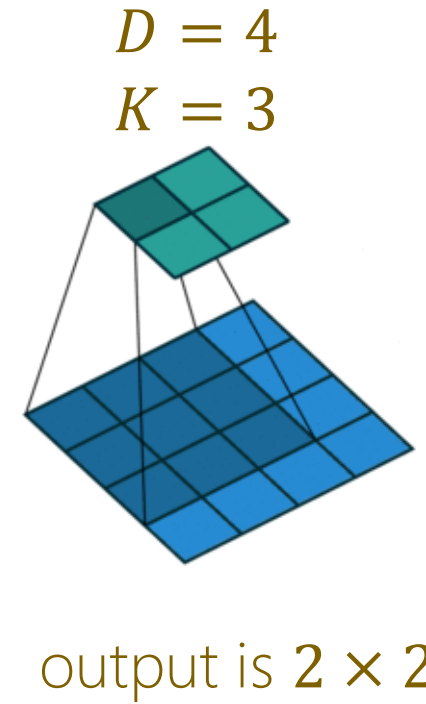
$$N(D - K + 1)(D - K + 1)$$

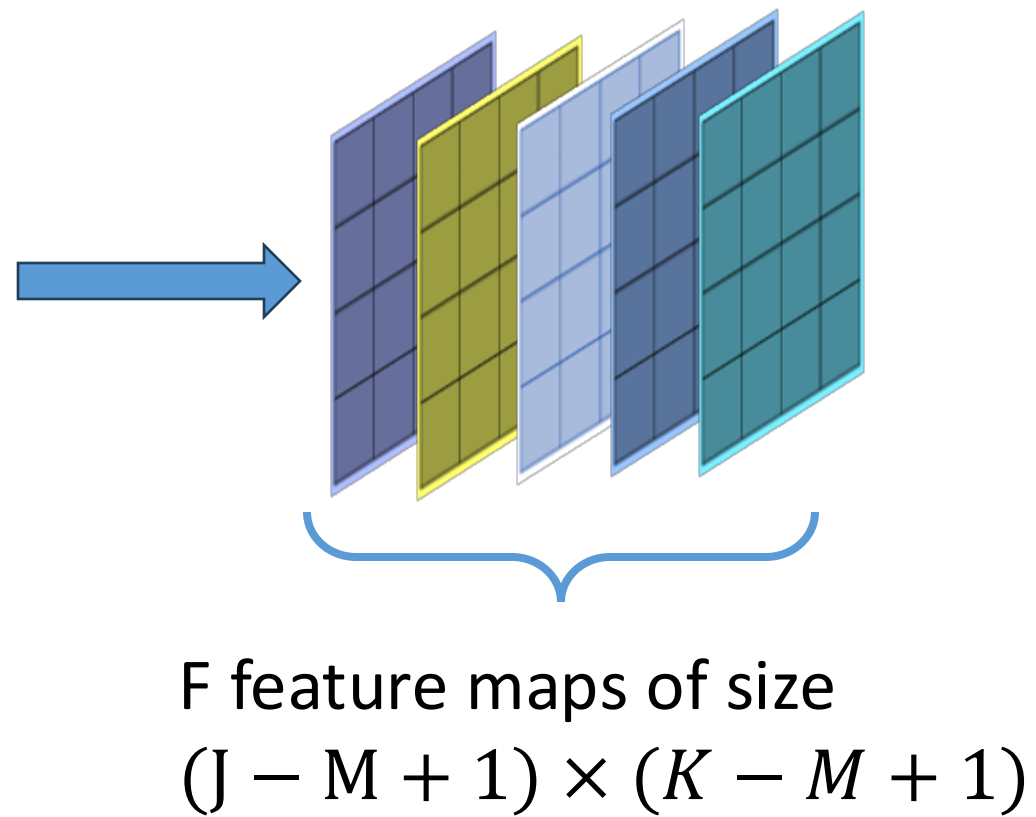
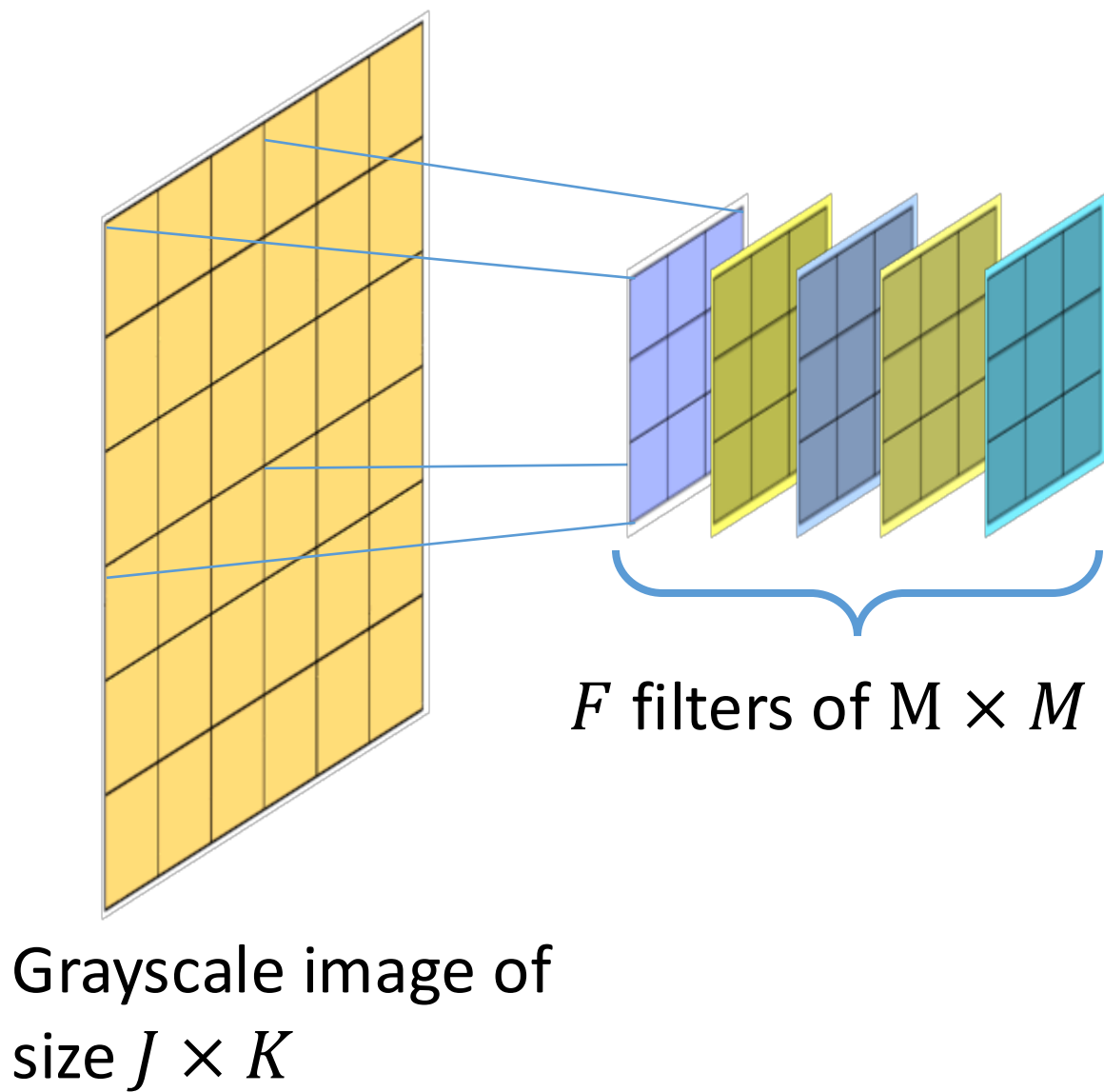


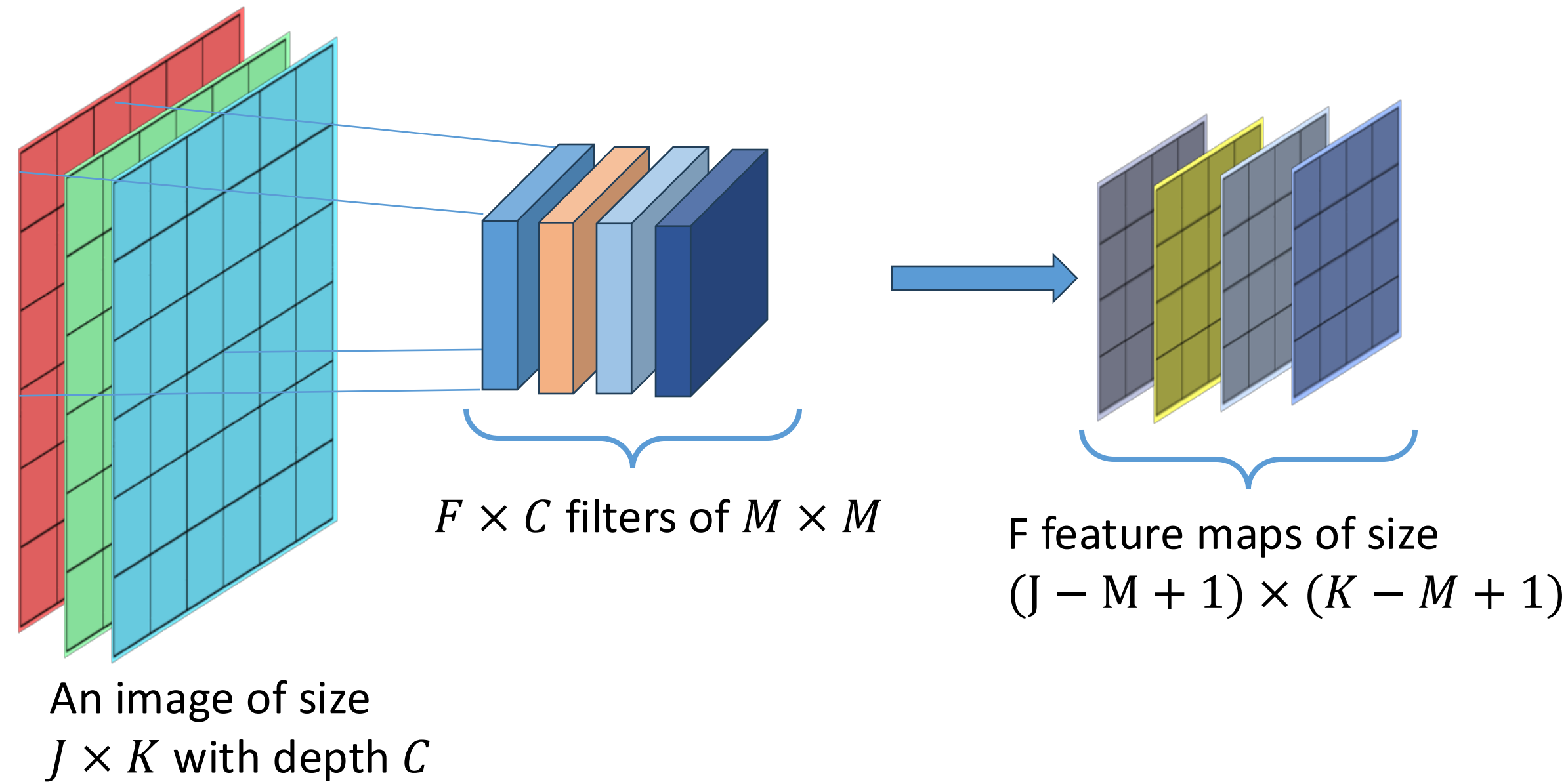
2D convolution

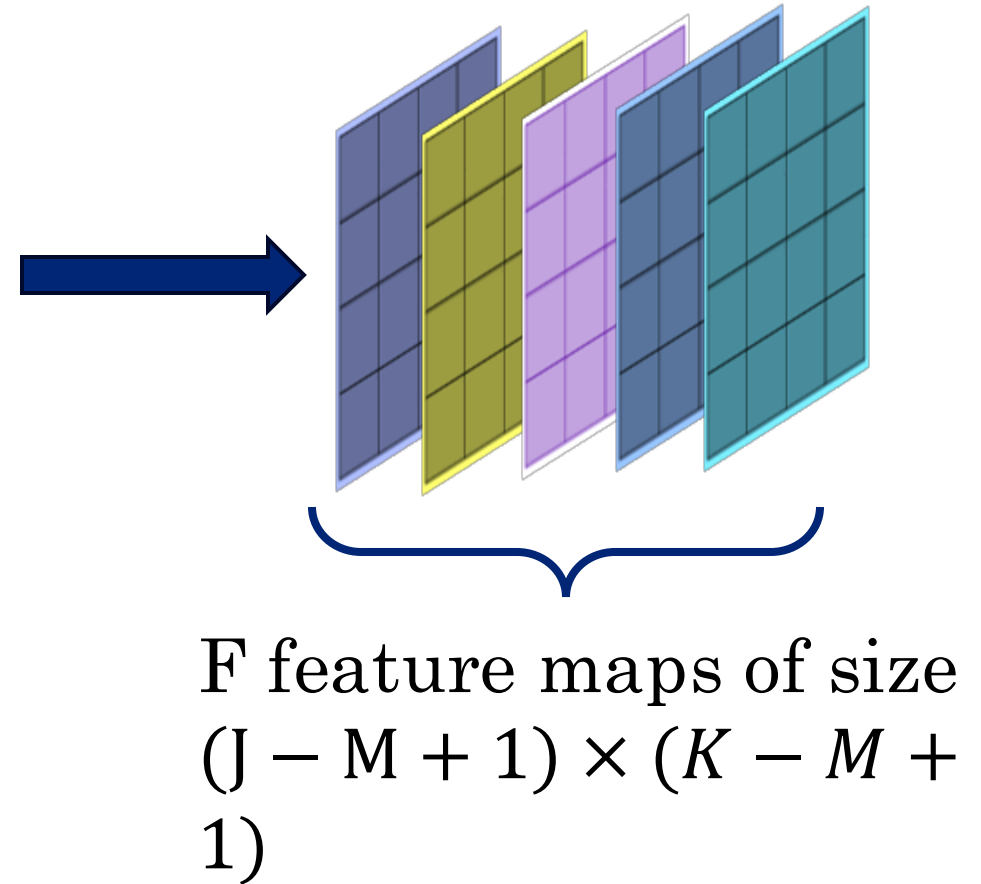
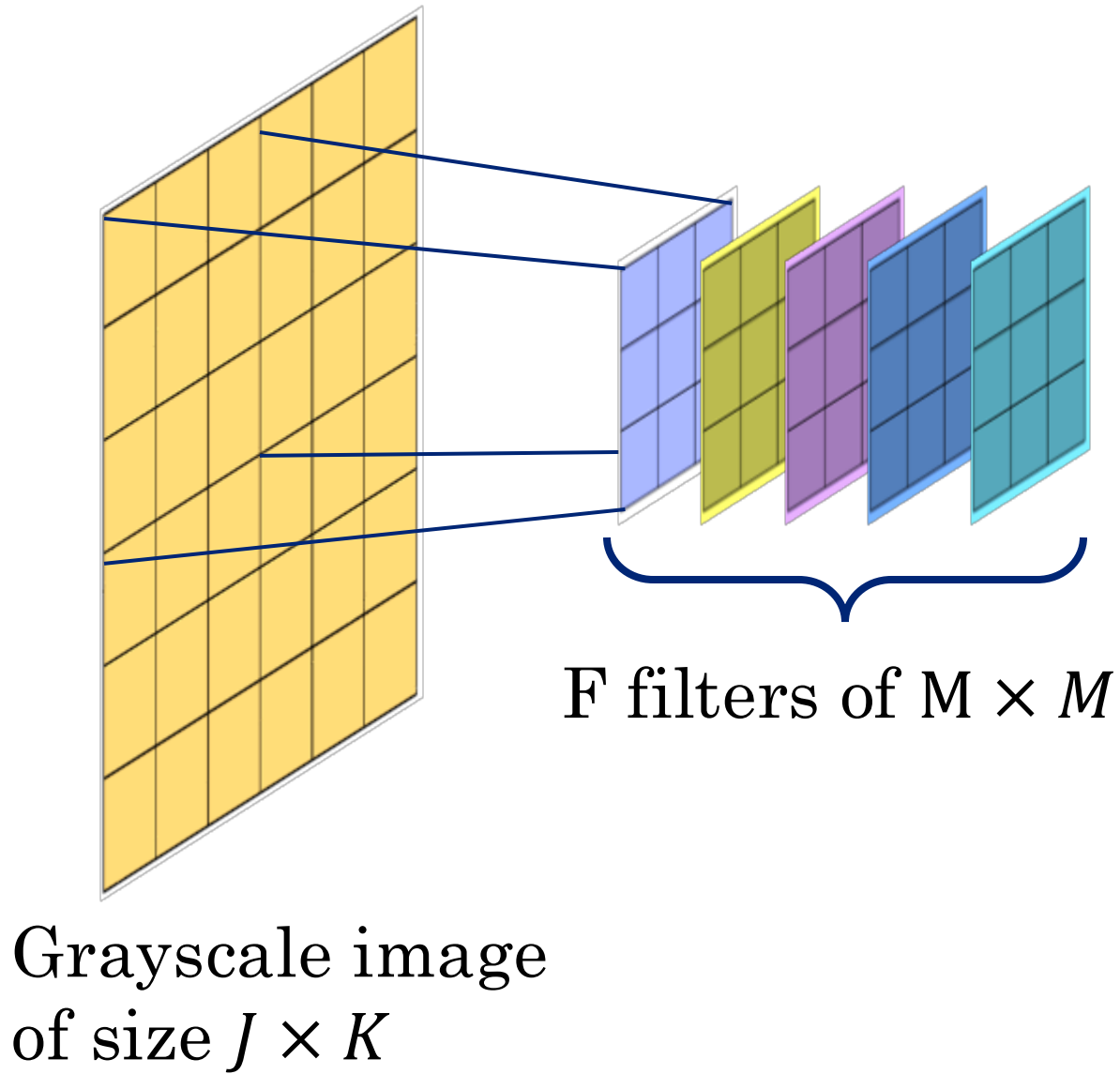
What is the time complexity for one convolutional layer (square filters & image)?

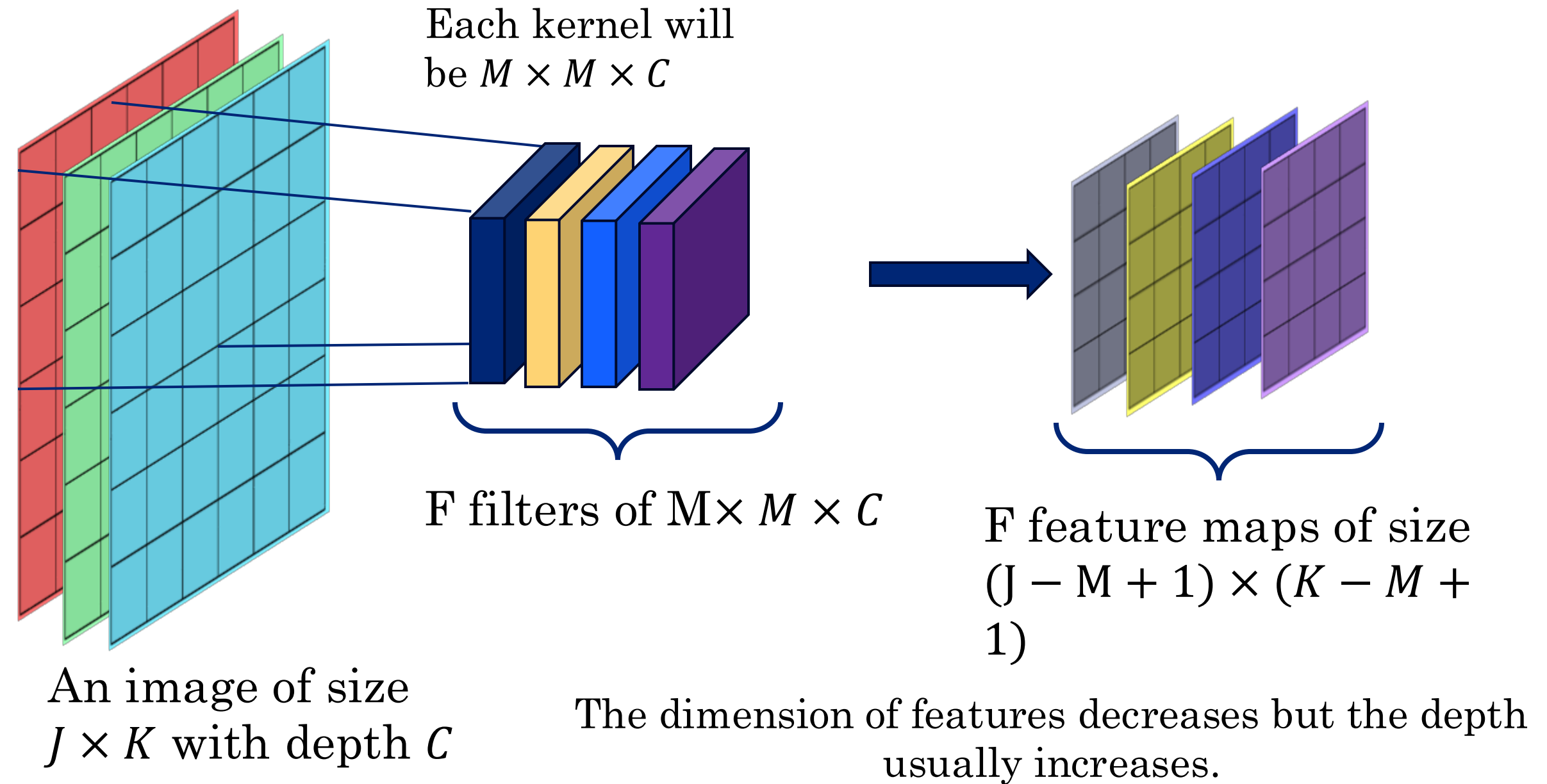
- Output from one filter has size $(D - K + 1)(D - K + 1) = (D - K + 1)^2$
- This takes $K^2(D - K + 1)^2$
- So for all F filters in one layer, we have $FK^2(D - K + 1)^2$











What If We Want to Keep the Same Image Dimensions?

- We can use padding.

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	1	1	1	0	0	0	0
0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line
detector

$\odot$

0

What If We Want to Keep the Same Image Dimensions?

- We can use padding.

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	1	1	1	0	0	0	0
0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

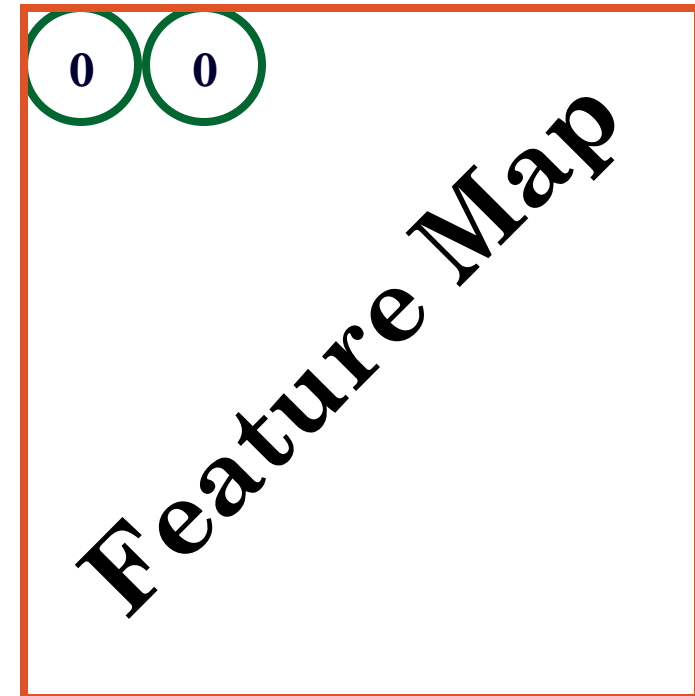
Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

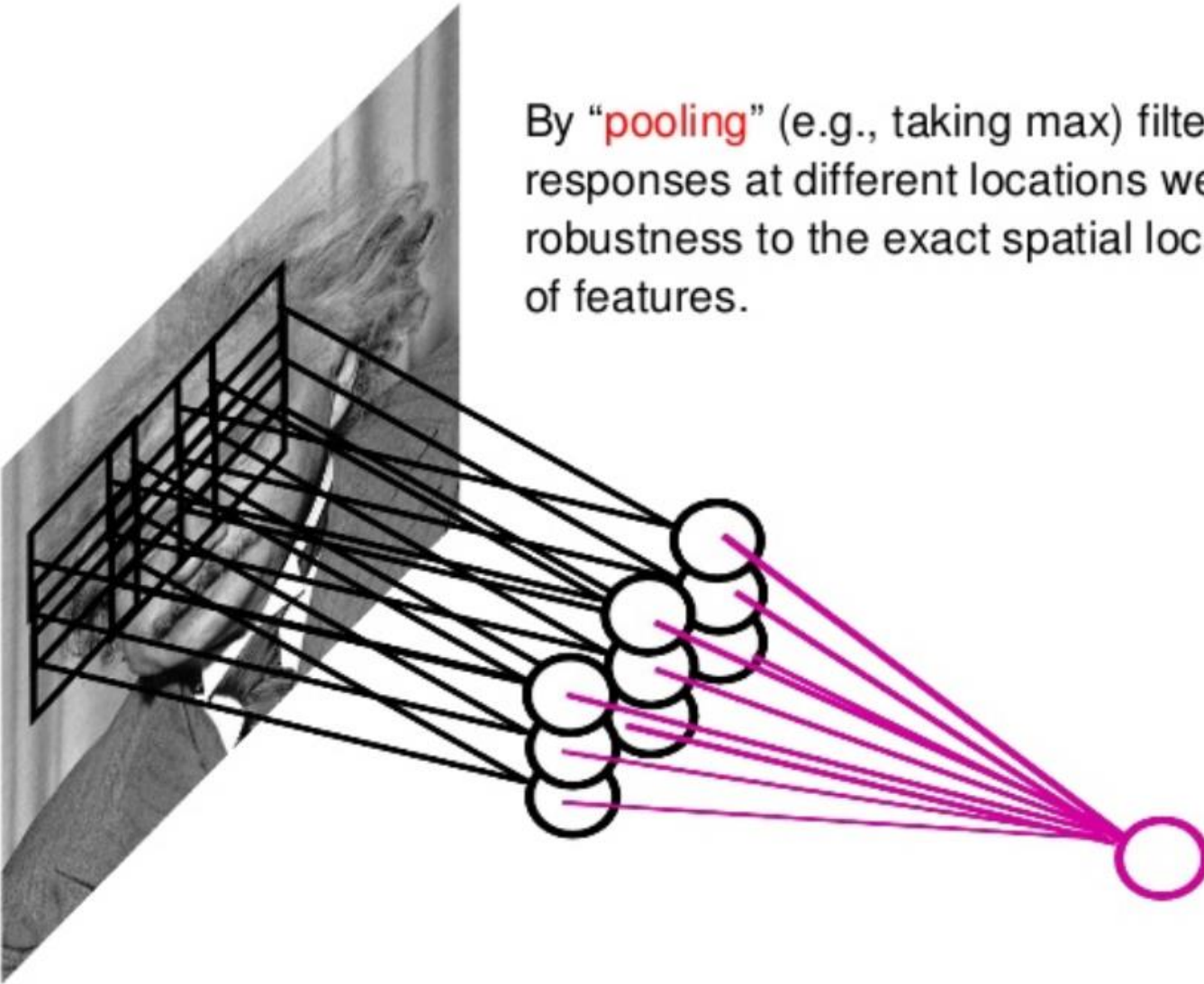
$\odot$

Horizontal line
detector



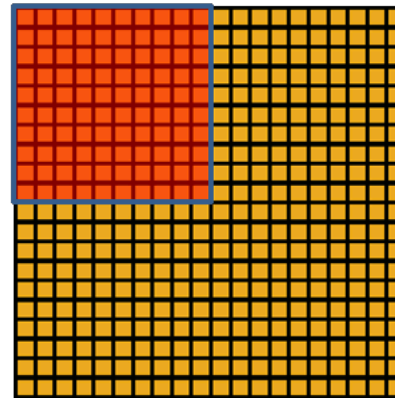
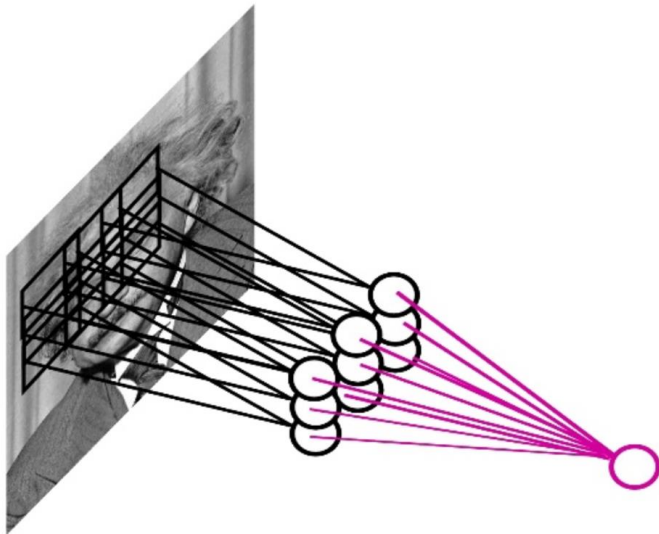
ConvNets—"Pooling" layers

By "**pooling**" (e.g., taking max) filter responses at different locations we gain robustness to the exact spatial location of features.

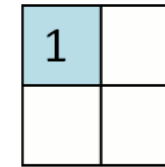


ConvNets—"Pooling" layers

- Pooling layers reduce the size of the convolutional output (the "feature maps").
- Also build in invariances to small transformations (e.g. if face is not perfectly centered when doing face detection).



Convolved
feature



Pooled
feature

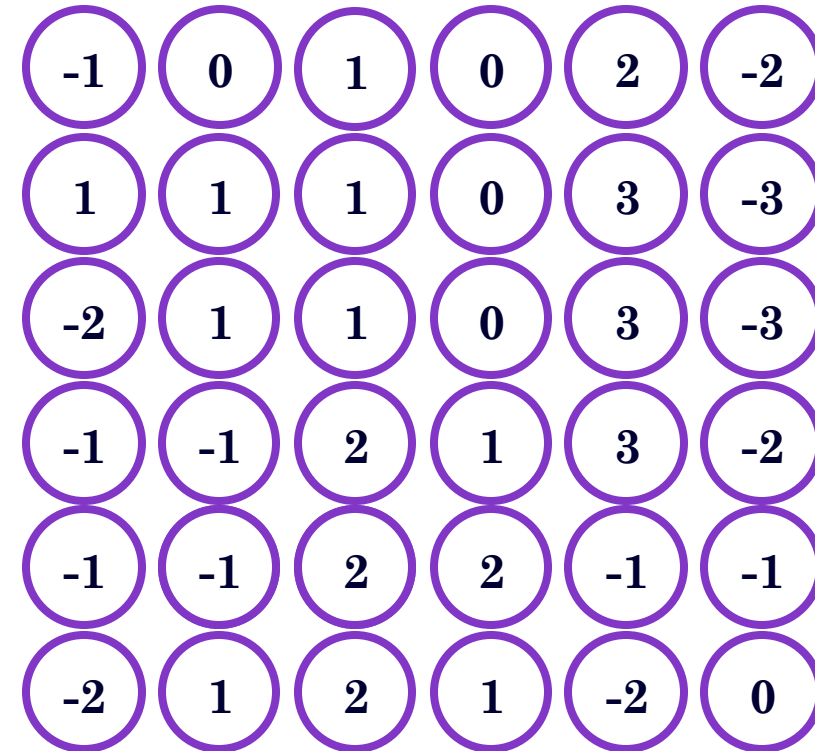
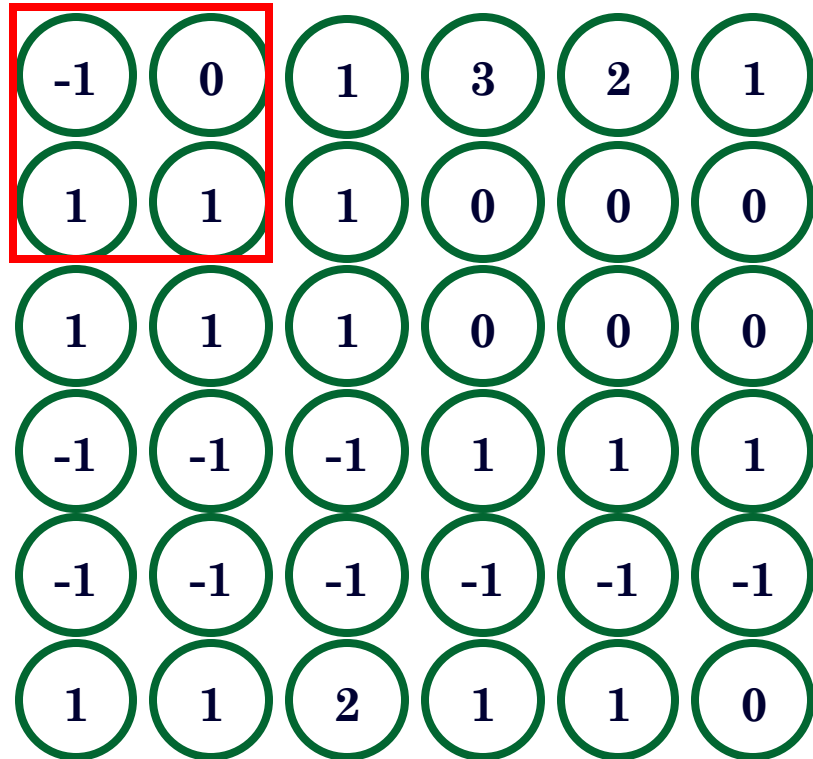
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



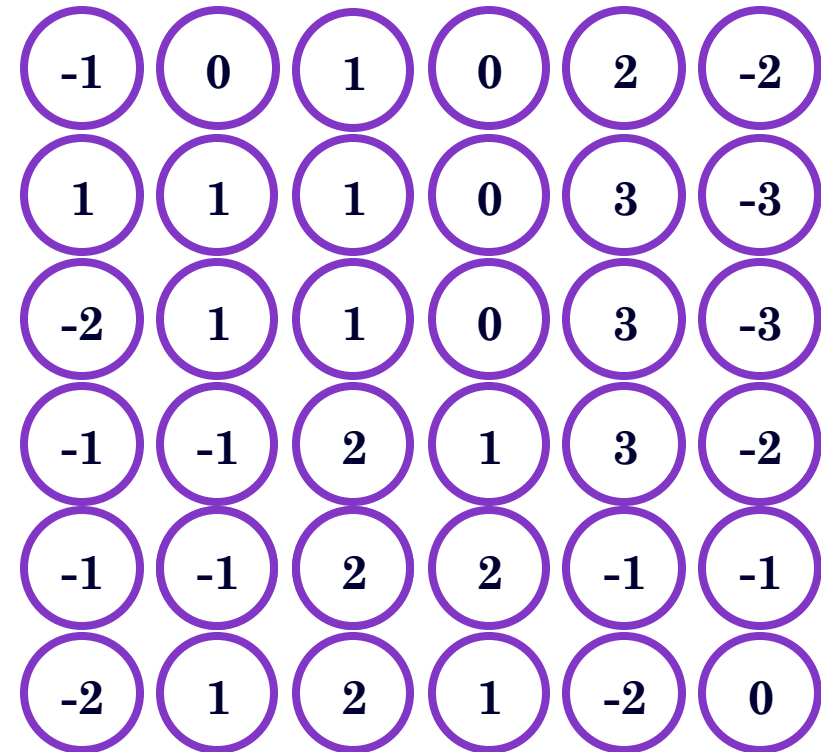
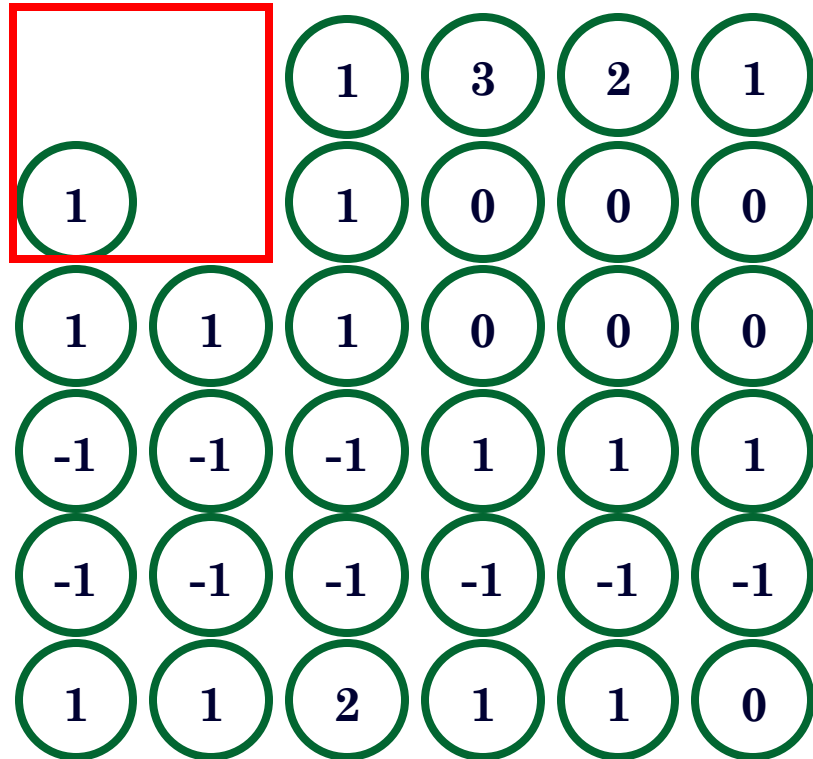
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



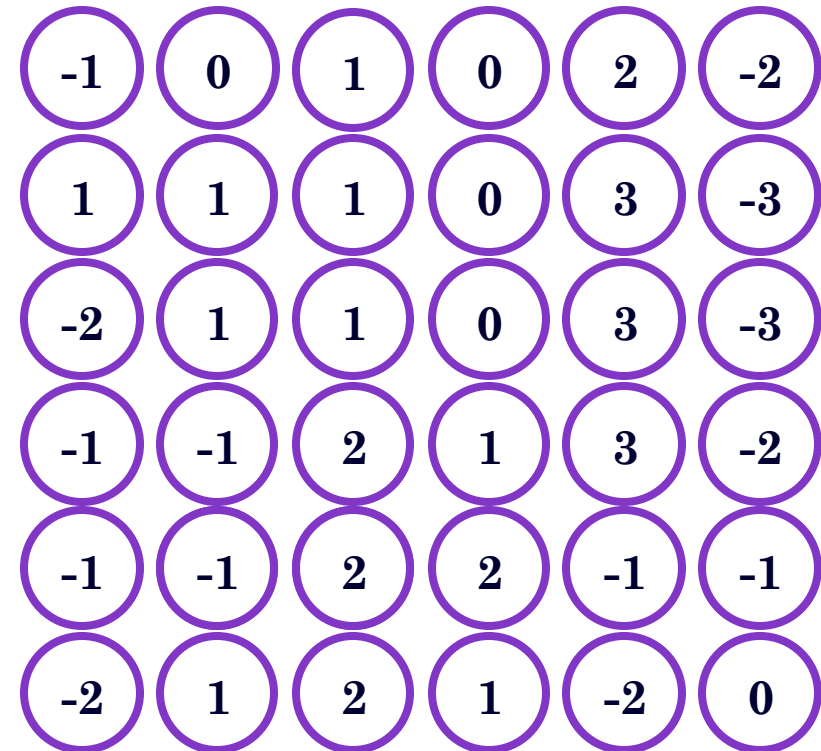
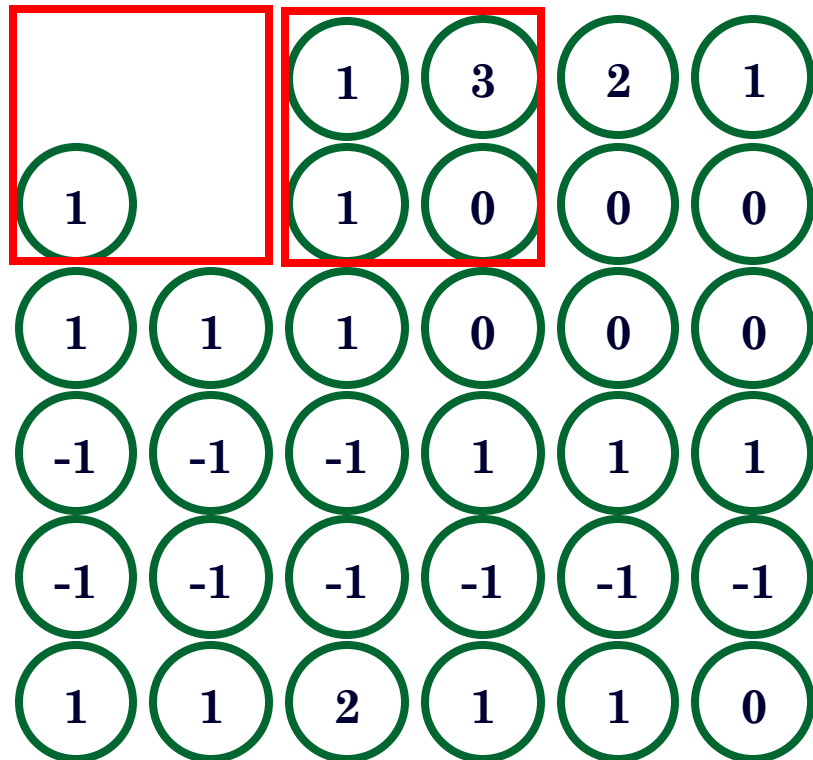
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



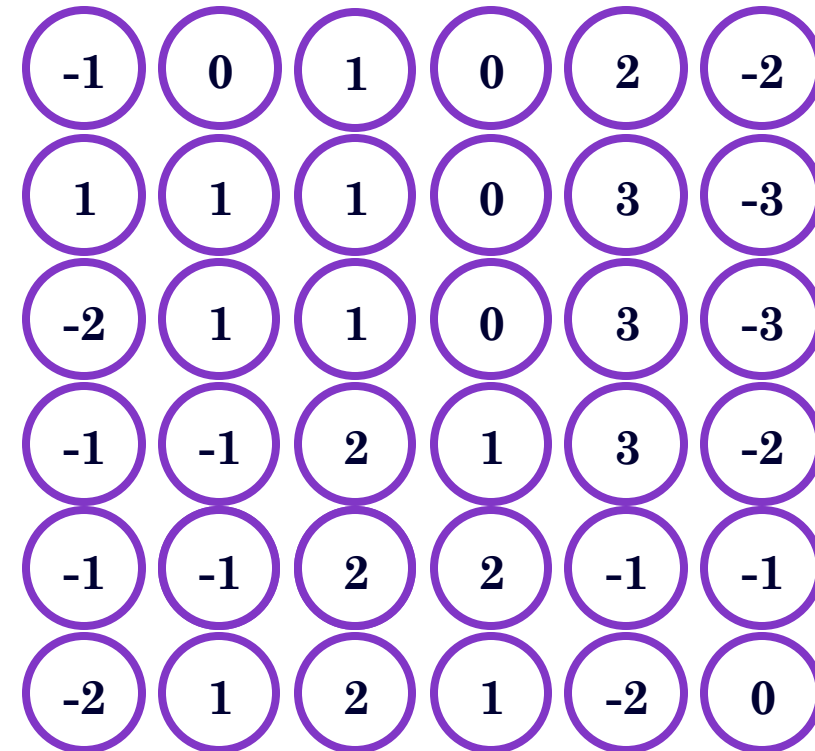
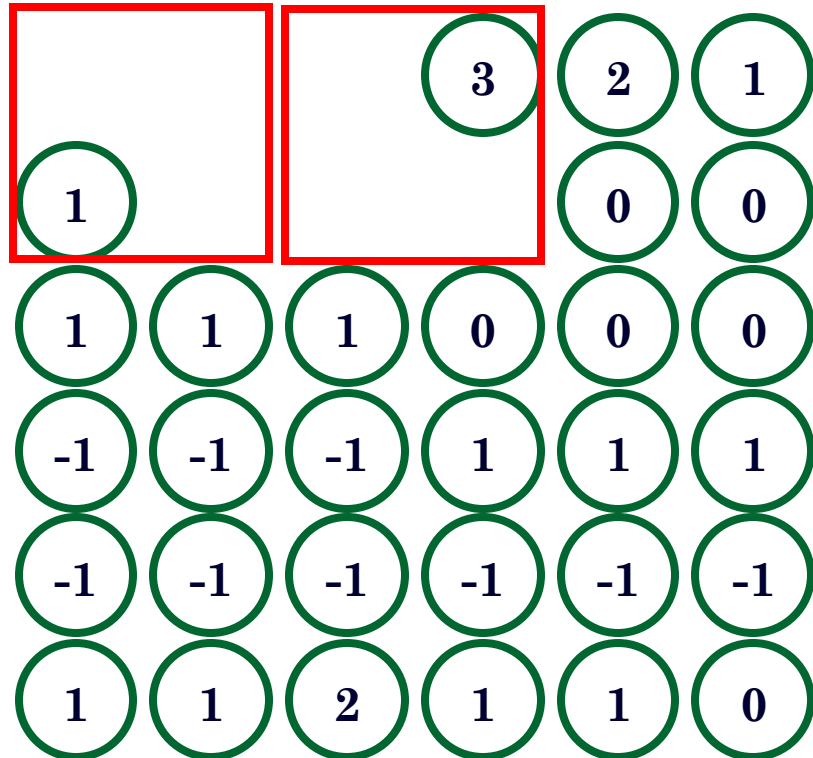
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



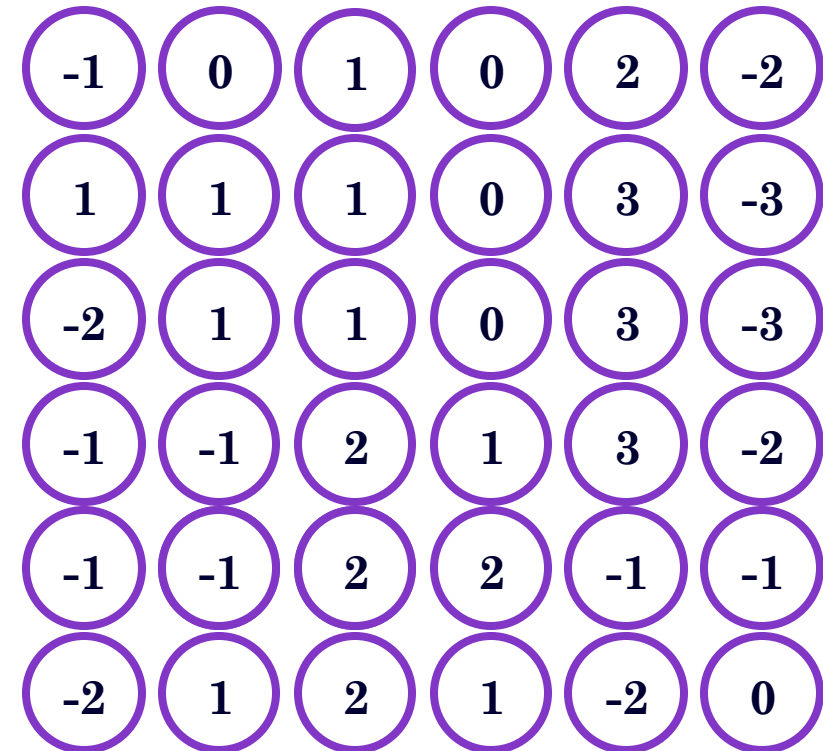
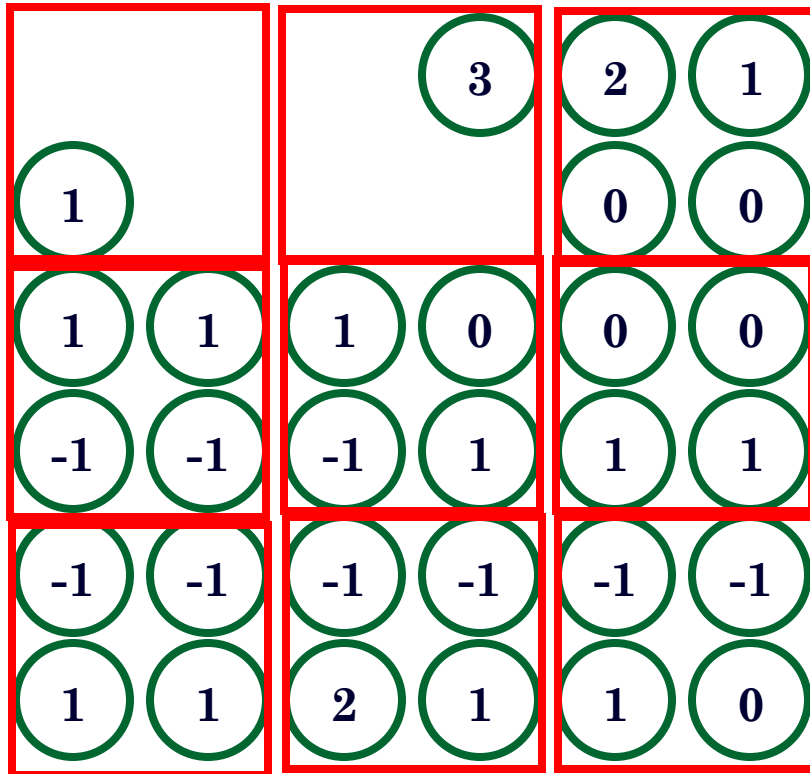
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



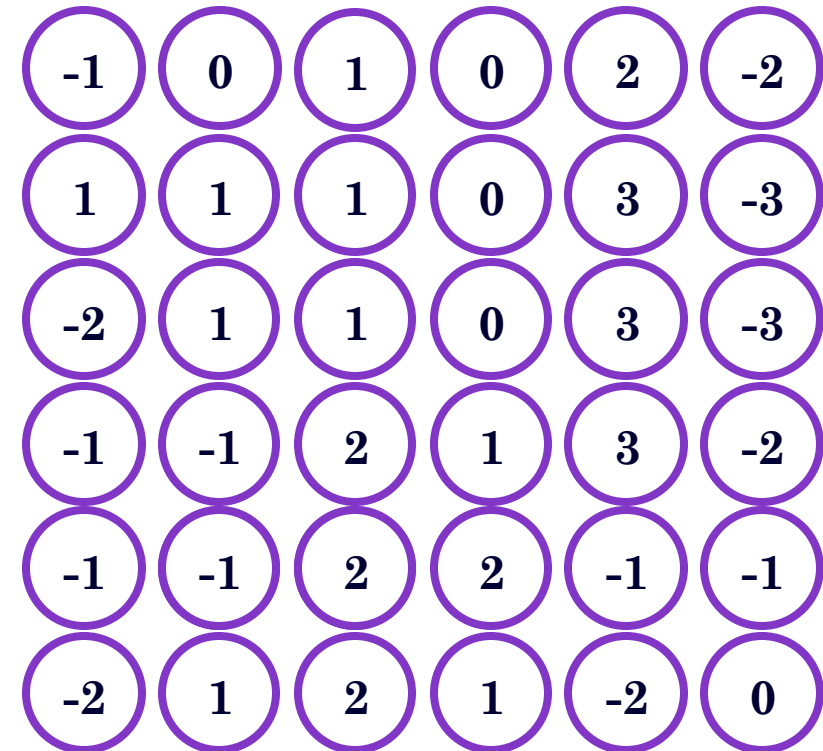
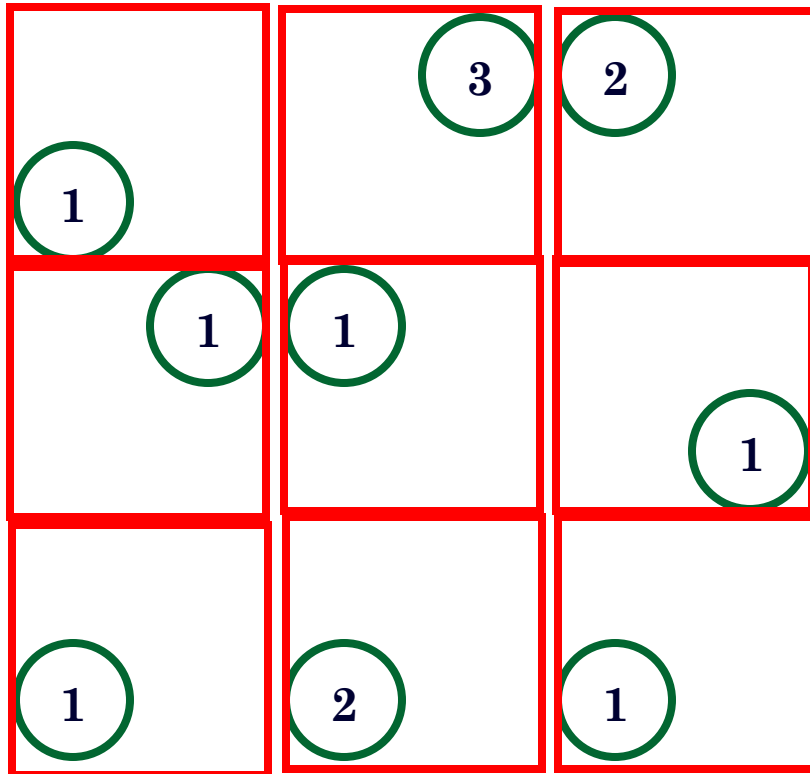
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



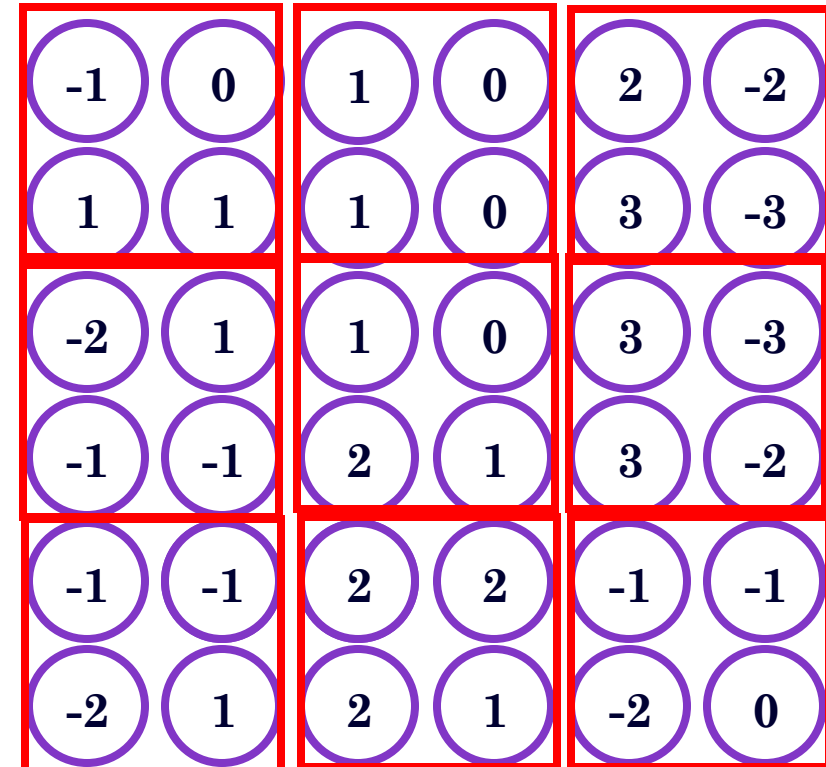
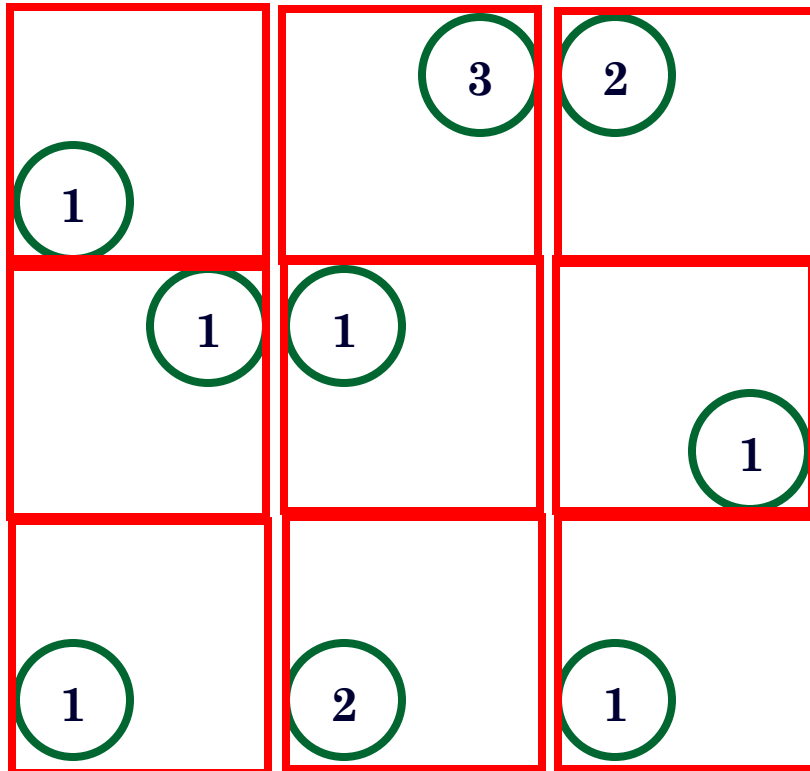
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



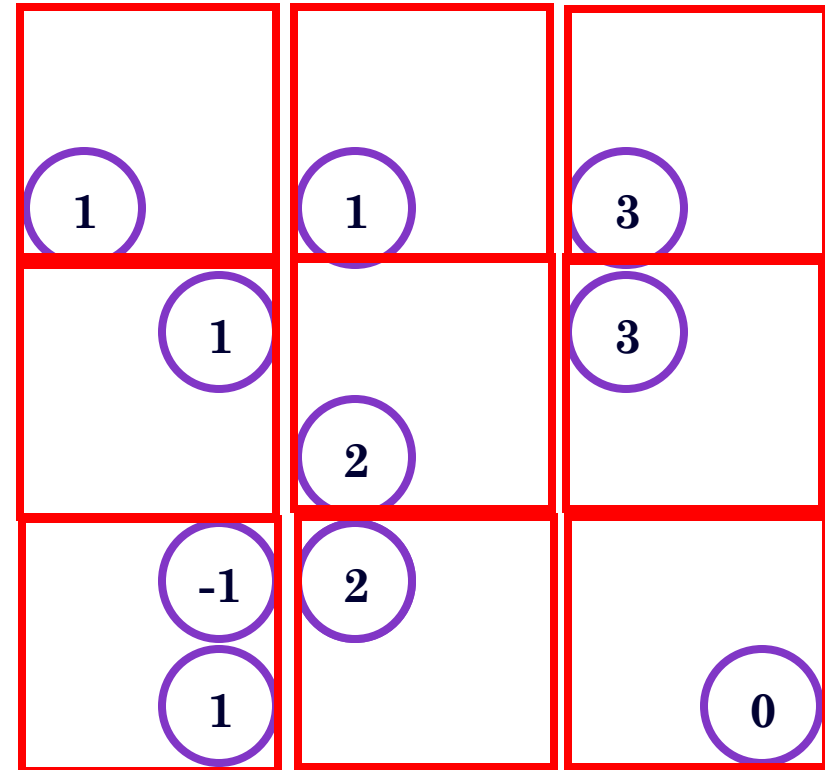
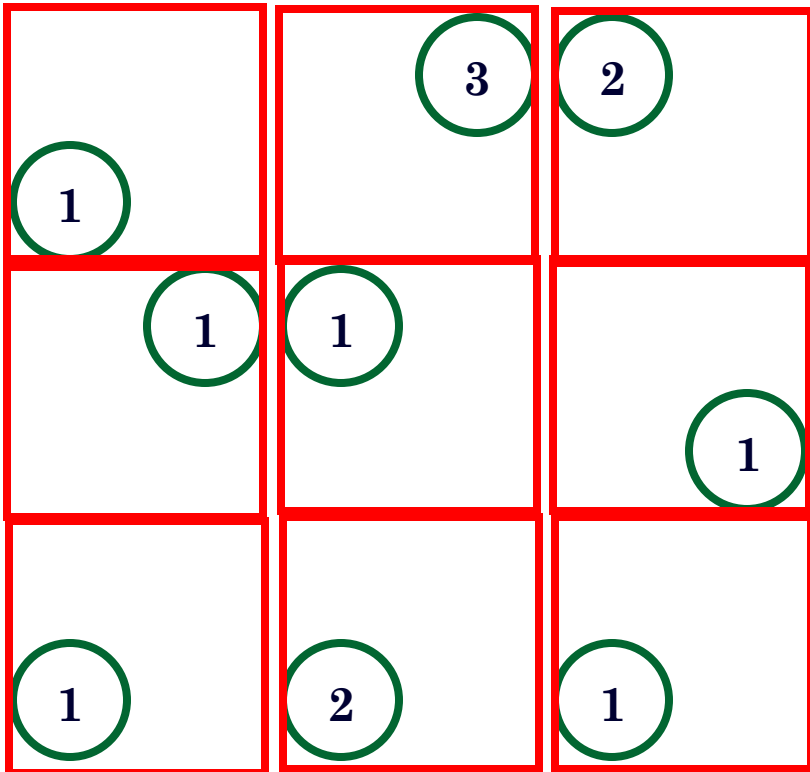
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



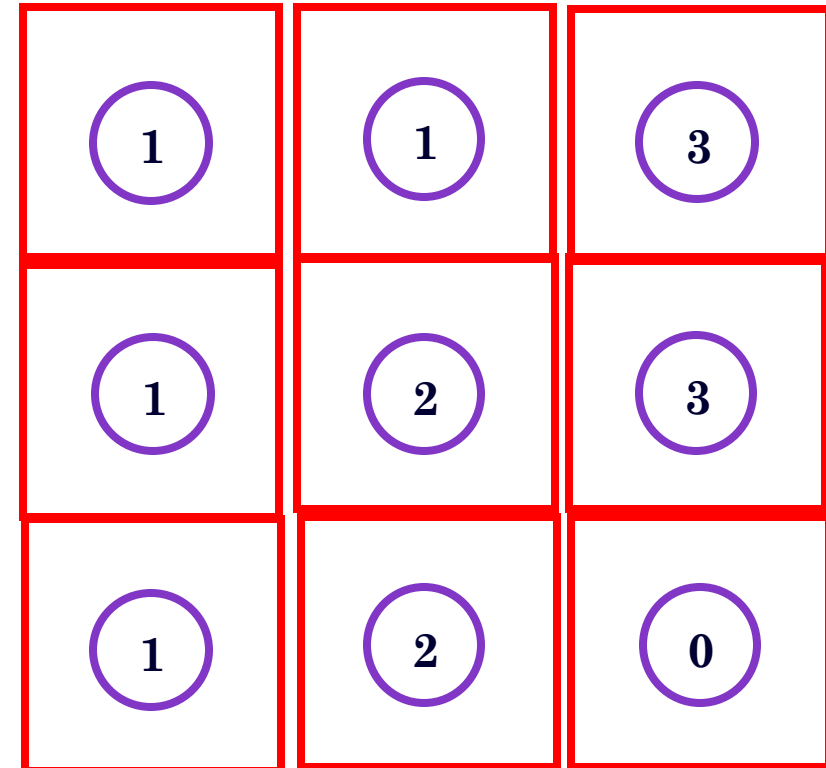
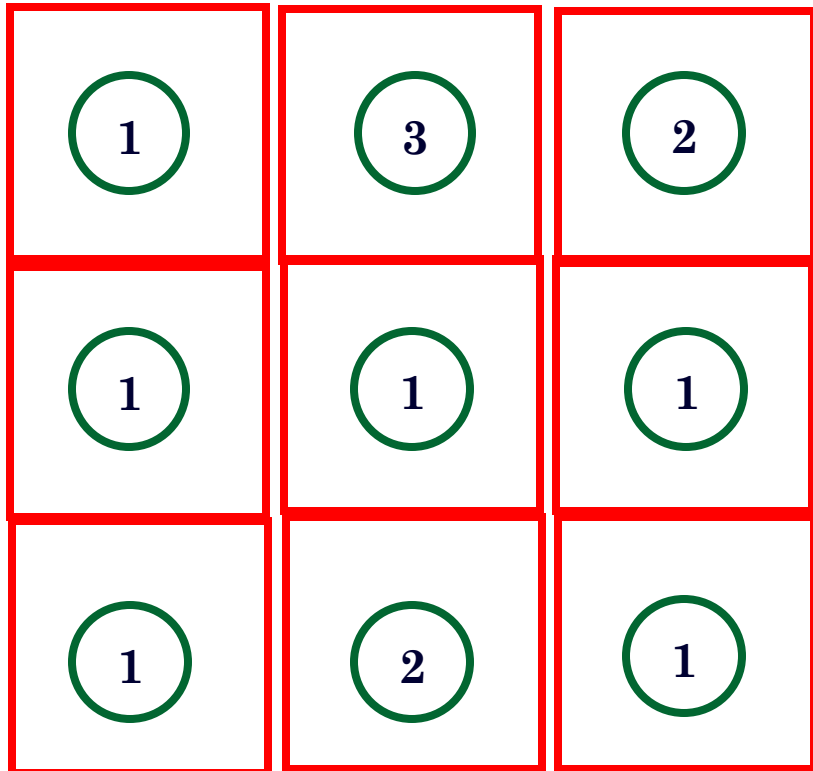
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



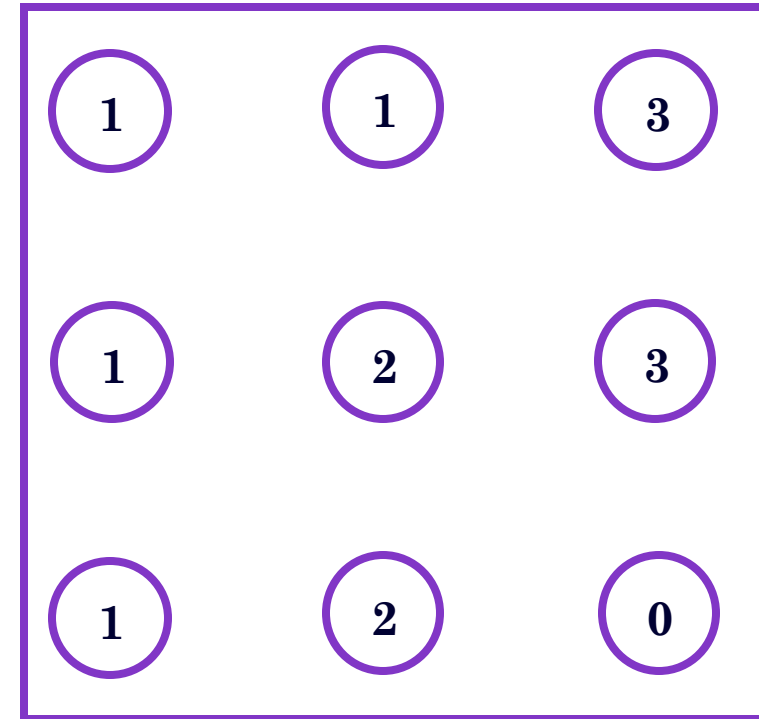
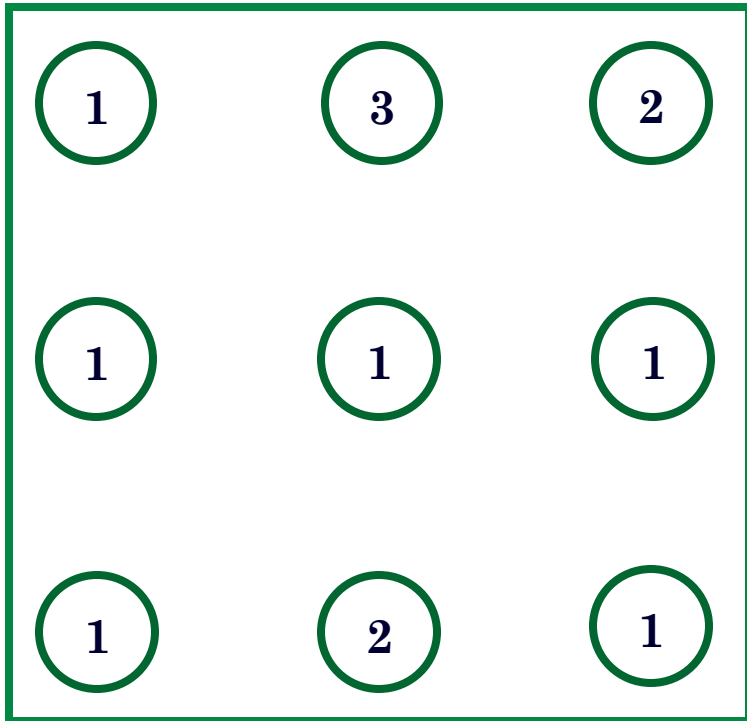
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

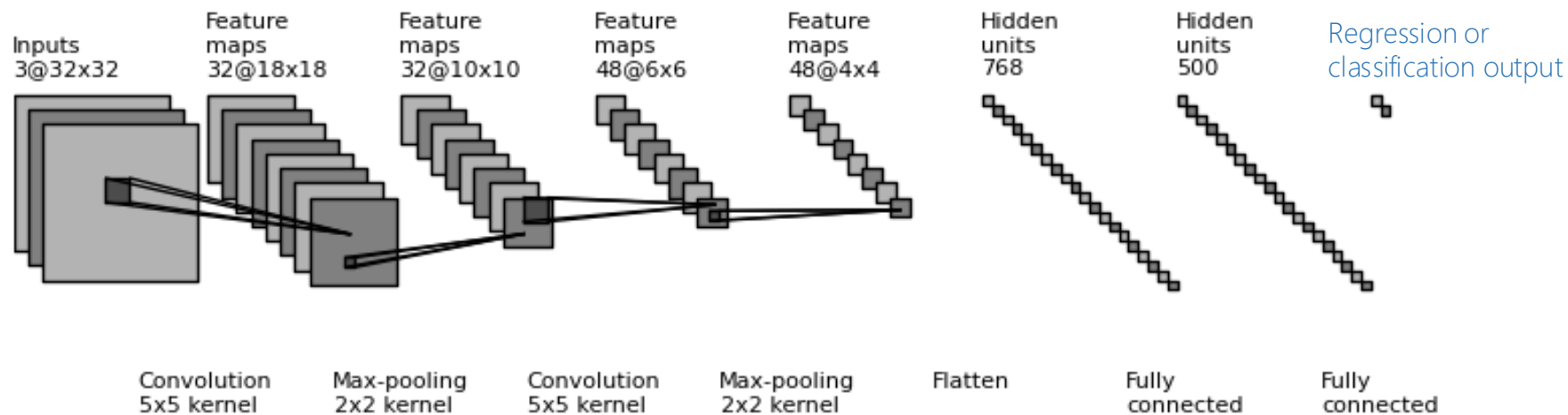
“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



ConvNets-stacking different layers

After feature construction layers (convolution+max pooling), typically a few (e.g. 2) fully connected layers are added in before pushing their results through a classifier (e.g. softmax), or regressor (e.g. linear regression).

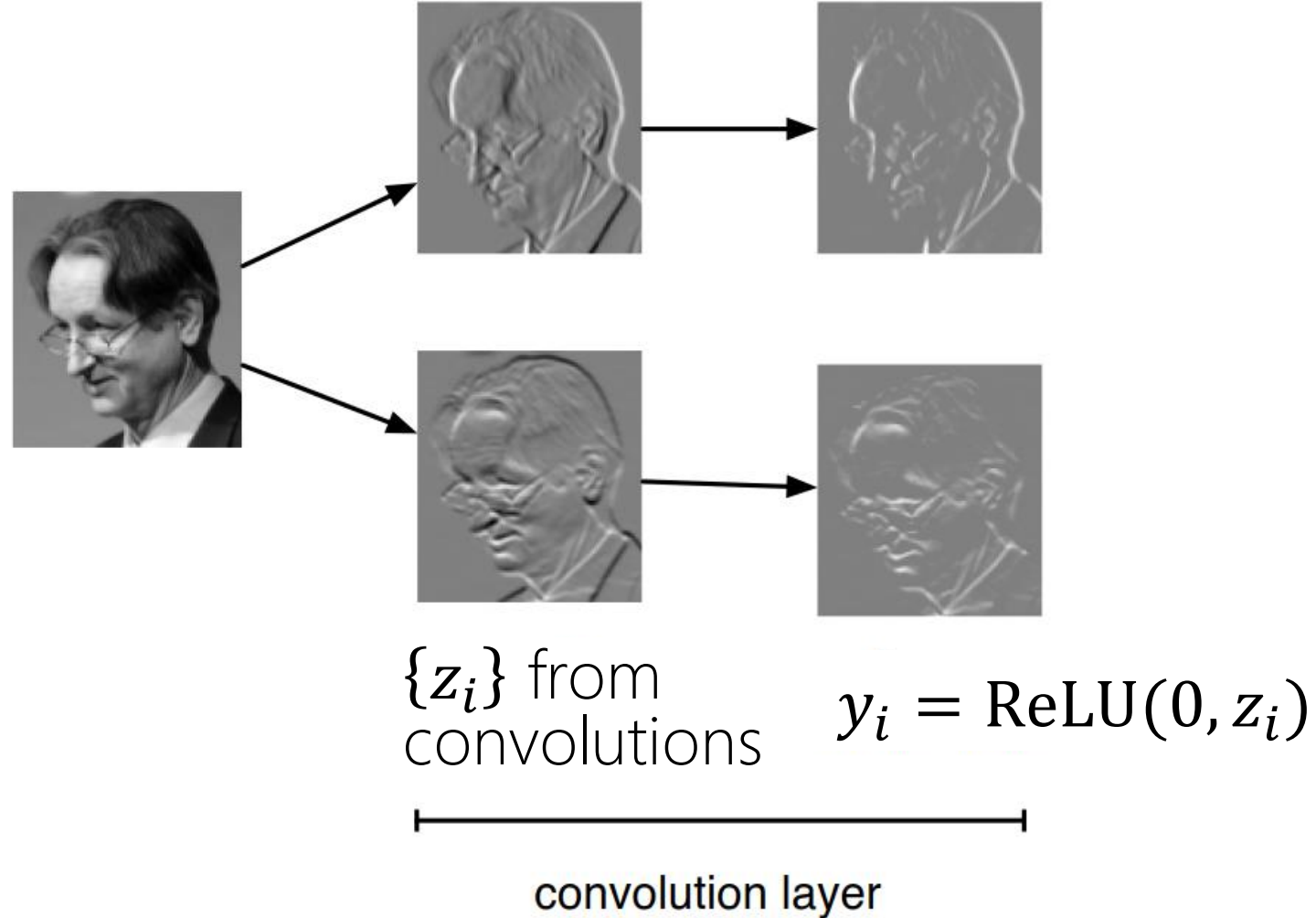


Convolutional Neural Networks (ConvNets)

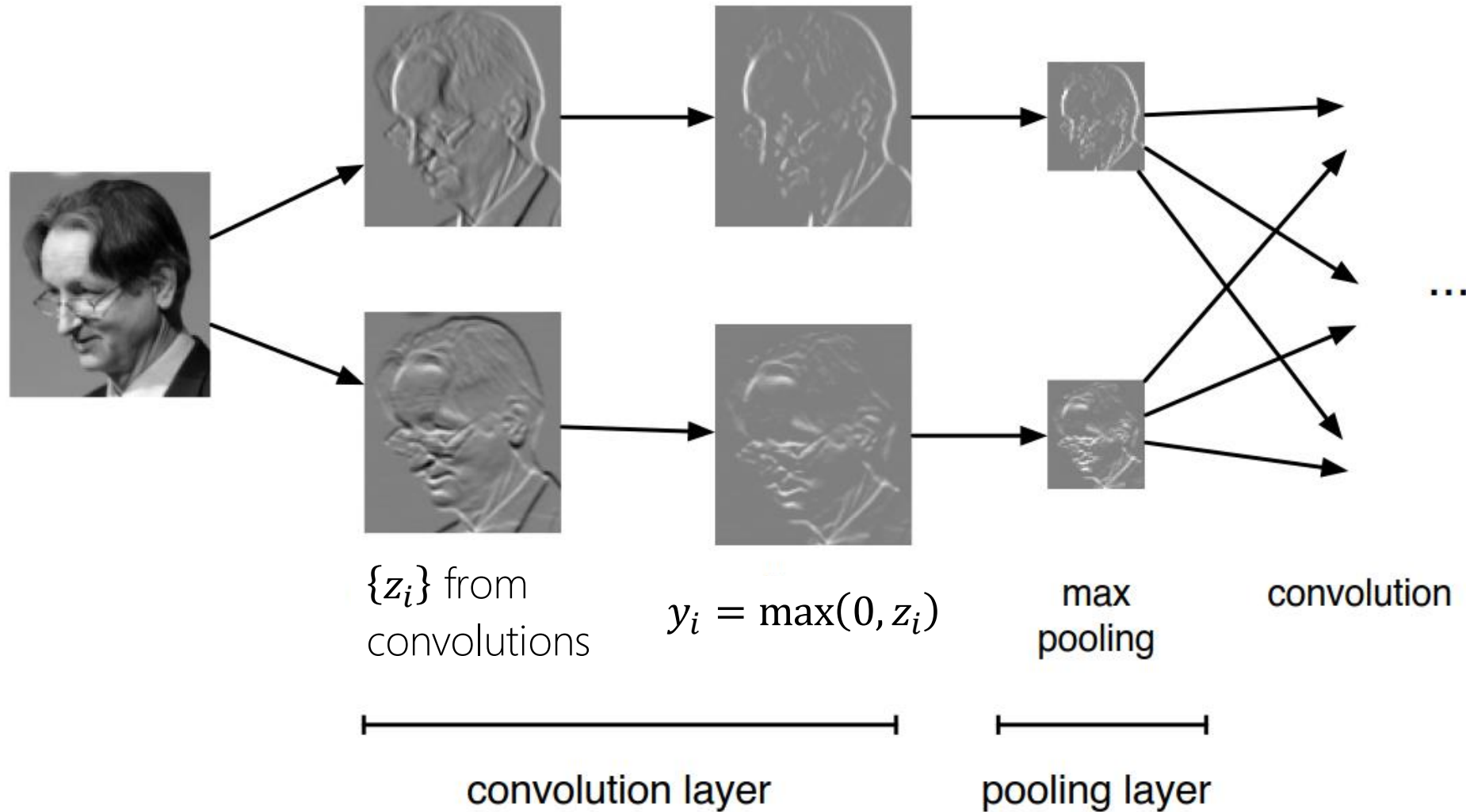
Two kinds of layers stacked “deeply”:

- i. “detection”/2D convolution layers (what we just saw), consisting of a set of filters; output is a set of “feature maps”.
- ii. pooling layers which sub-sample convolutional output; makes smaller versions of the “feature maps” (e.g. take the max).

ConvNets also need non-linearities (activations)

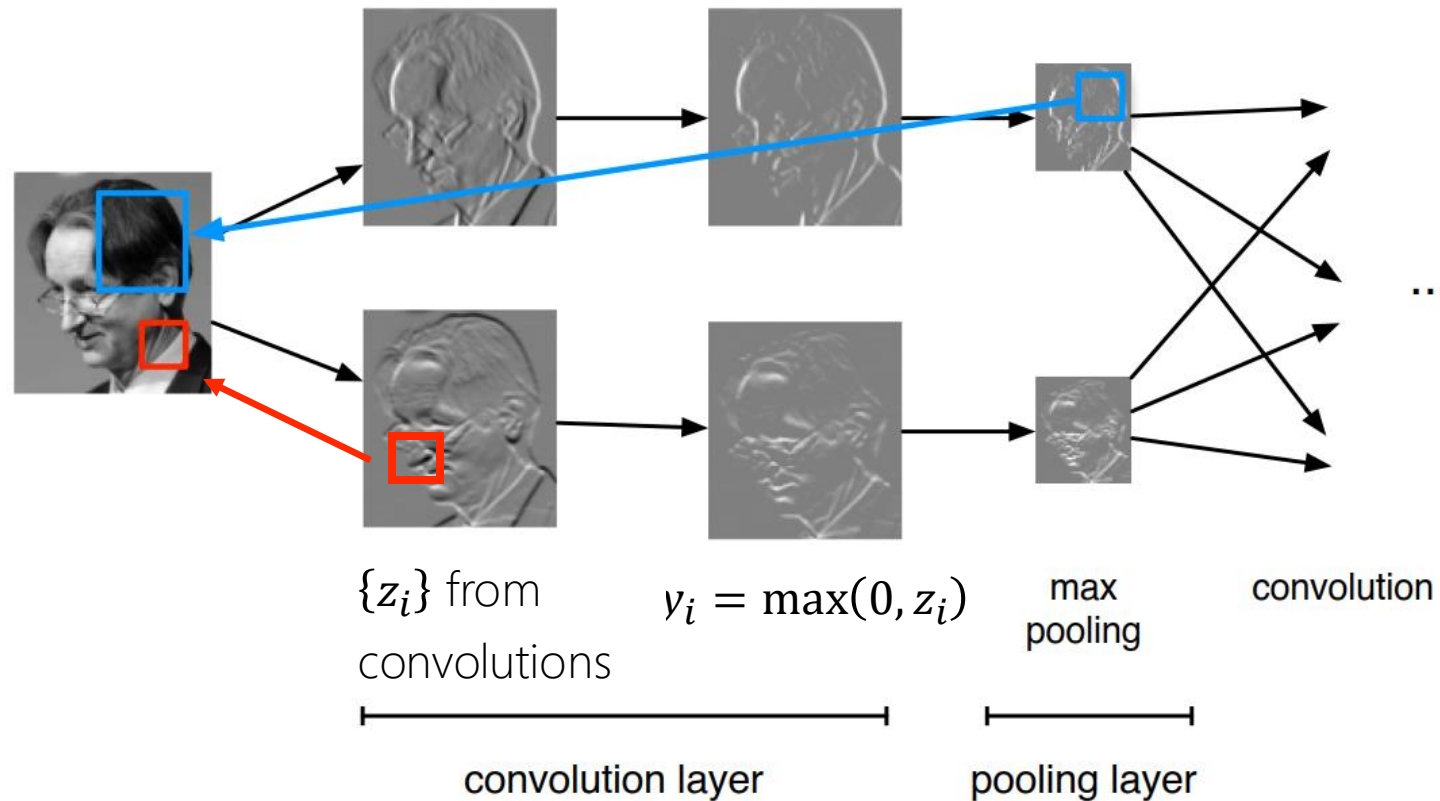


ConvNets: (conv + ReLU) + pooling



ConvNets: (conv + ReLU) + pooling

Because of pooling, higher-layer filters can cover a larger region of the input than equal-sized filters in the lower layers.



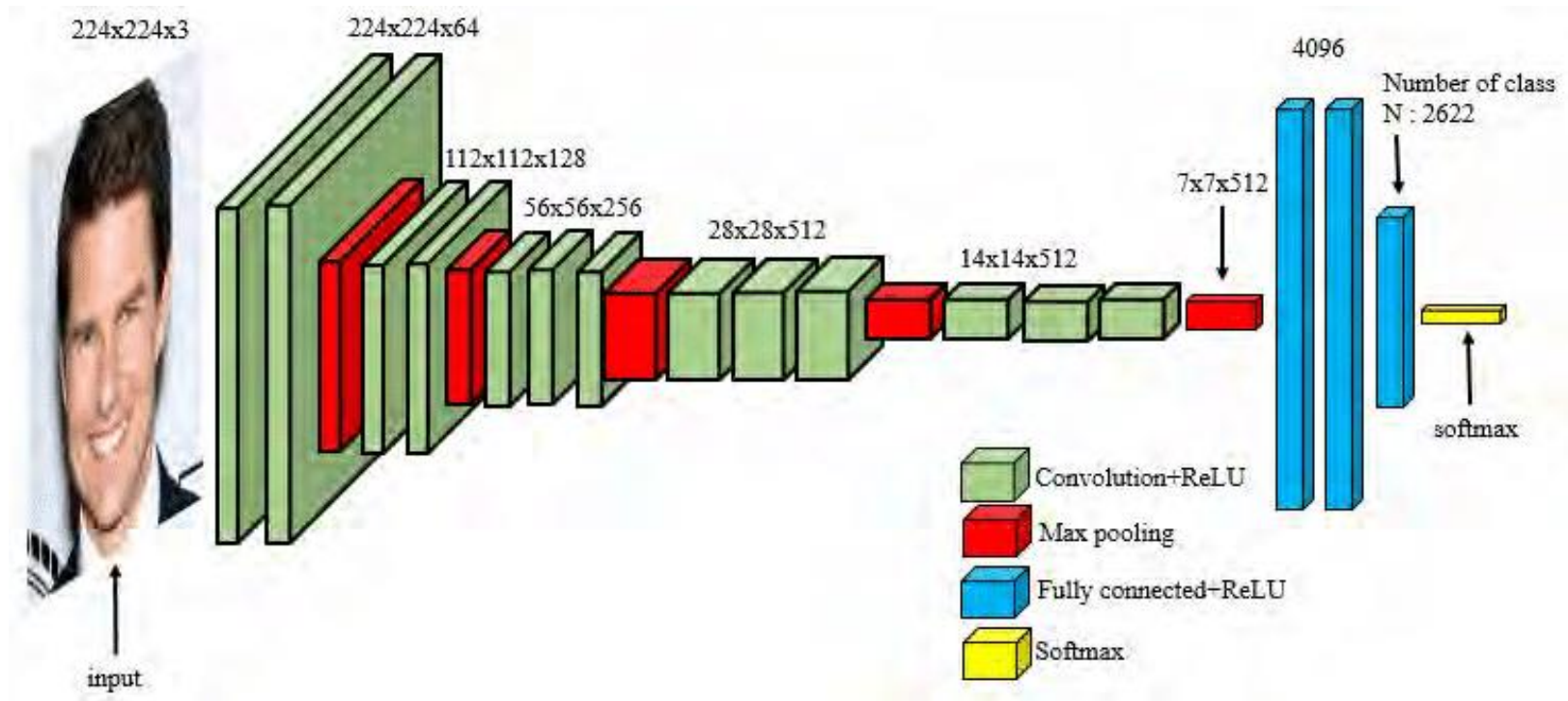
ConvNets: training

Gradient descent via backprop, as before:

1. Shared weights (i.e., filters) mean gradients are accumulated across all positions where the filter was applied.
2. Max-pooling: gradient only gets back-propagated through the neuron that “won” the max pool (technically, a “sub-gradient”).

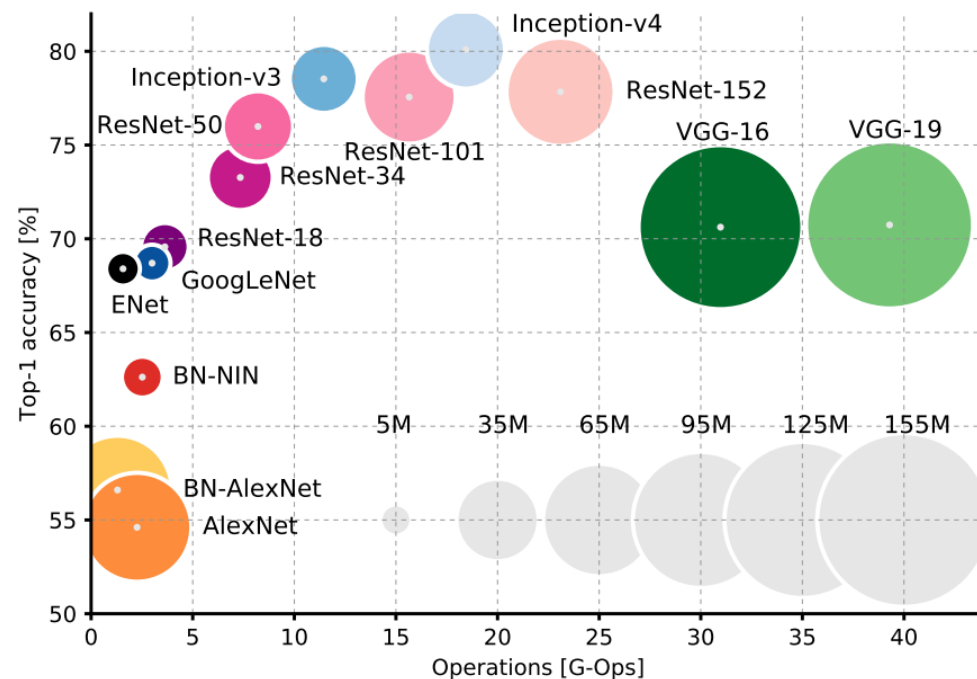
What architecture should we use?

- As throughout the course, use cross-validation to figure out a good architecture (depth, width, type of layer, etc.).
- A well-known network architecture, "VGG" to give idea of some #s:



What architecture should we use?

- They all depend on the fundamentals we have learned so far: layers, back-propagation for gradient descent, convolution, pooling, etc.
- Sometimes innovations like “ResNets” change the game (we may get to these).



Size of the blob is proportional to the number of parameters (legend in gray).

What architecture should we use?

A common thing to do that can work quite well is to:

- i. take a particular architecture that has been developed and fully-trained on some data set
- ii. “Freeze”/fix all the “feature extraction” layer params; re-train only on the top layer(s), which actually do the regression/classification.

